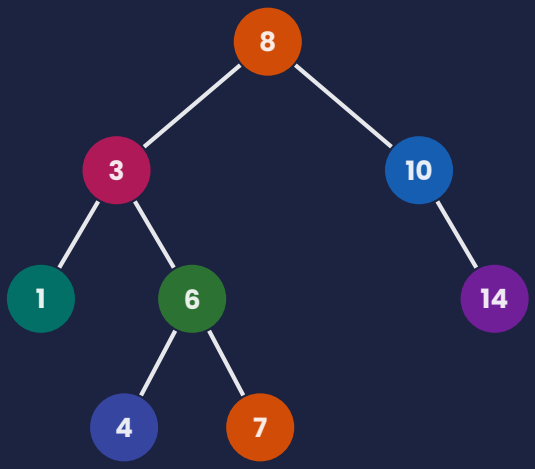
GATE CSE



Data Structures & Programming

The Complete Beginner-to-Exam Guide

From absolute zero to exam-ready: C tracing • pointers • recursion • asymptotics • linked lists • stacks & queues • trees • BST & AVL • heaps • hashing • revision sheet • mock test with full solutions

Contents

Read This First 1

Read this first 1

Where the marks come from (the weight map) 2

Study order and time plan 4

2 Start Here: How a Computer Stores Data 5

2.1 Memory is a long row of numbered boxes 5

2.2 What is a data structure? 6

2.3 Arrays: items side by side 6

2.4 Structures: bundling different things together 6

2.5 Pointers: a variable that stores an address 7

2.6 Functions and the call stack 7

2.7 Stack memory vs heap memory (asked in GATE 2026) 7

2.8 What is an algorithm, and what is complexity? 8

2.9 Quick self-test (answers below) 8

3 C Output Tracing, Loops & Control Flow 9

3.1 The one method that always works: the trace table 9

3.2 Operator facts you must know cold 10

3.3 Overflow and wrap-around (favourite NAT trap) 11

3.4 Increment and decrement 11

3.5 Logical operators and short-circuit evaluation 12

3.6 Bitwise operators (and the tricks that GATE loves) 12

3.7 Loops and counting iterations 13

3.8 Scope, `static` and global variables 14

3.9 `switch` and fall-through 15

3.10 Macros are text replacement (not functions) 15

3.11 Floating-point traps 15

3.12 `printf` essentials 15

3.13 Practice set — C tracing (GATE style) 16

4 Pointers, Arrays, Strings & Structures . . . 19

4.1 Pointer basics . . . 19

4.2 Pointer arithmetic — the single most important rule . . . 20

4.3 The four ++ and * combinations (memorise) . . . 20

4.4 Pointers and functions . . . 21

4.5 Arrays in memory — and address calculation (from your notes) . . . 21

4.6 Strings . . . 23

4.7 Structures and `sizeof` with padding . . . 24

4.8 Practice set — pointers, arrays, strings, structures . . . 24

5 Recursion & Function Calls . . . 27

5.1 What recursion is . . . 27

5.2 The method: draw the call tree . . . 28

5.3 Return-value recursion: patterns you must recognise . . . 29

5.4 `static` variables inside recursion . . . 30

5.5 Mutual recursion . . . 30

5.6 Passing parameters . . . 31

5.7 Practice set — recursion and function calls . . . 31

6 Asymptotic Analysis, Loop Complexity & Recurrences . . . 33

6.1 Why do we need it? (from scratch) . . . 33

6.2 Big-O (upper bound) . . . 33

6.3 Big-Omega (lower bound) . . . 34

6.4 Big-Theta (tight bound) . . . 34

6.5 Little-o and little-ω . . . 35

6.6 Properties (all used in "which statement is true" questions) . . . 36

6.7 Growth-rate ladder (learn this order) . . . 36

6.8 How to compare two functions: the techniques . . . 36

6.9 Worked examples from the lecture notes (all of them) . . . 38

6.10 Analysing loop complexity (all examples from the notes) . . . 40

6.11 Recurrences and the Master theorem (added — the most-asked style: 8 questions in 2018–2026) 42

6.12 Best case, average case, worst case . . . 44

6.13 Practice set — asymptotics (GATE style) . . . 44

7 Linked Lists . . . 47

7.1 From scratch: what is a linked list? . . . 47

7.2 Operations and their cost (from the notes) . . . 48

7.3 Circular linked list . . . 49

7.4 Doubly linked list . . . 50

7.5 Linked list vs array . . . 51

7.6 The code patterns GATE asks (added) . . . 51

7.7 Practice set — linked lists . . . 53

8 Stacks, Queues & Expression Evaluation . . . 55

8.1 Stack — Last In, First Out (LIFO) . . . 55

8.2 Stack permutations (very important) . . . 57

8.3 Queue — First In, First Out (FIFO) . . . 58

8.4 Queue using 2 stacks (a GATE favourite) . . . 60

8.5 Stack using 2 queues . . . 61

8.6 Checking balanced brackets with a stack . . . 61

8.7 Two stacks in one array . . . 62

8.8 Infix, prefix and postfix notation . . . 63

8.9 Practice set — stacks, queues & expressions (GATE style) . . . 65

9 Binary Trees & Traversals . . . 67

9.1 From scratch: what is a tree? . . . 67

9.2 Special kinds of binary trees . . . 68

9.3 Formulas for counting nodes (the heart of the NAT questions) . . . 69

9.4 Representing a binary tree in memory . . . 71

9.5 Counting binary trees . . . 72

9.6 Tree traversals . . . 73

9.7 Constructing a tree from traversals . . . 74

9.8 Practice set — binary trees (GATE style) . . . 77

10 Binary Search Trees (BST) . . . 79

10.1 From scratch: why a BST? . . . 80

10.2 Search and insertion . . . 80

10.3 Deletion — three cases (notes) . . . 83

10.4 Range search (notes) . . . 84

10.5 The “search sequence” trick (notes) — important for MSQ/NAT . . . 84
10.6 Other BST facts commonly tested . . . 85
10.7 Practice set — BST (GATE style) . . . 85

11 AVL Trees (Self-Balancing BST) . . . 87

11.1 Why AVL? (from scratch) . . . 88
11.2 Rotations — the repair tools . . . 88
11.3 Insertion . . . 91
11.4 Deletion . . . 93
11.5 How many nodes can an AVL tree of height h have? . . . 96
11.6 Quick facts for MCQ/MSQ . . . 97
11.7 Practice set — AVL (GATE style) . . . 98

12 Heaps & Priority Queues . . . 100

12.1 From scratch: what is a heap? . . . 100
12.2 Where can the smallest and the largest elements be? (notes) . . . 102
12.3 The k-th minimum in a min-heap (notes) . . . 103
12.4 Insertion in a heap (notes) . . . 104
12.5 Heapify (notes) . . . 106
12.6 Build Heap — the bottom-up approach (Floyd’s algorithm) . . . 106
12.7 Find-Min and Delete-Min (min-heap) . . . 108
12.8 Counting heaps (a favourite NAT) . . . 108
12.9 Probability that a structure is a heap (notes) — and the shortcut . . . 110
12.10 Heap sort (notes) . . . 111
12.11 Priority queue (notes) . . . 111
12.12 Practice set — heaps (GATE style) . . . 112

13 Hashing . . . 114

13.1 From scratch: what problem does hashing solve? . . . 114
13.2 Hash functions . . . 115
13.3 Collision resolution — the two families (notes) . . . 115
13.4 Open chaining (separate chaining) . . . 116
13.5 Open addressing — probing . . . 117
13.6 Linear probing (notes) . . . 118
13.7 Quadratic probing (notes) . . . 118

13.8 Double hashing (notes) . . . 119
13.9 Number of possible probe sequences (notes) . . . 119
13.10 Analysis of open addressing (assumption: uniform hashing) — notes . . . 120
13.11 More facts GATE uses (added) . . . 121
13.12 Worked layout examples (verified by simulation) . . . 121
13.13 Practice set — hashing (GATE style) . . . 122

14 Master Revision Sheet . . . 124

14.1 C tracing (T1 · 2.11 marks) — traps checklist . . . 124
14.2 Pointers, arrays, structs (T1 · 1.28) . . . 125
14.3 Recursion . . . 125
14.4 Asymptotics (T1 · 1.61) . . . 125
14.5 Linked lists . . . 126
14.6 Stacks & queues (T2 · 0.94) . . . 126
14.7 Binary trees (T2 · 0.83) . . . 126
14.8 BST & AVL (T3 · 0.83) . . . 126
14.9 Heaps (T2 · 1.22) . . . 127
14.10 Hashing (T2 · 0.67) . . . 127
14.11 Last-hour exam strategy . . . 127

15 Mixed Mock Test (28 questions) with Full Solutions . . . 128

Section A — Questions . . . 129
Section B — Answers with solutions . . . 131

START HERE

Read This First

KEY POINT

A replacement for the long lectures. Start from zero, learn every concept in plain English, see every worked example from the teacher's lecture notes, learn the shortcuts, and then practise GATE-style questions with full solutions.

Read this first

0.1 What this book is built from

Source	How it was used
Handwritten lecture notes (105 pages)	Every topic, every worked example and every shortcut in them is included, re-explained in simpler language. Where the notes jump a step, the step is filled in.
Your GATE PYQ analysis (1987–2026)	Used to decide *how much space* each topic gets. High-frequency topics get extra depth, extra examples and extra practice questions. Low-yield topics are kept short.
Added by me (not in the notes)	Three chapters that the PYQ analysis says carry the **most marks** but the lecture notes do not cover: **C output tracing, pointers/arrays/structures, and recursion**. Without them you would miss the single biggest marks cluster in this subject.

KEY POINT

A note on the subject name. Your message mentioned "computer networks" once, but the lecture notes and the weight analysis you gave are for **Data Structures (Programming & Data Structures)**. This book is therefore a *Data Structures* book. If you really wanted a Computer Networks book, tell me and I will build it from the same analysis (it has a Computer Networks section).

0.2 About the practice questions

The analysis lists **question styles and the years they appeared** (for example "counting heaps, array positions of min/max, levels — asked 2018, 2020, 2024, 2025"). It does not reproduce the full question texts. So the practice questions in this book are **original, GATE-style questions written to match those styles**. They are *not* copies of past papers. After finishing each chapter, go and solve the real previous-year questions of that topic yourself — that is the plan you described, and this book prepares you for it.

Every numeric answer in this book that involves computation (AVL rotations, heap building, heap counting, C program outputs, hash layouts) was **checked by actually running it**, not just worked by hand.

0.3 How each chapter is organised

1. **Exam weight box** — marks per paper, how often it appears.
2. **From scratch** — the idea in simple words.
3. **Core facts & formulas** — what to memorise.
4. **Worked examples from the lecture notes** — all of them.
5. **Shortcuts & traps** — what teachers teach to save time.
6. **GATE-style practice** with solutions.

Where the marks come from (the weight map)

Programming & Data Structures averages 8.8 marks per paper (range 6–12; last five papers average 8.3; GATE 2026 had 6). Three clusters alone give about half of those marks.

Rank	Topic cluster	Tier	Avg marks / paper	Years asked (of last 9)	Asked in 2024, 2025, 2026?
1	**C output tracing, loops & control flow**	T1	**2.11**	8 / 9	✓ ✓ ✓
2	**Pointers, arrays, strings & structures**	T1	**1.28**	8 / 9	✓ ✓ ✓
3	**Binary heap & priority queue**	T2	**1.22**	7 / 9	✓ ✓ ·
4	**Stacks, queues & expression evaluation**	T2	**0.94**	7 / 9	✓ ✓ ✓
5	Recursion & function calls	T3	0.89	5 / 9	✓ ✓ ✓
6	**Binary trees & traversals**	T2	0.83	6 / 9	· ✓ ✓
7	**BST & AVL trees**	T3	0.83	5 / 9	✓ ✓ ·
8	Linked lists	T3	0.56	5 / 9	· ✓ ✓
9	Arrays, hashing & general DS	T4	0.17	2 / 9	· ✓ ·

Two topics from your lecture notes are filed under **Algorithms** in the analysis, but they are taught in this book because the notes teach them here:

Topic	Tier	Avg marks / paper	Years asked (of 9)
Asymptotic notation, time complexity & recurrences	**T1**	**1.61** (trend ↑)	8 / 9
Hashing (open addressing, probing)	T2	0.67	6 / 9

NOTE

T1 = asked in ≥ 8 of last 9 papers (treat as certain); T2 = 6–7 of 9; T3 = 4–5 of 9 (lumpy but big when it comes); T4 = ≤ 3 of 9. Tiers are odds, not guarantees — GATE has no fixed quota.

Year-by-year marks (Programming & Data Structures part)

Cluster	18	19	20	21	22	23	24	25	26
C output tracing	2	**7**	–	1	2	1	2.5	2.5	1
Pointers / arrays / strings / structs	1	1	1	1.5	3	–	1	2	1
Binary heap & PQ	2	2	2	0.5	–	3	1	0.5	–
Stacks, queues, expressions	1	–	–	0.5	2	2	1	1	1
Recursion & functions	3	–	2	–	–	–	0.5	1	1.5
Binary trees & traversals	1	2	–	1.5	–	2	–	0.5	0.5
BST & AVL	–	–	**4**	0.5	1	–	1	1	–
Linked lists	–	–	1	–	1	1	–	1	1
Arrays, hashing, general DS	–	–	–	0.5	–	–	–	1	–
Subject total	10	12	10	6	9	9	7	10.5	6

What the examiner actually asks (2018–2026)

These are the **question styles** this book trains you for. Each chapter repeats the relevant ones.

- **C tracing:** trace a program with loops, `static` variables, bit and precision effects, and give the printed value — 16 questions, roughly half NAT and the rest MCQ/MSQ. Appeared in 10 of 13 papers/sets in 2018–2026.
- **Pointers:** pointer arithmetic / aliasing, "what value is printed" (7 questions); 2-D array address / row-major offset (2); structure `sizeof` / padding (2); string-function behaviour (1).
- **Heaps:** *counting heaps*, array positions of min/max, number of levels (4 questions: 3 NAT, 1 MCQ); *heap operations* — is this array a heap, insert/delete cost, priority-queue implementation (4 questions).
- **Stacks & queues:** operation-sequence puzzles — push/pop/enqueue order, capacity, possible outputs (6 questions, 3 NAT and 3 MSQ, 2-markers); implementing a queue with stacks or vice-versa.
- **Recursion:** trace a recursive C function (4 NAT); parameter passing and evaluation order (2); stack vs heap memory (1).
- **Binary trees:** node/leaf/height properties of full & complete trees (4: 2 NAT, 1 MCQ, 1 MSQ); traversal-based reconstruction or tracing (3).

- **BST/AVL:** build a BST from an insertion or traversal order and ask for height / post-order / shape (4); time complexity and properties of BST & AVL operations (4).
- **Linked lists:** code — reversal, deletion, counting nodes, insertion complexity (5 questions).
- **Asymptotics:** solve a recurrence (8 questions), order/compare growth rates (4), time complexity of given code (4).
- **Hashing:** insert into a hash table and give the probe sequence / final layout for linear, double hashing or chaining (5); simple-uniform hashing expected collisions (2).

Study order and time plan

If you have 4 weeks (about 2 hours a day):

Week	Chapters	Why this order
1	Ch 2 Foundations → Ch 3 C tracing → Ch 4 Pointers & arrays → Ch 5 Recursion	Highest marks (≈ 4.3 marks/paper combined) and they make later chapters easy
2	Ch 6 Asymptotics → Ch 7 Linked lists → Ch 8 Stacks & queues	Asymptotics is T1; stacks/queues is T2
3	Ch 9 Binary trees → Ch 10 BST → Ch 11 AVL → Ch 12 Heaps	Trees all build on each other
4	Ch 13 Hashing → Ch 14 Cheat sheet → Ch 15 Mock test → solve real PYQs	Revision and testing

If you have only 10 days: Ch 3, 4, 12, 8, 6, 9–11, then 5, 7, 13. Skip nothing in the first four.

Golden rules for this subject

1. **C tracing questions are marks you can always get** — they need patience, not genius. Practise tracing on paper.
2. **Counting questions** (number of heaps, number of BSTs, valid stack outputs) use **a few formulas**. Memorise them (Ch 14).
3. **Never memorise rotation pictures**. Learn the *rule* (find the first unbalanced node, look at the heavy side's child).
4. In MSQ questions, check **every** option on its own — one wrong inclusion or one missed option makes the whole answer wrong. (Check the current GATE brochure for the exact MSQ marking rule.)

CHAPTER 2

Start Here: How a Computer Stores Data

from absolute zero

EXAM WEIGHT

Exam weight: no direct questions, but *every* later chapter depends on it. Read it once, slowly. About 20 minutes.

2.1 Memory is a long row of numbered boxes

Imagine a street with houses numbered 0, 1, 2, 3, … Each house holds **one byte** (a small number from 0 to 255). The house number is its **address**.

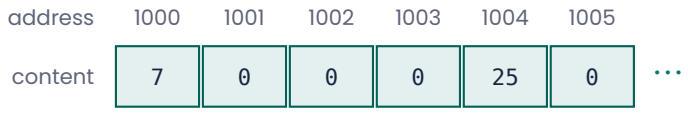


Figure 2.1 Memory as a row of numbered byte boxes (addresses 1000, 1001, …).

A **variable** is just a *name* for one or more consecutive houses.

```c
int x = 7;
```

An `int` normally needs **4 bytes**, so `x` occupies four houses, say 1000–1003. The **address of x** is 1000 (the first byte). In C you write `&x` to get that address.

Typical sizes (GATE usually states them; if not, assume these):

Type	Size in bytes
`char`	1
`short`	2
`int`	4
`long` (64-bit Linux)	8
`float`	4
`double`	8

Type	Size in bytes
any pointer (64-bit machine)	8 (32-bit machine: 4)

2.2 What is a data structure?

A **data structure** is a *way of arranging data in memory* so that some operations become fast.

- Want fast "look at the i-th item"? → **array** (items side by side).
- Want fast "insert at the front"? → **linked list** (items scattered, joined by arrows).
- Want "last in, first out"? → **stack**.
- Want "first in, first out"? → **queue**.
- Want "smallest first"? → **heap**.
- Want "find a key almost instantly"? → **hash table**.
- Want "sorted data with quick search and insert"? → **binary search tree / AVL tree**.

Each structure is judged by **how many steps** each operation takes. That count is its **time complexity** (Chapter 6).

2.3 Arrays: items side by side

```c
int a[5] = {10, 20, 30, 40, 50};
```

If `a` starts at address 1000 and each `int` is 4 bytes:

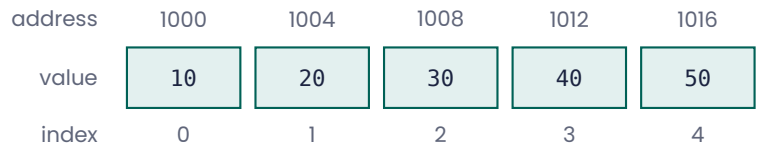


Figure 2.2 An `int` array stored contiguously: each element starts 4 bytes after the previous one.

Address of `a[i]` = `1000 + i × 4`. The computer jumps straight there — that is why array access is **O(1)**.

2.4 Structures: bundling different things together

```c
struct Student {
    int  roll;
    char grade;
    float marks;
};
```

A `struct` groups different variables under one name. Fields sit one after another in memory (with some padding — see Chapter 4).

2.5 Pointers: a variable that stores an address

A **pointer** is a variable whose *value is an address*.

```c
int x = 7;
int *p = &x;     // p holds the address of x
printf("%d", *p);// *p means "go to that address and read": prints 7
*p = 99;         // go to that address and write: x is now 99
```

Think of `p` as a **slip of paper with a house number written on it**. `*p` means "visit the house written on the slip".

- `&x` → address of x
- `*p` → the thing at the address stored in p

(Full detail with tracing questions in Chapter 4.)

2.6 Functions and the call stack

When a function is called, the computer creates a **box of memory for it** containing its **local variables**, **parameters** and the **return address**. This box is called an **activation record** (or *stack frame*).

- Calling a function → a new frame is **pushed** on top of the **call stack**.
- Returning → that frame is **popped** and **destroyed**; its local variables disappear.

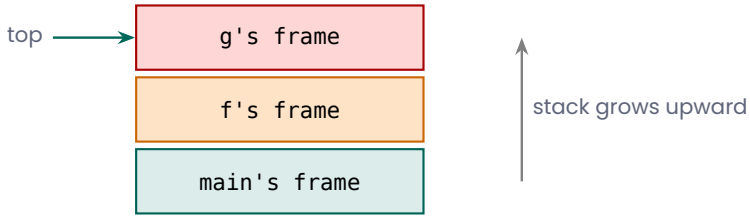


Figure 2.3 The call stack while `main()` calls `f()`, and `f()` calls `g()`.

This is why recursion works (every call gets its own frame) and why a recursion without an end crashes (**stack overflow**).

2.7 Stack memory vs heap memory (asked in GATE 2026)

	Stack	Heap
What lives here	local variables, parameters, return addresses	memory you ask for with `malloc`
Who frees it	automatic, when the function returns	**you**, with `free` (else a *memory leak*)
Lifetime	until the function returns	until you free it
Size	small, fixed	large, flexible

Why linked lists use the heap (from the lecture notes): if a node were created as an ordinary local variable, it would be allocated on the stack and **destroyed when that function's acti-**

vation record is deleted. We want nodes to survive after the function returns, so we allocate them on the heap:

```c
struct Node *p = malloc(sizeof(struct Node));
```

Also remember:

- **Global and `static` variables** live in a separate *static/data area*; they exist for the whole program.
- `malloc` returns an address; you store it in a pointer.

2.8 What is an algorithm, and what is complexity?

An **algorithm** is a step-by-step method. Its **time complexity** tells you *how the number of steps grows when the input size n grows*. We do not measure seconds (computers differ); we count steps and ignore small details:

NOTE

$3n^2 + 4$ behaves just like n^2 when n is large.

That idea — "ignore small details, keep the growth" — is **asymptotic analysis**, the subject of Chapter 6.

2.9 Quick self-test (answers below)

1. How many bytes does `int a[10]` occupy if `int` is 4 bytes?
2. If `a` starts at address 2000, what is the address of `a[3]`?
3. `int x=5; int *p=&x; *p=8;` — what is `x` now?
4. Where is a local variable stored — stack or heap?
5. Where is a `malloc`'d node stored?

ANSWER **s:** 1) 40. 2) 2000 + 3×4 = 2012. 3) 8. 4) Stack. 5) Heap.

CHAPTER 3

C Output Tracing, Loops & Control Flow

TIER 1

EXAM WEIGHT

Exam weight: 2.11 marks per paper — the single biggest cluster. Asked in 8 of the last 9 papers, and in 2024, 2025 and 2026. In 2019 this one cluster gave **7 marks**. **Style:** "What is printed / what is the value of the variable?" — a roughly even split of NAT (type the number) and MCQ/MSQ (16 questions in 2018–2026). Involves loops, `static` variables, bit tricks, integer overflow, floating-point precision.
Good news: you do not need clever ideas. You need a **method** and a list of **traps**. Both are below.

3.1 The one method that always works: the trace table

Never "feel" the answer. **Write a table** with one column per variable and one row per step.

```c
int main() {
    int i, s = 0;
    for (i = 1; i <= 20; i++) {
        if (i % 3 == 0) continue;
        if (i % 7 == 0) break;
        s += i;
    }
    printf("%d", s);
}
```

i	i%3==0?	i%7==0?	s after
1	no	no	1
2	no	no	3
3	yes → `continue`	–	3
4	no	no	7
5	no	no	12
6	yes → `continue`	–	12
7	no	yes → `break`	12

Output: 12.

`continue` = skip the rest of *this* round, go to the next. `break` = leave the loop completely.

3.2 Operator facts you must know cold

Precedence (high → low) — enough for GATE

Level	Operators	Associativity
1	`() [] -> .` postfix `++ --`	left → right
2	prefix `++ --`, unary `+ - ! ~ * &`, `sizeof`, casts	**right → left**
3	`* / %`	left → right
4	`+ -`	left → right
5	`<< >>`	left → right
6	`< <= > >=`	left → right
7	`== !=`	left → right
8	`&`	left → right
9	`^`	left → right
10	`|`	left → right
11	`&&`	left → right
12	`||`	left → right
13	`?:`	right → left
14	`= += -= …`	**right → left**
15	`,`	left → right

Memory aid for bitwise vs comparison: `==` binds *tighter* than `&`, so `a & b == c` means `a & (b == c)` — a classic trap.

Integer arithmetic

- **Integer division truncates toward zero:** `7/2 = 3`, `-7/2 = -3`.
- **`%` takes the sign of the left operand:** `-7 % 3 = -1`, `7 % -3 = 1`.
- If *either* operand is `float`/`double`, the division is real: `7/2.0 = 3.5`.
- `float r = 7/2;` → the **integer** division happens first (3), *then* converts: `r = 3.0`.
- `10/3*3 = 9` (left to right: 3 × 3), but `10*3/3 = 10`.
- Casting `(int)3.99 = 3` and `(int)-3.99 = -3` (truncate, not round).

Character arithmetic

`'A'+1` is the number 66; printed with `%c` it shows `B`. `'a' - 'A' = 32`.

3.3 Overflow and wrap-around (favourite NAT trap)

Type	What happens on overflow
`unsigned` types	**Wraps around** modulo 2^n (well-defined)
`char` / `signed`	Practically wraps in two's complement (GATE questions assume this)

Worked checks (all run and verified):

Code	Result	Why
`unsigned char c=255; c++;`	**0**	256 mod 256
`unsigned char uc=200; uc=uc+100;`	**44**	300 mod 256
`char ch=120; ch+=10;`	**−126**	130 does not fit in −128..127; 130 − 256 = −126
`char ch=127; ch++;`	**−128**	wraps
`unsigned u=0; u--;`	**4294967295**	$2^{32} - 1$
`int big=2147483647; unsigned ub=big; ub++;`	**2147483648**	as unsigned there is room

Shortcut for a k-bit unsigned type: answer = (value) **mod** 2^k.
Shortcut for signed k-bit: take mod 2^k, and if the result $\geq 2^{k-1}$ subtract 2^k.

Signed vs unsigned comparison trap

When a signed and an unsigned `int` are compared, the signed one is converted to unsigned:

```c
unsigned int ui = 3; int si = -5;
printf("%d", ui > si);   // prints 0 ! because -5 becomes 4294967291
```

3.4 Increment and decrement

- **Postfix** `i++`: use the old value, then increase.
- **Prefix** `++i`: increase first, then use.

```c
int i = 5;
int t = i++ + 10;    // t = 5 + 10 = 15, then i = 6
```

WARNING

expressions like `i = i++ + ++i` or `printf("%d %d", i++, i)` modify and read the same variable without a sequence point → **undefined behaviour**. GATE sometimes states "assume left to right" or avoids this. If a question depends on it, the question will say how to evaluate. Do not worry about such cases otherwise.

3.5 Logical operators and short-circuit evaluation

- `A && B`: if A is **false**, B is **never evaluated**.
- `A || B`: if A is **true**, B is **never evaluated**.
- Non-zero = true, zero = false. Result of a relational/logical expression is **1 or 0**.

```c
int a = 0, b = 0;
if (a++ && b++) { }
// a++ gives 0 (false) → b++ is skipped.  Final: a=1, b=0

a = 1; b = 1;
if (a++ || b++) { }
// a++ gives 1 (true) → b++ is skipped.   Final: a=2, b=1
```

Chained comparisons don't mean what maths means

`1 < 2 < 3` → `(1<2) < 3` → `1 < 3` → **1**.
`3 > 2 > 1` → `(3>2) > 1` → `1 > 1` → **0**.

Assignment inside `if`

```c
int k = 5;
if (k = 0) printf("T"); else printf("F k=%d", k);   // prints: F k=0
```

`k = 0` is an **assignment** (value 0 = false), not a comparison.

A stray semicolon

```c
for (i = 0; i < 3; i++);      // the loop body is the empty statement ;
printf("%d", i);              // prints 3
```

3.6 Bitwise operators (and the tricks that GATE loves)

Op	Meaning	Example (8-bit view)
&	AND	`5 & 3 = 0101 & 0011 = 0001 = 1`
|	OR	`5 | 3 = 0111 = 7`
^	XOR	`5 ^ 3 = 0110 = 6`
~	NOT (flip all bits)	`~5 = -6, ~0 = -1`
<<	shift left = $\times 2^n$	`1 << 10 = 1024`
>>	shift right = $\div 2^n$ (floor)	`-16 >> 2 = -4` (arithmetic shift)

The four tricks

1. **`x & (x-1)` clears the lowest set bit.**
   Repeating it until `x` becomes 0 **counts the 1-bits** (popcount). Number of loop rounds = number of 1s.

```c
unsigned x = 0xB6;        // 1011 0110  → five 1s
int cnt = 0;
while (x) { cnt++; x &= x - 1; }
// cnt = 5
```

   `cnt(255) = 8`, `cnt(1024) = 1`, `cnt(0xF0F0) = 8`.
2. **`x & -x` isolates the lowest set bit.** `12 & -12 = 4`.
3. **Power-of-two test:** `x > 0 && (x & (x-1)) == 0`.
4. **XOR facts:** `x ^ x = 0`, `x ^ 0 = x`, and the swap

```c
a ^= b;  b ^= a;  a ^= b;      // swaps a and b without a temp
```

Quick sums: `(a & b) + (a | b) = a + b` always. And `a ^ b = (a | b) − (a & b)`. Check: `(5&3)+(5|3)+(5^3) = 1+7+6 = 14`.

3.7 Loops and counting iterations

Plain counting

```c
for (i = 0; i < 5; i++)
    for (j = i; j < 5; j++)
        cnt++;
```

Inner runs 5, 4, 3, 2, 1 times → **15** $= 5 \cdot 6/2$.

Doubling loop (log pattern)

```c
n = 64;
for (i = 1; i <= n; i *= 2)
    for (j = 0; j < i; j++)
        cnt++;
```

i = 1, 2, 4, …, 64 → cnt = 1+2+4+…+64 = **127** $(= 2 \cdot 64 - 1)$.

Tripling loop

```c
h = 1; sum = 0;
while (h < 1000) { sum += h; h *= 3; }
// h: 1,3,9,27,81,243,729 → sum = 1093, loop ends with h = 2187
```

Post-decrement in `while`

```c
m = 5; t = 0;
while (m--) t += m;
// test uses old m, then decrements. Body sees m = 4,3,2,1,0 → t = 10.
// After the loop ends the test m-- (m was 0) leaves m = -1.
```

SHORTCUT

`while(m--)` runs **m times**, and afterwards `m = -1`.

do-while

Body runs **at least once**, test at the end.

```c
k = 0;
do { k += 3; if (k == 6) continue; k++; } while (k < 12);
```

Trace: k=3→ 4; test 4<12; k=7→ 8; k=11→ 12; test 12<12 false. **k = 12.** (`continue` in a do-while jumps to the *test*, not the top.)

3.8 Scope, `static` and global variables

- A **local** variable is created at each call and destroyed at return.
- A **static local** variable is created **once**, keeps its value between calls, and is initialised **only once**.
- A **global** variable lives for the whole program.
- **Inner scope hides outer scope** (shadowing); the outer variable is unchanged.

```c
int f(int n) {
    static int c = 0;     // initialised only on the first call
    c += n;
    return c;
}
int main() {
    int a = f(1);   // c = 1
    int b = f(2);   // c = 3
    int c = f(3);   // c = 6
    printf("%d", a + b + c);   // 1 + 3 + 6 = 10
}
```

```c
void sh() {
    int x = 20;
    { int x = 30; x++; }   // inner x (30→31) vanishes at the }
    x++;                   // outer x: 20→21
```

```
    printf("%d", x);        // 21
}
```

3.9 switch and fall-through

Without `break`, execution **falls through** into the next `case`.

```c
int k = 3;
switch (k) {
    case 1: printf("one ");
    case 3: printf("three ");     // starts here
    case 4: printf("four ");      // falls through
            break;
    default: printf("def ");
}
// prints: three four
```

3.10 Macros are text replacement (not functions)

```c
#define SQ(x)  x*x
SQ(2+3)         // becomes 2+3*2+3 = 11   (not 25!)
#define SQ2(x) ((x)*(x))
SQ2(2+3)        // ((2+3)*(2+3)) = 25
100/SQ(5)       // 100/5*5 = 100          (not 4!)
#define M(x,y) x*y+x
M(1+2,3)        // 1+2*3+1+2 = 10
```

Rule: substitute the text literally, *then* apply normal precedence.

3.11 Floating-point traps

- `0.1 + 0.2 == 0.3` is **false** (0.30000000000000004).
- Adding `0.1f` ten times in `float` gives `1.0000001`, so `sum == 1.0f` is **false**.
- A `float` has about 7 significant decimal digits, a `double` about 15–16.
- `float r = 7/2;` gives 3.0 (integer division happened first).
- Comparing floats with `==` almost always fails in GATE "gotcha" questions.

3.12 printf essentials

Format	Prints
`%d`	signed int
`%u`	unsigned int
`%c`	character
`%f`	float/double
`%x` / `%o`	hexadecimal / octal (`255 → ff`, `8 → 10`)

Format	Prints
`%s`	string

- `printf` **returns the number of characters printed.** `printf("%d", printf("ab"));` prints `ab2` (the inner prints `ab`, returns 2, outer prints 2).
- A literal starting with `0` is **octal**: `07+1 = 8`. `0x` is hex.
- `sizeof('a')` is 4 in C (a character constant has type `int`), but `sizeof(char)` is 1.

3.13 Practice set — C tracing (GATE style)

VERIFIED

All answers below were produced by compiling and running the code.

Q1 NAT · 1 MARK What is printed?

```c
unsigned x = 0xB6; int cnt = 0;
while (x) { cnt++; x &= x - 1; }
printf("%d", cnt);
```

ANSWER **5.** `0xB6 = 1011 0110` has five 1-bits; each round removes one.

Q2 NAT · 1 MARK

```c
int f(int n){ static int c = 0; c += n; return c; }
int main(){ int a=f(1), b=f(2), c=f(3); printf("%d", a+b+c); }
```

ANSWER **10.** c becomes 1, 3, 6.

Q3 NAT · 1 MARK

```c
int n = 64, cnt = 0, i, j;
for (i = 1; i <= n; i *= 2)
    for (j = 0; j < i; j++) cnt++;
printf("%d", cnt);
```

ANSWER **127.**

Q4 NAT · 1 MARK

```c
char ch = 120; ch += 10; printf("%d", ch);
```

ANSWER **−126.**

Q5 NAT · 2 MARKS

```c
int p = 0, q = 5;
int r = (p++ || q--) && (++p);
printf("%d %d %d", r, p, q);
```

ANSWER **1 2 4.** `p++` yields 0 (p becomes 1) → false, so `q--` is evaluated (5, true; q becomes 4). Then `++p` makes p = 2 (non-zero, true). `r = 1`.

Q6 MCQ · 1 MARK What is printed?

```c
#define M(x,y) x*y+x
printf("%d", M(1+2,3));
```

(A) 12 **(B)** 10 **(C)** 9 **(D)** 6

ANSWER **(B) 10.** 1+2*3+1+2.

Q7 NAT · 1 MARK

```c
unsigned char uc = 200; uc = uc + 100; printf("%d", uc);
```

ANSWER **44.**

Q8 MCQ · 1 MARK

```c
float s = 0; int i;
for (i = 0; i < 10; i++) s += 0.1f;
printf("%d", s == 1.0f);
```

(A) 1 **(B)** 0 **(C)** compile error **(D)** infinite loop

ANSWER **(B) 0.** The float sum is 1.0000001.

Q9 NAT · 1 MARK How many times does `cnt++` execute?

```c
for (i = 0; i < 5; i++)
    for (j = i; j < 5; j++) cnt++;
```

ANSWER **15.**

Q10 NAT · 2 MARKS

```c
int m = 5, t = 0;
while (m--) t += m;
printf("%d %d", t, m);
```

ANSWER **10 -1.**

Q11 MCQ · 1 MARK `printf("%d", 3 > 2 > 1);` prints
(A) 1 **(B)** 0 **(C)** 3 **(D)** error
Answer: (B) 0. `(3>2)=1`, then `1>1 = 0`.

Q12 MSQ · 2 MARKS Which statements are true for the code below?

```c
int x = 12;
printf("%d %d %d", x & -x, x & (x-1), x | (x+1));
```

(A) first value is 4 **(B)** second value is 8 **(C)** third value is 13 **(D)** all three are powers of two
ANSWER **A, B, C.** `12 & -12 = 4`, `12 & 11 = 8`, `12 | 13 = 13`; 13 is not a power of two, so D is false.

Q13 NAT · 1 MARK `printf("%d", printf("ab"));` — how many characters appear on the screen in total?
Answer: 3 (`a`, `b`, and the digit `2`).

Q14 NAT · 2 MARKS Value printed?

```c
int h = 1, sum = 0;
while (h < 1000) { sum += h; h *= 3; }
printf("%d %d", sum, h);
```

ANSWER **1093 2187.**

Q15 NAT · 1 MARK `printf("%d", (5 & 3) + (5 | 3) + (5 ^ 3));` **Answer: 14.**

Self-check list before you move on

SELF-CHECK

- ☐ I can build a trace table without hesitation.
- ☐ I know which side of `&&` / `||` can be skipped.
- ☐ I can wrap a value into `char`/`unsigned char` quickly.
- ☐ I know `x & (x-1)` counts 1-bits.
- ☐ I know macros are text substitution.

CHAPTER 4

Pointers, Arrays, Strings & Structures

4

TIER 1

EXAM WEIGHT

Exam weight: 1.28 marks per paper, asked in 8 of the last 9 papers, and in 2024, 2025 and 2026.
Styles asked (2018–2026): pointer arithmetic / aliasing — "what value is printed" (7 questions: 4 NAT, 3 MCQ); **2-D array addressing / row-major offsets** (2); **structure `sizeof` / padding** (2); string-function behaviour (1).
This chapter also contains the **"Arrays" part of your lecture notes** (row-major / column-major addressing and n-dimensional indexing).

4.1 Pointer basics

```c
int x = 7;
int *p = &x;      // p stores the address of x
```

Expression	Meaning
`&x`	address of `x`
`p`	the address stored in `p`
`*p`	the value at that address (same as `x`)
`&p`	the address of the pointer variable itself
`int **pp = &p;`	`pp` points to `p`; `**pp` is `x`

Two names, one box (aliasing): after `int *p = &x;`, changing `*p` changes `x`, and changing `x` changes `*p`. This is the heart of most "what is printed" questions.

```c
int x = 10, *p = &x, **pp = &p;
**pp = *p + 5;      // x = 15
*p   = **pp * 2;    // x = 30
printf("%d", x);    // 30
```

A pointer that points nowhere should hold `NULL` (0). Reading `*NULL` crashes.

4.2 Pointer arithmetic — the single most important rule

KEY POINT

`p + 1` does not add 1 byte. It moves to the next *element*: it adds `sizeof(*p)` bytes.

If `p` is `int*` (4 bytes) and holds 1000: `p+1 = 1004`, `p+3 = 1012`.
If `p` is `char*`: `p+1 = 1001`. If `p` is `double*`: `p+1 = 1008`.

Operation	Result
`p + n`, `p - n`	pointer moved n **elements**
`p2 - p1` (same array)	number of **elements** between them (an integer, not bytes)
`p++`	move to next element
`p1 < p2`	compares addresses
`*(p + n)`	the n-th element after p

C

```c
int a[] = {5, 10, 15, 20, 25};
int *p = a + 1, *q = a + 4;
printf("%ld %d", q - p, *(q-2) + *p);   // 3 25
```

`q−p = 3` elements. `*(q-2) = a[2] = 15`, `*p = a[1] = 10`; total 25.

Array name = address of the first element

`a` (array name) behaves like `&a[0]` in expressions, and these are all the **same thing**:

ARRAY–POINTER IDENTITY

```
a[i]  ==  *(a + i)  ==  *(i + a)  ==  i[a]
```

(Yes, `i[a]` is legal C.) But `a` itself is **not** a variable you can change: `a++` is an error, while `p++` is fine.

`sizeof(a)` for `int a[10]` is **40** (whole array), but `sizeof(p)` for a pointer is **8** (64-bit) — a favourite trap. When an array is passed to a function it **decays to a pointer**, so `sizeof` inside the function gives the pointer size.

4.3 The four ++ and * combinations (memorise)

Let `p` point to `a[0]`, with `a = {10,20,30,40,50}`:

Expression	What it does	Value	Side effect
`*p++`	read `*p`, **then move the pointer**	10	`p` now points to `a[1]`
`(*p)++`	read `*p`, **then increase the pointed value**	10	`a[0]` becomes 11
`*++p`	**move the pointer first**, then read	20	`p` points to `a[1]`
`++*p`	**increase the pointed value first**, then read	11	`a[0]` becomes 11

Reason: ++ and * are both prefix/unary operators with right-to-left associativity, and postfix ++ binds tighter than *. So `*p++` = `*(p++)`.

4.4 Pointers and functions

C is **call by value**: the function gets a *copy* of each argument.

```c
void swapv(int a, int b)   { int t=a; a=b; b=t; }      // swaps copies — no effect
void swapp(int *a, int *b) { int t=*a; *a=*b; *b=t; }  // swaps originals
```

To let a function change the caller's variable, pass its **address**.

```c
void f(int *a, int b) { *a += b; b *= 2; }
int x = 3, y = 4;
f(&x, y);      // x = 7;  b was a copy
f(&y, x);      // y = 4 + 7 = 11
printf("%d %d", x, y);   // 7 11
```

Arrays are passed by address (they decay), so the function changes the original:

```c
void g(int a[], int n) {            // a[] is really int *a
    for (int i = 0; i < n/2; i++) { int t=a[i]; a[i]=a[n-1-i]; a[n-1-i]=t; }
}
int r[] = {1,2,3,4,5}; g(r,5);       // r is now {5,4,3,2,1}
```

4.5 Arrays in memory — and address calculation (from your notes)

1-D array

If `A[L … U]` starts at address `b`, each element is `sz` bytes:

KEY POINT

$$\mathbf{addr(A[i]) = b + (i - L) \times sz} \text{ (for C, L = 0, so b + i × sz)}$$

2-D array: row-major vs column-major

Take the 3×3 table

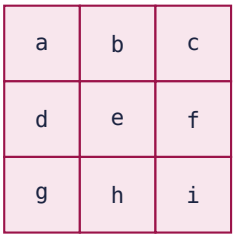


Figure 4.1 A 3 × 3 table, used to compare row-major and column-major storage.

- **Row-major order (RMO)** — C's way: each **row** is stored one after another: `a b c | d e f | g h i`
- **Column-major order (CMO)** — each **column** stored one after another: `a d g | b e h | c f i`

For a 2-D array `A[L1…U1][L2…U2]` with **r rows**, **c columns**, base address `b`, element size `sz`:

NOTE

addr(A[i][j]) = b + [(i − L1) × c + (j − L2)] × sz
CMO: addr(A[i][j]) = b + [(j − L2) × r + (i − L1)] × sz

How to remember: in RMO you *skip whole rows* (each has `c` elements) then walk `j` steps. In CMO you *skip whole columns* (each has `r` elements) then walk `i` steps.

Worked example 1. `int m[4][5]`, base 1000, `int` = 4 bytes. Address of `m[2][3]`?

- RMO: 1000 + (2×5 + 3) × 4 = **1052**
- CMO: 1000 + (3×4 + 2) × 4 = **1056**

Worked example 2 (non-zero lower bounds). `A[−2…5][3…9]`, base 500, element size 2 bytes. Address of `A[1][5]`?

- rows = 5 − (−2) + 1 = 8, columns = 9 − 3 + 1 = 7.
- RMO: 500 + [(1+2)×7 + (5−3)] × 2 = 500 + 23×2 = **546**
- CMO: 500 + [(5−3)×8 + (1+2)] × 2 = 500 + 19×2 = **538**

n-dimensional arrays (lecture notes)

Let N_i = size of dimension i, n_i = index in dimension i.

KEY POINT

In RMO, the *last* index changes fastest. In CMO, the *first* index changes fastest.

For the 3-D array `a[N1][N2][N3]`:

NOTE

addr = b + (n1 · N2 · N3 + n2 · N3 + n3) × sz
CMO: addr = b + (n3 · N1 · N2 + n2 · N1 + n1) × sz

Pattern to remember: in RMO each index is multiplied by the **product of all sizes to its right**; in CMO each index is multiplied by the **product of all sizes to its left**.

How to *see* an n-D array (the notes' picture trick):

- `a[N1][N2][N3]` in **RMO** = **N1 arrays, each of size N2 × N3**.
- `a[N1][N2][N3]` in **CMO** = **N3 arrays, each of size N1 × N2**.

Notes example — `a[2][4][3]`. Suppose memory shows … `000, 001, 002, 010, 011, …`. The *last* index changes fastest → **row-major**. So `a` is a collection of **2 arrays of 4 × 3**.

Now suppose memory shows … `000, 010, 020, 001, 011, 021, …`. Here the *middle* index changes fastest — that is **neither** RMO nor CMO. For genuine CMO the sequence must be … `000, 100, 010, 110, 020, 120, …` (first index fastest), which describes **3 arrays of 2 × 4**.

Worked example 3. `a[2][4][3]`, element `a[1][2][1]`, base 0, size 1:

- RMO: 1 · 4 · 3 + 2 · 3 + 1 = **19**
- CMO: 1 · 2 · 4 + 2 · 2 + 1 = **13**

SHORTCUT

write the offset as a mixed-radix number. RMO radices read left-to-right; CMO radices read right-to-left.

4.6 Strings

A C string is a `char` array ending with the **null character `'\0'`** (value 0).

```c
char s[]  = "data";   // array of 5 chars: d a t a \0     (modifiable)
char *p   = "data";   // pointer to a read-only literal
```

Expression	Value
`strlen(s)` (counts up to `\0`, not including it)	4
`sizeof(s)` (array, includes `\0`)	5
`sizeof(p)` (pointer, 64-bit)	8

Typical loop: `while (s[i]) i++;` finds the length. Always remember `'0'` is the **character** 48, so `s[i]-'0'` converts a digit character to its number.

```c
char str[] = "GATE2026"; int i=0, c=0;
while (str[i]) { if (str[i]>='0' && str[i]<='9') c += str[i]-'0'; i++; }
printf("%d", c);        // 2+0+2+6 = 10
```

Common library functions: `strlen`, `strcpy(dest, src)`, `strcmp(a,b)` (0 if equal), `strcat(a,b)`. `strcpy` copies including the `\0`.

4.7 Structures and `sizeof` with padding

Computers read memory fastest when a value of size k sits at an address that is a **multiple of k**. So the compiler inserts **padding bytes**.

Rules (use these in GATE):

1. Each member is placed at the next address that is a **multiple of its own size** (alignment = size for basic types).
2. The **total size is rounded up to a multiple of the largest member's alignment**.

Worked examples (verified):

```c
struct A { char c; int i; char d; };   // c@0, pad 3, i@4..7, d@8, pad 3  → 12
struct B { char c; char d; int i; };   // c@0, d@1, pad 2, i@4..7         → 8
struct S { char a; short b; char c; int d; };
        // a@0, pad1, b@2..3, c@4, pad3, d@8..11                        → 12
struct D { double x; char y; int z; };
        // x@0..7, y@8, pad3, z@12..15                                  → 16
struct E { char c; double d; short s; };
        // c@0, pad7, d@8..15, s@16..17, pad6 (to multiple of 8)        → 24
struct F { char a; char b; char c; };  // no padding needed               → 3
```

SHORTCUT

declare members from **largest to smallest** (or group same sizes together). `struct A` (12) vs `struct B` (8) hold the same data.

A **union** shares one block of memory for all members; its size is the size of the **largest** member (rounded for alignment): `union { int i; char c[8]; }` → 8.

Accessing: `s.field` for a struct variable, `ptr->field` for a pointer to struct (`ptr->field` ≡ `(*ptr).field`). A struct can contain a **pointer to its own type** — that is how linked-list nodes are made (Chapter 7).

4.8 Practice set — pointers, arrays, strings, structures

VERIFIED

All outputs were obtained by compiling and running (64-bit; `int`=4).

Q1 NAT · 1 Output?

```c
int a[]={5,10,15,20,25}; int *p=a+1, *q=a+4;
printf("%ld %d", q-p, *(q-2) + *p);
```

ANSWER **3 25.**

Q2 NAT · 1

```c
int b[]={2,4,6,8,10}; int *p=b; int s=0;
while (p < b+5) { s += *p; p += 2; }
printf("%d", s);
```

ANSWER **18.** p visits `b[0], b[2], b[4]` → 2+6+10.

Q3 NAT · 2

```c
int x=10, *p=&x, **pp=&p;
**pp = *p + 5;   *p = **pp * 2;
printf("%d", x);
```

ANSWER **30.**

Q4 NAT · 2 `int m[4][5]` stored row-major from address 1000, `int` = 4 bytes. Address of `m[2][3]`?
Answer: 1052. (1000 + (2 · 5+3) · 4.)

Q5 NAT · 2 `A[−2…5][3…9]`, base 500, 2-byte elements, **column-major**. Address of `A[1][5]`?
Answer: 538.

Q6 MCQ · 1 Size of `struct { char a; short b; char c; int d; }` assuming usual alignment?
(A) 8 **(B)** 9 **(C)** 12 **(D)** 16 — **Answer: (C) 12.**

Q7 MCQ · 1 What is `sizeof(struct{double x; char y; int z;})`?
(A) 13 **(B)** 16 **(C)** 24 **(D)** 8 — **Answer: (B) 16.**

Q8 NAT · 1 Digit-sum of the characters in `"GATE2026"` using the loop in §4.6. **Answer: 10.**

Q9 NAT · 1 For `char s[]="data"; char *p="data";` compute `sizeof(s) + strlen(s) + sizeof(p)` (64-bit).
Answer: 5 + 4 + 8 = 17.

Q10 NAT · 2

```c
void f(int *a, int b) { *a += b; b *= 2; }
int x=3, y=4;  f(&x,y);  f(&y,x);
printf("%d %d", x, y);
```

ANSWER **7 11.**

Q11 NAT · 2

```c
void g(int a[], int n){ for(int i=0;i<n/2;i++){int t=a[i];a[i]=a[n-1-i];a[n-1-i]=t;} }
int a[]={1,2,3,4,5}; g(a,5);
printf("%d", a[0]*10 + a[4]);
```

ANSWER **51** (array becomes 5 4 3 2 1).

Q12 NAT · 1

```c
int m[3][3]={{1,2,3},{4,5,6},{7,8,9}};
printf("%d %d", *(*(m+1)+1), *(m[2]+2) - **m);
```

ANSWER **5 8.** `*(*(m+1)+1) = m[1][1] = 5`; `*(m[2]+2) = m[2][2] = 9`; `**m = m[0][0] = 1`; $9-1=8$.

Q13 MSQ · 1 Which of the following are equivalent to `a[i]`?
(A) `*(a+i)` **(B)** `*(i+a)` **(C)** `i[a]` **(D)** `a+i`
Answer: A, B, C. `a+i` is an address, not the element.

Q14 NAT · 1 `int t[]={1,2,3,4}; int *p=t; p=p+2; printf("%d %d", p[-1], p[1]);`
Answer: 2 4.

Q15 NAT · 2 `int a[2][4][3]` stored in column-major order, base 0, element size 1. Offset of `a[1][3][2]`?
Answer: $n3 \cdot N1 \cdot N2 + n2 \cdot N1 + n1 = 2 \cdot 2 \cdot 4 + 3 \cdot 2 + 1 = 16 + 6 + 1 = \mathbf{23}$.

Q16 NAT · 2 Given `int mm[3][4]` with `mm[i][j] = i*10 + j` and `int *pp = &mm[0][0];`, what is `*(pp + 7)`?
Answer: 13. Offset 7 in row-major = row 1, column $3 \rightarrow 1 \cdot 10+3$.

Q17 MCQ · 1 After `int a[]={10,20,30,40,50}; int *q=a; int v=*q++;` the value of `v` and `*q` are **(A)** 10, 10 **(B)** 10, 20 **(C)** 20, 20 **(D)** 11, 10 — **Answer: (B).**

Self-check

SELF-CHECK

- ☐ I never add bytes to pointers — I add elements.
- ☐ I know `*p++`, `(*p)++`, `*++p`, `++*p` by heart.
- ☐ I can write the RMO/CMO formula for 2-D and 3-D arrays.
- ☐ I can compute struct size with padding in under a minute.

CHAPTER 5

Recursion & Function Calls

5

TIER 3 0.89 marks/paper and in the last 3 papers

EXAM WEIGHT

Exam weight: 0.89 marks per paper; asked in 5 of 9 papers, and in 2024, 2025 and 2026. In 2018 it gave 3 marks.
Styles (2018–2026): trace a recursive C function — return value or printed output (4 NAT questions); parameter passing and evaluation order (2 MCQ); runtime memory, stack vs heap (1 MCQ, GATE 2026).
Recursion is also the **engine behind trees, BST, AVL and heaps** — if you are comfortable here, the later chapters become easy.

5.1 What recursion is

A function is **recursive** if it calls itself. It needs two parts:

1. **Base case** — a situation answered directly (stops the calls).
2. **Recursive case** — solves the problem using a *smaller* version of itself.

C

```c
int fact(int n) {
    if (n <= 1) return 1;          // base case
    return n * fact(n - 1);        // recursive case
}
```

What really happens (stack frames)

Each call gets its own **frame** (Chapter 2, §2.6) with its own `n`. Calls pile up on the stack; when the base case returns, frames are popped one by one and each waits-then-finishes its multiplication.

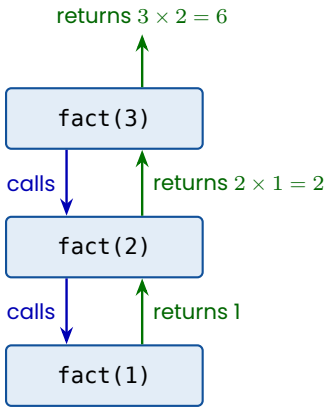


Figure 5.1 Call chain of `fact(3)`: calls go down, return values come back up.

Recursion depth = the maximum number of frames on the stack at once. A recursion with no base case, or with too much depth, causes **stack overflow**.

5.2 The method: draw the call tree

For every GATE recursion question:

1. Write the first call at the top.
2. Under it write each call it makes, **in the order the code makes them**.
3. Mark where each `printf` happens (before or after the recursive call).
4. Fill in return values from the bottom up.

Before vs after the recursive call

```c
void p(int n) {
    if (n == 0) return;
    printf("%d ", n);     // BEFORE recursion  → prints on the way down
    p(n - 1);
    printf("%d ", n);     // AFTER recursion   → prints on the way back up
}
p(3);     // prints: 3 2 1 1 2 3
```

- Print **before** the call → outputs in call order (like preorder in trees).
- Print **after** the call → outputs in reverse (like postorder).
- Two recursive calls with a print between them behave like **inorder**:

```c
void pr(int n) {
    if (n > 0) {
```

```
        pr(n - 1);
        printf("%d ", n);
        pr(n - 2);
    }
}
pr(3);   // prints: 1 2 3 1
```

Trace: `pr(3)` → `pr(2)` → `pr(1)` → `pr(0)`(nothing), print **1**, `pr(-1)`(nothing); back in `pr(2)`: print **2**, `pr(0)`; back in `pr(3)`: print **3**, `pr(1)`: `pr(0)`, print **1**, `pr(-1)`. Output `1 2 3 1`.

5.3 Return-value recursion: patterns you must recognise

Function	What it computes	Example
`g(n) = n%10 + g(n/10)` , `g(0)=0`	digit sum	`g(2025) = 5+2+0+2 = 9`
`f(n) = (n&1) + f(n>>1)`	number of 1-bits	`f(45) = 4` ($45 = 101101_2$)
`gcd(a,b) = b==0 ? a : gcd(b, a%b)`	Euclid's GCD	`gcd(48,18) = 6` (calls: (48,18)→(18,12)→(12,6)→(6,0))
`pw(x,n)`: `t = pw(x,n/2)`; return `n odd ? t*t*x : t*t`	fast power, depth $\approx \log_2 n$	`pw(3,5)=243`
`h(n) = h(n-1) + h(n-1)`, `h(0)=1`	2^n	`h(4) = 16`
`fib(n) = fib(n-1)+fib(n-2)`	Fibonacci, exponential time	`fib(6) = 8`

Counting the number of calls

For `fib` with base `fib(0)=0, fib(1)=1`:

FORMULA

$$\text{Number of calls to compute fib(n)} = 2 \cdot \text{fib(n+1)} - 1$$

Check: `fib(7) = 13`, so `fib(6)` makes $2 \cdot 13 - 1 = \mathbf{25}$ calls; `fib(5)` makes $2 \cdot \text{fib}(6) - 1 = 2 \cdot 8 - 1 = \mathbf{15}$ calls. (Both verified by running.)

For `h(n)` that calls itself twice: calls $= 2^{n+1} - 1$. `h(4)` makes **31** calls.

For a general recursion, write a **recurrence** for the number of calls: `C(n) = 1 + C(n-1) + C(n-2)`. Solve by expanding or by the master theorem (Chapter 6).

Another typical one — return value *and* count together

C

```c
int cnt = 0;
int f(int n) { cnt++; if (n <= 0) return 0; return f(n-1) + f(n-2) + 1; }
```

`f(0)=0, f(1)=f(0)+f(-1)+1=1, f(2)=2, f(3)=4, f(4)=7`; the calls made by `f(4)` are **15**.

Ackermann (appears as a "trace" question)

```c
int A(int m,int n){ if(m==0) return n+1; if(n==0) return A(m-1,1); return A(m-1,
A(m,n-1)); }
```

`A(2,2) = 7`, `A(2,3) = 9`. Pattern: `A(2,n) = 2n+3`; `A(1,n) = n+2`; `A(0,n) = n+1`.

McCarthy's 91 function

```c
int f91(int n){ if(n>100) return n-10; return f91(f91(n+11)); }
```

For **any n $\leq$ 100 it returns 91**. (`f91(50) = 91`.) Questions that look scary often hide a simple closed form — try small inputs first.

5.4 `static` variables inside recursion

A `static` local is **shared by all the calls** (one copy only).

```c
int sf(int n) {
    static int c = 0;
    c++;
    if (n > 0) sf(n - 1);
    return c;
}
// sf(3): calls with n = 3,2,1,0  → c is incremented 4 times → returns 4
```

Note the return reads `c` **after** all the deeper calls have finished, so it sees the final total.

Contrast with a normal local variable, where each frame has its own copy.

5.5 Mutual recursion

```c
int od(int n){ return n==0 ? 0 : ev(n-1); }
int ev(int n){ return n==0 ? 1 : od(n-1); }
```

`ev(n)` is 1 exactly when n is even. `ev(7) = 0`, `ev(10) = 1`.

Method	What the function receives	Can it change the caller's variable?

5.6 Passing parameters

Method	What the function receives	Can it change the caller's variable?
Call by value (C)	a **copy** of the value	No
Call by reference	the **variable itself** (an alias)	Yes
Pointer passing (C's way to imitate reference)	a copy of the **address**	Yes, through `*p`

C has only call by value. Passing a pointer is still by value (the address is copied) but the pointed-to data can be changed.

```c
void inc(int *p, int n) { if (n == 0) return; *p += n; inc(p, n-1); }
int x = 0; inc(&x, 4);   // x = 4+3+2+1 = 10
```

Order of evaluation of function arguments

In C the order in which the arguments of a function call are evaluated is **unspecified**. So `f(i++, i++)` or `printf("%d %d", i++, i)` is not reliable. GATE questions on this topic either state the order, or ask which statement is *true* about a language rule ("the value may differ between compilers"). If a trace question depends on it and does not specify, treat it as undefined.

Static vs dynamic scope (a classic MCQ)

```c
int x = 10;
void g() { printf("%d", x); }
void f() { int x = 20; g(); }
int main() { f(); }
```

- **Static (lexical) scoping** — C and most languages: `g` looks at where it is **written** → global `x` → prints **10**.
- **Dynamic scoping**: `g` looks at who **called** it → `f`'s `x` → would print 20.

5.7 Practice set — recursion and function calls

VERIFIED

All numeric answers verified by running the programs.

Q1 NAT · 2 How many calls to `fibc` are made (including the first) by `fibc(6)` if `fibc(n) { if (n<=1) return n; return fibc(n-1)+fibc(n-2); }`?
Answer: 25.

Q2 NAT · 1 `g(n) = (n==0) ? 0 : n%10 + g(n/10)`. Value of `g(2025)`? **Answer: 9.**

Q3 NAT · 2 How many integers are printed by `pr(3)` of §5.2, and what is their sum?
Answer: printed `1 2 3 1` → 4 numbers, sum **7**.

Q4 MCQ · 2 What does `p(3)` of §5.2 (print before and after) print?
(A) 3 2 1 **(B)** 1 2 3 **(C)** 3 2 1 1 2 3 **(D)** 1 2 3 3 2 1 — **Answer: (C).**

Q5 NAT · 2 Value returned by `sf(3)` in §5.4? **Answer: 4.**

Q6 NAT · 1 How many calls does `gcd(48, 18)` make (counting the first call)? **Answer: 4.**

Q7 NAT · 1 `f(n) = (n==0) ? 0 : (n&1) + f(n>>1)`. Find `f(45)`. **Answer: 4.**

Q8 NAT · 2 For `f(n){cnt++; if(n<=0) return 0; return f(n-1)+f(n-2)+1;}` find the value returned by `f(4)` **and** the number of calls it makes; report their sum.
Answer: value 7, calls 15 → **22**.

Q9 NAT · 1 `ev(7)` of §5.5 returns? **Answer: 0.**

Q10 NAT · 2 `A(2,3)` (Ackermann) = ? **Answer: 9.**

Q11 NAT · 1 After `int x=0; inc(&x,4);` what is `x`? **Answer: 10.**

Q12 NAT · 2 `h(n)` with `h(0)=1` and `h(n)=h(n-1)+h(n-1)`. What is `h(4)`, and how many calls are made in total? Give calls.
Answer: h(4)=16; calls = **31**.

Q13 MCQ · 1 In the static-vs-dynamic scoping program of §5.6, what does C print?
(A) 10 **(B)** 20 **(C)** 30 **(D)** undefined — **Answer: (A) 10.**

Q14 NAT · 1 `pw(3,5)` of §5.3. What is the **maximum recursion depth** (number of simultaneously active calls)?
Answer: 4. n goes 5 → 2 → 1 → 0, i.e. four frames.

Q15 NAT · 1 `f91(50)` returns? **Answer: 91.**

Q16 MSQ · 1 Which statements are correct?
(A) Local variables of a function live in the activation record on the stack.
(B) Memory obtained by `malloc` lives on the heap and survives the function's return.
(C) A recursive function with no base case will eventually cause a stack overflow.
(D) In C, passing an array to a function copies the whole array.
Answer: A, B, C. (D is false — an array decays to a pointer, so only the address is copied.)

Self-check

SELF-CHECK

- ☐ I draw a call tree before answering.
- ☐ I can tell "print before" from "print after" the call.
- ☐ I know calls(fib n) = $2 \cdot \text{fib}(n+1) - 1$.
- ☐ I know C is call-by-value and uses static scope.

CHAPTER 6

Asymptotic Analysis, Loop Complexity & Recurrences

6

TIER 1

EXAM WEIGHT

Exam weight (filed under Algorithms in your analysis): 1.61 marks per paper, trend ↑, asked in 8 of the last 9 papers, including 2024, 2025 and 2026.
Styles (2018–2026):

- **Solve a recurrence** (Master theorem / substitution) and pick the bound — 8 questions
- **Order / compare growth rates**, $\Theta - O - \Omega$ statements, arrange functions — 4 questions
- **Time complexity of given code** (loops, arrays) — 4 questions

Your lecture notes cover the 2nd and 3rd styles in great detail (Parts 6.1–6.9). Recurrences are **not** in your notes but are the **most-asked style**, so Part 6.11 is added.

6.1 Why do we need it? (from scratch)

Suppose algorithm A takes $3n^2 + 4$ steps and B takes n^2 steps for input size n. Which is "faster"? For very large n the $+4$ and the 3 hardly change the *shape* of growth: both grow like n^2.

KEY POINT

Goal of asymptotic analysis: simplify the running time by getting rid of the details.
Example: $3n^2 + 4 \simeq n^2$.

We only compare **how fast a function grows** when n becomes huge.

6.2 Big-O (upper bound)

KEY POINT

$T(n)$ is $O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that **for all** $n > n_0$: $T(n) \leq c \cdot g(n)$.

In words: *from some point onward, T(n) never rises above a constant multiple of g(n).* Big-O says "**at most this fast-growing**".

Example 1 — $n^2 = O(2^n)$

Compare values:

n	1	2	3	4	5	6	7	8	9
n^2	1	4	9	16	25	36	49	64	81
2^n	2	4	**8**	16	32	64	128	256	512
n^2 vs 2^n	$<$	$=$	$>$	$=$	$<$	$<$	$<$	$<$	$<$

The only place $n^2 > 2^n$ is $n = 3$. After that $n^2 \le 2^n$ for ever, so $c = 1$ works. (The notes pick $n_0 = 5$, a safe choice; any $n_0 \ge 3$ is correct in the "for all n $> n_0$" form.)

Example 2 — find (c, n_0) for $f(n) = n + \log n$

We need $n + \log n \le c \cdot n$. Take $c = 2$: $n + \log n \le 2n \iff \log n \le n$, true for all $n \ge 1$. So $(\mathbf{c}, n_0) = (\mathbf{2}, \mathbf{1})$. There are **infinitely many** such pairs (any bigger c also works).

Example 3 — is $2^{2n} = O(2^n)$?

Need $2^{2n} \le c \cdot 2^n \implies 2^n \le c$. But 2^n keeps growing, so no constant c can ever work. **No.**

Example 4 — is $2^{n+1} = O(2^n)$?

$2^{n+1} = 2 \cdot 2^n \le c \cdot 2^n \implies c \ge 2$. **Yes**, e.g. $(c, n_0) = (3, 1)$.

LESSON

adding a constant to the exponent → just a constant factor (fine). **Multiplying** the exponent (like $2n$ instead of n) → squares the function (not fine).

6.3 Big-Omega (lower bound)

KEY POINT

T(n) is Ω(g(n)) if there exist **$c > 0$, $n_0 \ge 0$** such that **for all $n > n_0$: $T(n) \ge c \cdot g(n)$**.

"**At least this fast-growing.**"

- Is $10n^2 = \Omega(100n^2)$? Take $c = 1/100$, $n_0 = 1$: $10n^2 \ge (1/100)(100n^2) = n^2$ ✓ **Yes.**
- Is $6 \cdot 2^n + n^2 = \Omega(2^n)$? $6 \cdot 2^n + n^2 \ge 1 \cdot 2^n$ ✓ with $c = 1$, $n_0 = 1$. **Yes.**

6.4 Big-Theta (tight bound)

The problem with O and Ω alone: they may not convey the exact growth. For $f(n) = 3n^2 + 2$:

- $f = O(n^2)$ but also $f = O(n^5)$ (true but useless),
- $f = \Omega(n^2)$ but also $f = \Omega(n)$ and $f = \Omega(1)$.

You could report $f = O(n^5)$ and be mathematically correct, but it gives no good estimate. So we use Θ:

FORMULA

$$\mathbf{T(n) = \Theta(g(n)) \iff T(n) = O(g(n)) \text{ AND } T(n) = \Omega(g(n))}$$

i.e. there exist c_1, c_2, n_0 such that $c_1 \cdot \mathbf{g(n)} \leq \mathbf{T(n)} \leq c_2 \cdot \mathbf{g(n)}$ for all $n \geq n_0$.

Example: `f(n) = 3n² + 2`. Take `c₁ = 1`, `c₂ = 4`: `n² ≤ 3n² + 2 ≤ 4n²` for `n ≥ 2` (the right side needs `n² ≥ 2`). So `3n² + 2 = Θ(n²)`.

The comparison dictionary (memorise)

Statement	Meaning	Think of it as
`f(n) = O(g(n))`	f grows **no faster** than g	$f \leq g$
`f(n) = Ω(g(n))`	f grows **no slower** than g	$f \geq g$
`f(n) = Θ(g(n))`	f and g grow at the **same rate**	$f = g$
`f(n) = o(g(n))`	f grows **strictly slower** than g	$f < g$
`f(n) = ω(g(n))`	f grows **strictly faster** than g	$f > g$

And: **if `f = Θ(g)` then both `f = O(g)` and `f = Ω(g)`** — and the converse is also true.

6.5 Little-o and little-ω

DEFINITION

`T(n) = o(g(n))` if **for every c > 0** there is an `n₀` with `T(n) < c·g(n)` for all `n > n₀`.
ω: `T(n) = ω(g(n))` if **for every c > 0** there is an `n₀` with `T(n) > c·g(n)` for all `n > n₀`.

Two small differences from Big-O (notes):

1. **Little-o needs it for every c > 0**; Big-O needs it for **some** c.
2. Big-O allows `T(n) ≤ c·g(n)`; Little-o is **strict**: `T(n) < c·g(n)`.

The limit shortcut (very useful in GATE)

Compute `L = lim (n→∞) f(n)/g(n)`:

L	Conclusion
`0`	`f = o(g)` (so also `O(g)`)
a positive constant	`f = Θ(g)`
`∞`	`f = ω(g)` (so also `Ω(g)`)

Not every pair is comparable: `n` and `n^{1+sin n}` oscillate — neither is O of the other.

6.6 Properties (all used in "which statement is true" questions)

Let `f, g, h` be positive functions.

1. **Transitivity:** `f=Θ(g)` and `g=Θ(h)` $\Longrightarrow$ `f=Θ(h)`. Same for `O`, `Ω`, `o`, `ω`.
2. **Reflexivity:** `f = Θ(f)`, `f = O(f)`, `f = Ω(f)` (but **not** `f = o(f)`).
3. **Symmetry (Θ only):** `f=Θ(g)` $\Longleftrightarrow$ `g=Θ(f)`.
4. **Transpose symmetry:** `f = O(g)` $\Longleftrightarrow$ `g = Ω(f)`, and `f = o(g)` $\Longleftrightarrow$ `g = ω(f)`.
5. `f = o(g)` $\Longrightarrow$ `f = O(g)`; `f = ω(g)` $\Longrightarrow$ `f = Ω(g)`.
6. **Sums and products:** `O(f) + O(g) = O(max(f,g))`; if `f = O(g)` then `f·h = O(g·h)`.

NOTATION WARNING

`O(g(n))` is really a **set of functions**. Writing `f(n) = O(g(n))` actually means `f(n) ∈ O(g(n))`. We use `=` only for convenience.

6.7 Growth-rate ladder (learn this order)

GROWTH-RATE LADDER

$$1 < \log\log n < \log n < (\log n)^k < n^{\varepsilon} < n < n\log n < n^2 < n^3 < n^k$$
$$< \ 2^n < n! < n^n < 2^{2^n}$$

(`ε` is any positive constant, `k` any constant. The shorter version in your notes: `log n < n < n log n < n² < n³ < 2ⁿ < n!`.)

Two useful facts:

- **For any $\varepsilon > 0$ and any k: `n^ε > (log n)^k`** eventually. Polynomials beat polylogarithms.
- **The base of a log does not matter** asymptotically: `log₂ n = Θ(log₁₀ n)` because `log₂ n = log₁₀ n / log₁₀ 2` (a constant factor).

Three useful sums

Sum	Value
`1 + 2 + 3 + … + n`	`n(n+1)/2` $\rightarrow \Theta(n^2)$
`1² + 2² + … + n²`	`n(n+1)(2n+1)/6` $\rightarrow \Theta(n^3)$
`1 + 1/2 + 1/3 + … + 1/n`	$\approx$ `log n` (harmonic series)
`log 1 + log 2 + … + log n`	`log n! = Θ(n log n)`
`1 + 2 + 4 + … + 2^k`	`2^{k+1} − 1` (geometric: $\approx$ last term $\times$ 2)

6.8 How to compare two functions: the techniques

Step 0 — Ignore constants only when it is *safe*

NOTE

It is safe to ignore a constant only if the function can be written as `c·g(n)` **or** `g(n) + c`.

- `f(n) = 2n² + 5` = `2·[n² + 5/2]` → ignore the 2 → `n² + 5/2` → ignore the 5/2 → **`n²`**.
- `f(n) = 2n^{5+n}` = `2·[n^{5+n}]` → ignore the 2. But **there is no way to remove the 5** — it sits in the exponent: `n^{5+n} = n⁵·nⁿ`, a multiplier that **depends on n**. So `f = n^{5+n}`.
- `n² + 5` ≃ `n²`, but `n^{5+n}` ≠ `n^{n}`.
- Multipliers: `2n²` ≃ `n²`, but `2^{2n}` ≠ `2^{n}`.

Step 1 — Cancel common factors

If both functions share a factor, cancel it first: compare `n² log n` and `n (log n)^{10}` by cancelling `n` → compare `n log n` with `(log n)⁹`.

Step 2 — The log method (the main weapon)

Take the logarithm of **both** functions and compare the results.

NOTE

1. Cancel out common terms.
2. Ignore constants (only in the safe forms above).
3. Take `log` of both `f(n)` and `g(n)`:
   - `log f > log g` $\Longrightarrow$ **f > g**
   - `log f < log g` $\Longrightarrow$ **f < g**
   - `log f = log g` (asymptotically) $\Longrightarrow$ **can't say** (use another method)

Why "can't say"? Example: `n²` vs `n³` gives `2 log n` vs `3 log n` → equal after ignoring constants. Yet `n² < n³`. After taking logs, constants get lost, so equal logs prove nothing.

KEY POINT

The log never flips an asymptotic order. If `a < b` asymptotically, then either `log a < log b` or `log a = log b` asymptotically — never `>`.

Step 3 — When logs tie and the base is the same

Divide by the common base-power and compare exponents (see Example H below).

Step 4 — Substitute for nested logs

When you see `log log n`, substitute `n = 2^{2^k}` so that `log n = 2^k` and `log log n = k`.

6.9 Worked examples from the lecture notes (all of them)

For each, f vs g; "log" means $\log_2$.

A. 2^n vs n^n. Logs: $n \cdot \log 2 = n$ vs $n \log n$. $n < n \log n \implies 2^n < n^n$.

B. $n^2 \log n$ vs $n (\log n)^{10}$. Both contain the factor $n \log n$. Cancel it:
$n^2 \log n = (n \log n) \cdot n$ and $n (\log n)^{10} = (n \log n) \cdot (\log n)^9$.
So compare n with $(\log n)^9$. Take logs: $\log n$ vs $9 \log \log n$. Put $K = \log n$: K vs $9 \log K$. Since $K > \log K$ (polynomial beats log), K wins. $\implies$ **$n^2 \log n$ grows faster** ($n^2 \log n = \Omega(n (\log n)^{10})$).

C. $f = (\log n)^{\log \log n}$, $g = (\log \log n)^{\log n}$. Let $n = 2^{2^k}$: $\log n = 2^k$, $\log \log n = k$.
$f = (2^k)^k = 2^{k^2}$; $g = k^{2^k}$. Logs: k^2 vs $2^k \log k$. Since $2^k \log k \gg k^2$, $\mathbf{f < g \implies f = O(g)}$.

D. $f = n \cdot 2^n$, $g = 4^n$. $4^n = (2^2)^n = (2^n)^2$. Cancel one 2^n: n vs 2^n. $n < 2^n \implies \mathbf{f = O(g)}$.

E. $f = n^2 \log n$, $g = n^{100}$. Cancel n^2: $\log n$ vs n^{98}; $\log n < n^{98} \implies \mathbf{f = O(g)}$.

F. $f = n^{\log n}$, $g = 2^n$. Logs: $\log n \cdot \log n = \log^2 n$ vs $n \log 2 = n$. $\log^2 n < n \implies \mathbf{f = O(g)}$.

G. $f = \log_2 n$, $g = \log_{10} n$. $\log_2 n = \log_{10} n \,/\, \log_{10} 2 \to$ ignoring the constant they are equal $\implies \mathbf{f = \Theta(g)}$.

H. $f = 2^n$, $g = n^{\sqrt{n}}$. Logs: n vs $\sqrt{n} \log n$. Cancel $\sqrt{n}$: $\sqrt{n}$ vs $\log n$. $\sqrt{n} = n^{1/2} > \log n$ $\implies \mathbf{f = \Omega(g)}$ (f is larger).

I. $f = n^{\log_2 n}$, $g = n^{\log_{10} n}$. Take logs: $(\log_2 n) \cdot (\log_2 n)$ vs $(\log_{10} n) \cdot (\log_2 n)$. These differ only by the constant $\log_2 10$, so after "ignoring constants" they tie and **the log method cannot decide** (this is exactly the "can't say" case).
Use another method — the bases are the same (n), so look at the **ratio**:
$f/g = n^{\log_2 n - \log_{10} n} = n^{(1 - 1/\log_2 10) \cdot \log_2 n}$. The exponent is a positive constant times $\log n$, so the ratio **grows without limit**. Hence $\mathbf{f = \Omega(g)}$ (f is bigger; in fact $f = \omega(g)$).

LESSON

Lesson: when the log method ties, **constants inside the exponent matter** — don't throw them away.

J. Compare 2^{2^n}, $n!$, 4^n, 2^n (call them A, B, C, D).

- A vs B: logs 2^n vs $n \log n \to$ A > B.
- C vs D: $4^n = (2^n)^2$ vs $2^n \to$ C > D.
- B vs C: logs $n \log n$ vs $2n \to$ B > C.
- **Order: A > B > C > D.**

K. Stirling's approximation (to compare $n!$ with n^n).
$n! \approx \sqrt{(2\pi n)} \cdot (n/e)^n$.
$n!/n^n \approx \sqrt{(2\pi n)} \cdot (1/e)^n \to 0$. So **$n! = O(n^n)$ and in fact $n! = o(n^n)$.**

L. Compare $n^{\sqrt{n}}$, 2^n, $n^{\log n}$, $n!$ (A, B, C, D). Take logs:
A: $\sqrt{n} \log n$; B: n; C: $\log^2 n$; D: $n \log n$.

Order of logs: $\log^2 n < \sqrt{n} \log n < n < n \log n$.
Order: C < A < B < D.

M. Compare $3\,n^{\sqrt{n}}$, $2^{\sqrt{n}\log n}$, $n!$ (A, B, C).
$2^{\sqrt{n}\log n} = (2^{\log n})^{\sqrt{n}} = n^{\sqrt{n}}$, so **A = B** (the factor 3 is a safe constant). And $n!$ is much larger (log: $n \log n$ vs $\sqrt{n} \log n$). **A = B < C.**

N. Compare $n^{1/3}$, e^n, $n^{7/4}$, $n \log^9 n$, 1.000001^n (A–E).
Exponentials beat polynomials: $e^n > 1.000001^n$ (bigger base) > any polynomial. Among polynomials: $n^{7/4} > n \log^9 n$ (since $n^{3/4} > \log^9 n$) > $n^{1/3}$.
Order: B > E > C > D > A.

O. Compare 2^n, $n^{3/2}$, $n \log n$, $n^{\log_2 n}$ (A–D).
A vs D: logs n vs $\log^2 n \to$ A > D. B vs C: $n^{3/2}$ vs $n \log n \to n^{1/2}$ vs $\log n \to$ B > C. D vs B: logs $\log^2 n$ vs $(3/2) \log n \to$ D > B.
Order: A > D > B > C.

P. Piecewise functions.
$g_1(n) = n^3$ $(0 \le n \le 10000)$, n^2 $(n > 10^4)$; $g_2(n) = n$ $(0 \le n \le 100)$, n^3 $(n > 100)$.

FORMULA

Always consider a large value past all break-points, e.g. $10^4 + 1$.
For large n: $g_1 = n^2$, $g_2 = n^3 \implies g_1 = \mathbf{O}(g_2)$.

Q. Which is false?
(A) $100\,n \log n = O(n \log n / 100)$ — true (constants don't matter).
(B) **$\sqrt{\log n} = O(\log \log n)$ — FALSE.** Put $k = \log n$: $\sqrt{k} > \log k$. (Rule: $k^{\varepsilon} > (\log k)^m$.)
(C) If $0 < x < y$, $n^x = O(n^y)$ — true.
(D) $2^n \neq O(n^k)$ — true (exponential beats polynomial).
ANSWER **B.**

R. Which is correct?
(A) $(n+k)^m = \Theta(n^m)$ for constants k, m — **correct** ($(n+k)^m = n^m + m\,n^{m-1}\,k + \ldots$, dominated by n^m).
(B) $2^{n+1} = O(2^n)$ — **correct** ($2 \cdot 2^n$).
(C) $2^{2n+1} = O(2^n)$ — **incorrect** ($2 \cdot 2^{2n}$, divided by 2^n leaves $2 \cdot 2^n$, unbounded).
ANSWER **A and B.**

S. Reasoning with relations.
Given: $f = O(g)$ and $g \neq O(f) \implies$ **g grows strictly faster than f** ($g > f$).
Also: $g = O(h)$ and $h = O(g) \implies$ **g and h grow at the same rate** ($g = \Theta(h)$).
Which statement is false?
(A) $f + g = O(h + h)$ — true ($f+g = O(g) = O(h)$).
(B) $f = O(h)$ — true ($f < g = h$).
(C) $h \neq O(f)$ — true.
(D) **$f \cdot h \neq O(g \cdot h)$ — FALSE**, because $f = O(g)$ implies $f \cdot h = O(g \cdot h)$.
ANSWER **D.**

6.10 Analysing loop complexity (all examples from the notes)

Method: (1) find how many times the **inner** loop runs for each value of the **outer** variable; (2) **sum** over the outer loop; (3) keep the dominant term.

6.10.1 Single loops — how does the variable move?

Loop	Values taken by `i`	Count	Complexity
`for (i=1; i<=N; i++)`	1, 2, 3, …	N	**O(N)**
`for (i=1; i<=N; i=i*2)`	1, 2, 4, 8, …, 2^k=N	k = $\log_2$N	**O(log N)**
`for (i=1; i*i<=N; i++)`	1, 2, …, k with k^2=N	k = $\sqrt{N}$	**O($\sqrt{N}$)**
`for (i=1; i<2^N; i=i*2)`	2^0, 2^1, …, 2^k with $2^k = 2^N$	k = N	**O(N)**
`for (i=2; i<=N; i=i*i)`	2, 2^2, 2^4, 2^8, …, 2^{2^k}=N	k = log log N	**O(log log N)**
`for (i=n²; i>=2; i=√i)`	n^2, n, n^{1/2}, n^{1/4}, … → 2	k ≈ log log n	**O(log log n)**

Why does `i=i*i` give log log N? i = 2^{2^k}. Setting `2^{2^k} = N` → `2^k = log N` → `k = log log N`.

Why does `i=√i` give log log n? Start `n²`; after k steps `i = (n²)^{1/2^k}`. It reaches 2 when `2^k = log₂(n²)`, so `k = log₂ log₂(n²) = log₂(2 log₂ n) = 1 + log₂ log₂ n.` → **O(log log n)**.

TRAP

`for (i=1; i<n; i=i*i)` is an **infinite loop** ($1 \cdot 1 = 1$ forever). The notes start the squaring loop from 2 for that reason.

6.10.2 Nested loops (the notes' examples, in order)

1. Triangular loop

```c
for (i = 0; i < N; i++)
    for (j = i+1; j < N; j++)
        stmt;
```

Inner runs `N-i-1` times. Sum = `Σ(N-i-1) = (N-1)+(N-2)+…+0 = N(N-1)/2`. **O(N^2).**

2. Inner bound depends on i^2

```c
for (i = 1; i <= N; i++)
    for (j = 1; j <= i*i; j++)
        stmt;
```

`Σ i² = N(N+1)(2N+1)/6`. **O(N^3).**

3. Inner variable multiplies by i (`j *= i`, `j ≤ i`)
For `i ≥ 2` the values are `1, i, i², …` and `j ≤ i` stops after the second step, so the inner loop runs a constant number of times (K = 1 in the notes, ignoring the degenerate `i=1`). Total **O(N)**.

4. Inner loop jumps by i (`j += i`, `j ≤ N`)

```c
for (i = 1; i <= N; i++)
    for (j = 1; j <= N; j += i)
        stmt;
```

Inner runs `N/i` times. Sum = `N(1 + 1/2 + 1/3 + … + 1/N) = N · log N`. **O(N log N).**
(This harmonic-sum pattern is very common. Memorise: "j += i up to N, i = 1…N" ⟹ N log N.)

5. Inner loop doubles up to i

```c
for (i = 1; i <= N; i++)
    for (j = 1; j <= i; j = j*2)
        stmt;
```

Inner runs `log i` times. Sum = `Σ log i = log(N!) = θ(N log N)`. **O(N log N).**

6. Outer doubles, inner linear in i

```c
for (i = 1; i <= N; i = i*2)
    for (j = 1; j <= i; j++)
        stmt;
```

Outer values `i = 1, 2, 4, …, 2^k = N`. Inner runs `i` times. Total = `1 + 2 + 4 + … + 2^k = 2^{k+1} − 1 = 2N − 1`. **O(N)** — *not* N log N!

> **NOTE**
> The geometric sum is dominated by its **last term**. This is the classic trap.

7. Outer doubles, inner up to i^2

```c
for (i = 1; i <= N; i = i*2)
    for (j = 1; j <= i*i; j++)
        stmt;
```

Total = `1 + 2² + 4² + … + (2^k)² = 1 + 4 + 16 + … + 4^k = (4^{k+1} − 1)/3`. With `2^k = N` → `4^k = N²`, so total ≈ `(4N² − 1)/3`. **O(N^2).**

8. Three loops: squaring, linear, jumping by j

```c
for (i = 2; i <= n; i = i*i)
    for (j = 1; j <= n; j++)
        for (k = 1; k <= n; k = k + j)
            stmt;
```

- `i` loop: `log log n` iterations.
- For each `j`, `k` loop runs `n/j` times; `Σ_{j=1}^{n} n/j = n log n`.
- Total = `log log n × n log n` = **O(n log n · log log n).**

6.10.3 Cheat-sheet for loops

Pattern	Result
`i++` to N	N
`i*=2` to N	log N
`i*=i` to N (start ≥ 2)	log log N
`i+=k` to N	N/k
triangular `j` from `i` to N	$N^2/2$
outer `i*=2`, inner `j ≤ i`	2N (linear)
`j += i` for `i = 1…N`, `j ≤ N`	N log N
`j *= 2` up to `i`, `i = 1…N`	N log N
geometric sum	$\approx$ last term $\times$ constant
harmonic sum	log N

6.11 Recurrences and the Master theorem (added — the most-asked style: 8 questions in 2018–2026)

A **recurrence** expresses the running time of a recursive algorithm in terms of smaller inputs. Example (merge sort): split into 2 halves, solve each, merge in linear time:

FORMULA

$$\mathbf{T(n) = 2 \cdot T(n/2) + n}$$

6.11.1 Master theorem (for `T(n) = a·T(n/b) + f(n), a ≥ 1, b > 1`)

Compute the **watershed** `n^{log_b a}` and compare it with `f(n)`:

Case	Condition	Answer
1	`f(n)` grows **polynomially slower** than `n^{log_b a}`	`T(n) = Θ(n^{log_b a})`
2	`f(n) = Θ(n^{log_b a} · log^k n)`, `k ≥ 0`	`T(n) = Θ(n^{log_b a} · log^{k+1} n)`
3	`f(n)` grows **polynomially faster** than `n^{log_b a}` (plus regularity: `a·f(n/b) ≤ c·f(n)` for some `c<1`)	`T(n) = Θ(f(n))`

Quick recipe: the *bigger* of `n^{log_b a}` and `f(n)` wins; if they are equal (up to log powers), **multiply by one more `log n`**.

6.11.2 Worked examples (each also checked numerically)

Recurrence	a, b	`n^{log_b a}`	f(n)	Answer
`T(n)=2T(n/2)+n`	2, 2	n	n	tie, k=0 → $\Theta(\mathbf{n \log n})$
`T(n)=4T(n/2)+n`	4, 2	n^2	n	case 1 → $\Theta(n^2)$
`T(n)=3T(n/2)+n`	3, 2	n^{1.585}	n	case 1 → $\Theta(\mathbf{n^{\log_2 3}})$
`T(n)=3T(n/2)+n²`	3, 2	n^{1.585}	n^2	case 3 → $\Theta(n^2)$

Recurrence	a, b	n^{log_b a}	f(n)	Answer
T(n)=T(n/2)+1	1, 2	$n^0 = 1$	1	tie → $\Theta(\log n)$ (binary search)
T(n)=T(n/2)+n	1, 2	1	n	case 3 → $\Theta(n)$
T(n)=2T(n/2)+n log n	2, 2	n	n log n	tie with k=1 → $\Theta(n \log^2 n)$
T(n)=3T(n/3)+n log n	3, 3	n	n log n	k=1 → $\Theta(n \log^2 n)$
T(n)=4T(n/2)+n² log n	4, 2	n^2	$n^2 \log n$	k=1 → $\Theta(n^2 \log^2 n)$
T(n)=2T(n/4)+√n	2, 4	$\sqrt{n}$	$\sqrt{n}$	tie → $\Theta(\sqrt{n} \log n)$
T(n)=8T(n/2)+n²	8, 2	n^3	n^2	case 1 → $\Theta(n^3)$

6.11.3 When the Master theorem does not apply

- **Subtraction recurrences** `T(n) = a·T(n−c) + f(n)`: expand.
  - `T(n) = T(n−1) + n` → `n + (n−1) + … = n(n+1)/2` → $\Theta(n^2)$
  - `T(n) = T(n−1) + log n` → `log n!` → $\Theta(n \log n)$
  - `T(n) = T(n−1) + 1` → $\Theta(n)$
  - `T(n) = 2T(n−1) + 1` → `2^n − 1` → $\Theta(2^n)$
  - `T(n) = T(n−1) + T(n−2) + 1` (Fibonacci-like) → exponential, $\Theta(\varphi^n)$ with $\varphi \approx 1.618$
- **Unequal splits**: `T(n) = T(n/3) + T(2n/3) + n`. Draw the recursion tree: every level costs ≈ n, the depth is `log_{3/2} n`. → $\Theta(n \log n)$.
- **Non-polynomial gap**: `T(n) = 2T(n/2) + n/log n`. `f` is smaller than `n` but **not polynomially** smaller, so case 1 fails. Summing the tree: level i costs `n/(log n − i)`, total `n·H(log n)` ≈ `n log log n`. → $\Theta(n \log \log n)$. *(A favourite trap — GATE writes this expecting you to notice the theorem fails.)*
- **Square-root recurrences**:
  - `T(n) = T(√n) + 1` → put `n = 2^m` → `S(m) = S(m/2)+1` → `S = log m` → $\Theta(\log \log n)$.
  - `T(n) = 2T(√n) + log n` → `S(m) = 2S(m/2)+m` → `m log m` → $\Theta(\log n \cdot \log \log n)$.

6.11.4 The substitution (guess-and-prove) method — 30-second version

Guess the answer, then check by induction. Example: `T(n) = 2T(n/2) + n`, guess `T(n) ≤ c·n log n`:
`T(n) ≤ 2·c(n/2)log(n/2) + n = c·n log n − c·n + n ≤ c·n log n` for `c ≥ 1`. ✓

6.11.5 Binary search, merge sort, etc. in one line

Algorithm	Recurrence	Result
Binary search	`T(n) = T(n/2) + 1`	$\Theta(\log n)$
Merge sort	`T(n) = 2T(n/2) + n`	$\Theta(n \log n)$
Quick sort (best/avg)	`T(n) = 2T(n/2) + n`	$\Theta(n \log n)$
Quick sort (worst)	`T(n) = T(n−1) + n`	$\Theta(n^2)$
Heapify (Ch 12)	`T(h)=T(h−1)+1` per node	$\Theta(\log n)$

6.12 Best case, average case, worst case

An algorithm may take different time on different inputs of the same size.

- **Worst case**: the maximum over all inputs. Linked-list search for the last element = O(n).
- **Best case**: the minimum. Search finds the element at the head = O(1).
- **Average case**: the expected time over a distribution of inputs (hashing, Ch 13).

Big-O/Ω/Θ are **not** the same as worst/best/average. You can write the **O bound of the best case** and the Ω **bound of the worst case**. A statement like "the best-case running time is O(n)" is perfectly sensible.

6.13 Practice set — asymptotics (GATE style)

Q1 MCQ · 1 Which of the following is **true**?
(A) $n^{\log n} = O(2^n)$ **(B)** $2^{2n} = O(2^n)$ **(C)** $n! = O(2^n)$ **(D)** $\sqrt{(\log n)} = O(\log \log n)$
Answer: (A). Logs: $\log^2 n < n$. **(B)** false (ratio 2^n). **(C)** false ($n! \gg 2^n$). **(D)** false: with $k = \log n$, $\sqrt{k}$ grows faster than $\log k$.

Q2 MCQ · 1 Arrange in **increasing** order of growth:
$f_1 = n^{10}$, $f_2 = (\log n)^{\log n}$, $f_3 = n^{\log n}$, $f_4 = 2^{\sqrt{n}}$, $f_5 = n!$
(A) f_1, f_2, f_3, f_4, f_5 **(B)** f_2, f_1, f_3, f_4, f_5 **(C)** f_1, f_3, f_2, f_4, f_5 **(D)** f_1, f_2, f_4, f_3, f_5
Answer: (A). Logs: f_1: $10 \log n$; f_2: $\log n \cdot \log \log n$; f_3: $\log^2 n$; f_4: $\sqrt{n}$; f_5: $n \log n$. Note $(\log n)^{\log n} = n^{\log \log n}$; $\log \log n$ eventually exceeds 10, so $f_2 > f_1$.

Q3 MCQ · 2 Time complexity of

```c
for (i = 1; i <= n; i++)
    for (j = 1; j <= n; j += i) stmt;
```

(A) Θ(n) **(B)** Θ(n log n) **(C)** $\Theta(n^2)$ **(D)** Θ(n $\log^2$ n)
ANSWER **(B).** Inner loop runs n/i times; Σ n/i = n·H(n) = θ(n log n).

Q4 MCQ · 1 Time complexity of

```c
for (i = n; i >= 1; i = i/2)
    for (j = 1; j <= i; j++) stmt;
```

(A) Θ(n log n) **(B)** Θ(n) **(C)** $\Theta(n^2)$ **(D)** Θ(log n)
ANSWER **(B).** n + n/2 + n/4 + … ≈ 2n.

Q5 MCQ · 1 T(n) = 3T(n/3) + n log n. Then T(n) is
(A) Θ(n log n) **(B)** Θ(n $\log^2$ n) **(C)** $\Theta(n^2)$ **(D)** Θ(n)
Answer: (B). a=b=3, watershed n; f = n log n ⟹ tie with k=1 ⟹ extra log.

Q6 MCQ · 1 T(n) = 2T(n/2) + n/log n. Then T(n) is
(A) Θ(n) **(B)** Θ(n log n) **(C)** Θ(n log log n) **(D)** $\Theta(n^2)$
Answer: (C). Master theorem fails (gap not polynomial). Summing levels gives n·(1 + 1/2 + … + 1/log n).

Q7 MCQ · 1 `T(n) = T(√n) + 1`. Then T(n) is
(A) Θ(log n) **(B)** Θ(log log n) **(C)** $\Theta(\sqrt{n})$ **(D)** Θ(1)
Answer: (B).

Q8 MCQ · 2 `T(n) = T(n/3) + T(2n/3) + n`. Then T(n) is
(A) Θ(n) **(B)** Θ(n log n) **(C)** $\Theta(n^2)$ **(D)** $\Theta(2^n)$
Answer: (B).

Q9 NAT · 2 `T(1)=1, T(n) = T(n−1) + n`. What is T(100)?
Answer: 5050. (`1+2+…+100`.) *(In general `n(n+1)/2`.)*

Q10 MSQ · 2 Let `f(n) = n²` and `g(n) = n² log n`. Which are true?
(A) `f = O(g)` **(B)** `f = o(g)` **(C)** `g = O(f)` **(D)** `f = Θ(g)`
Answer: A and B. `f/g = 1/log n → 0`.

Q11 NAT · 1 `Σ_{i=1}^{n} (n/i)` is Θ of which of: n, n log n, n^2? **Answer: n log n.**

Q12 MCQ · 1 What does the following loop cost?

```c
for (i = 2; i <= n; i = i*i) stmt;
```

(A) Θ(log n) **(B)** Θ(log log n) **(C)** $\Theta(\sqrt{n})$ **(D)** infinite loop
ANSWER **(B).** (Had it started at `i = 1` it would be (D).)

Q13 MCQ · 2 `f = n^{log₂ n}`, `g = n^{log₁₀ n}`. Then
(A) `f = O(g)` **(B)** `f = Θ(g)` **(C)** `f = ω(g)` **(D)** cannot be compared
Answer: (C). The log method ties; the ratio `n^{c·log n}` with `c>0` is unbounded.

Q14 MSQ · 2 `f(n) = O(g(n))` and `g(n) ≠ O(f(n))`, and `g = Θ(h)`. Which must be true?
(A) `f = O(h)` **(B)** `h ≠ O(f)` **(C)** `f·h = O(g·h)` **(D)** `f = Θ(h)`
Answer: A, B, C. (D need not hold: f grows strictly slower.)

Q15 MCQ · 1 Which is the correct Θ for `f(n) = Σ_{i=1}^{n} i·log i`?
(A) Θ(n log n) **(B)** $\Theta(n^2 \log n)$ **(C)** $\Theta(n^2)$ **(D)** $\Theta(n^3)$
Answer: (B). (Numerically `Σ i log i ≈ ½ n² log n`.)

Q16 MCQ · 1 2^n versus 3^n: is `3^n = O(2^n)`?
Answer: No. `(3/2)^n → ∞`. This is the same lesson as `2^{2n}` vs 2^n: the base matters.

Q17 NAT · 2 `T(n) = 2T(n/2) + n`, `T(1) = 1`, n a power of 2. Exact value of T(8)?
Answer: 32. T(2)=2 · 1+2=4, T(4)=2 · 4+4=12, T(8)=2 · 12+8=32. (= `n log n + n = 24+8`.)

Q18 MCQ · 1 `T(n) = 8T(n/2) + n²`. Then T(n) is
(A) $\Theta(n^2)$ **(B)** $\Theta(n^3)$ **(C)** $\Theta(n^3 \log n)$ **(D)** $\Theta(n^4)$
Answer: (B). Watershed `n^{log₂8} = n³ ≫ n²`.

Self-check

SELF-CHECK

- ☐ I can apply the log method and know when it fails.
- ☐ I know the ladder `log < n^ε < n < n log n < n² < 2ⁿ < n! < nⁿ`.
- ☐ I recognise the harmonic (`N log N`) and geometric (`≈ last term`) loop sums.

- ☐ I can solve `aT(n/b)+f(n)` with the Master theorem and name the 3 traps (subtraction, unequal split, `n/log n`).

CHAPTER 7

Linked Lists

7

TIER 3 0.56 marks/paper

EXAM WEIGHT

Exam weight: asked in 5 of the last 9 papers (2020, 2022, 2023, 2025, 2026). Styles: code on a linked list — reversal, deletion, counting nodes, insertion complexity (5 questions in 2020–2026).
The lecture notes cover the definition, the three kinds of list and all complexities. The **code-tracing** part (reversal etc.) is added because that is what GATE actually asks.

7.1 From scratch: what is a linked list?

An **array** stores items side by side, so inserting in the middle forces you to shift everything. A **linked list** stores each item in a separate little box called a **node**, and each node also stores **the address of the next node**.

DEFINITION

A linked list is a collection of nodes, which may or may not be contiguous in memory. Each node stores data along with a pointer to some other node. (notes)

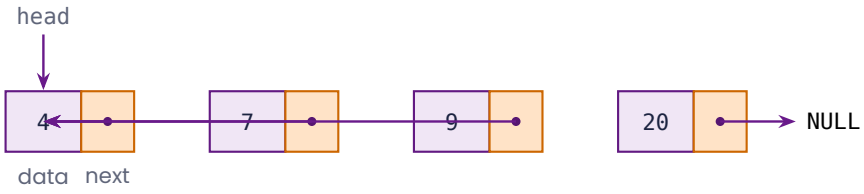


Figure 7.1 A singly linked list: each node holds data and a pointer to the next node.

The last node's `next` is `NULL`. We only keep the address of the first node, called **head**.

```c
struct Node {
    int data;
    struct Node *next;
};
```

Where do the nodes live?

If a node is created as a normal local variable, it sits on the **stack** and is **destroyed when the function's activation record is deleted**. So we make nodes on the **heap**:

```c
struct Node *p = malloc(sizeof(struct Node));
```

7.2 Operations and their cost (from the notes)

Assume the standard **singly linked list with only a `head` pointer**.

Operation	Best	Average	Worst	Why
Accessing the k-th element	O(1)	**O(n)**	O(n)	must walk from head
Traversal (visit all)	O(n)	O(n)	O(n)	always touches every node
Insertion at beginning	O(1)	O(1)	O(1)	just rewire head
Insertion in between	–	**O(n)** (avg)	O(n)	must first find the position
Insertion at end	O(n)	O(n)	O(n)	must walk to the last node
Deletion at beginning	O(1)	O(1)	O(1)	
Deletion in between	–	**O(n)** (avg)	O(n)	find the predecessor first
Deletion at end	O(n)	O(n)	O(n)	need the node *before* the last

Notes' summary: accessing: O(n) average & worst, O(1) in best case. Traversal: O(n) in all cases. Insert/delete: **beginning O(1) in all cases; in-between O(n) average; end O(n) in all cases.**

KEY IDEA

the *rewiring* of pointers is O(1). The cost is in **finding the place** — that's what costs O(n).

Insert at the beginning (O(1))

```c
struct Node *n = malloc(sizeof(struct Node));
n->data = x;
n->next = head;     // new node points to old first node
head = n;           // head now points to the new node
```

Insert after a given node `p` (O(1) once you hold `p`)

```c
n->next = p->next;
p->next = n;
```

Order matters! If you do `p->next = n` first, you lose the rest of the list.

Delete the first node (O(1))

```c
struct Node *t = head;
head = head->next;
free(t);
```

Delete the node after p (O(1))

```c
struct Node *t = p->next;
p->next = t->next;
free(t);
```

7.3 Circular linked list

KEY POINT

The last node, rather than pointing to NULL, points back to the first node.

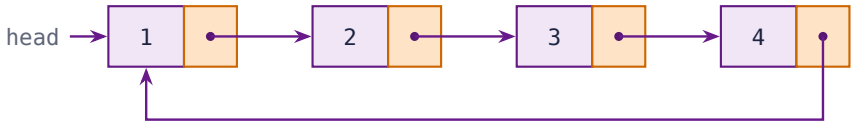


Figure 7.2 A circular linked list: the last node points back to the first.

Traversal (the notes' code): process the head, then walk until you come back to it.

```c
process(head);
p = head->next;
while (p != head)
    process(p), p = p->next;
```

There is no NULL to stop at — the stopping test is `p != head`.

Complexities with only a `head` pointer (notes):

- Traversal: **O(n)** in all cases.
- Insertion and deletion: **at the beginning O(n)** (all cases) — because the **last node must be updated to point to the new first node, and you must walk to the last node to do it**; **in between O(n)** average; **at the end O(n)** (all cases).

SHORTCUT

keep a pointer to the **last** node instead of the first. Then insertion at the beginning **and** the end are both O(1) (`last->next` *is* the first node). This is why circular lists are used for queues.

7.4 Doubly linked list

KEY POINT

Each node has two pointers: previous and next. The previous pointer points to the previous node in addition to the next pointer pointing to the next node.

```c
struct DNode {
    struct DNode *prev;
    struct DNode *next;
    int data;
};
```

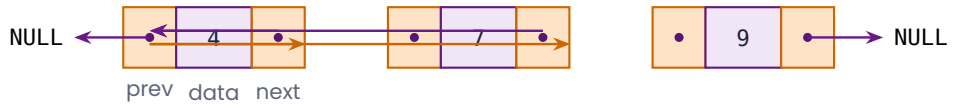


Figure 7.3 A doubly linked list: every node has a previous and a next pointer.

Complexities (notes, with a head pointer only): traversal **O(n)**; insertion/deletion **beginning O(1)**, **in between O(n)** (to find the place), **end O(n)** (to reach the last node).

What the doubly list buys you:

- You can go **backwards**.
- If you are **already given a pointer to the node to delete**, you can delete it in **O(1)** (you know both neighbours): `p->prev->next = p->next; p->next->prev = p->prev;`. In a singly list you would need the predecessor (O(n)).
- With a **tail pointer** too, insertion and deletion at **both ends are O(1)**.
- Cost: **extra memory** for the second pointer, and more pointers to update.

Comparison table (singly with head only unless stated)

	Singly	Singly + tail	Doubly + head + tail	Circular, last-pointer
Insert at front	O(1)	O(1)	O(1)	O(1)
Insert at end	O(n)	**O(1)**	O(1)	O(1)
Delete at front	O(1)	O(1)	O(1)	O(1)
Delete at end	O(n)	**O(n)** (need predecessor)	**O(1)**	O(n)
Delete a node given its pointer	O(n)	O(n)	**O(1)**	O(n)
Search	O(n)	O(n)	O(n)	O(n)

7.5 Linked list vs array

	Array	Linked list
Access `i`-th element	**O(1)**	O(n)
Insert/delete at front	O(n) (shift)	**O(1)**
Size	fixed (or reallocate)	grows freely
Extra memory	none	one pointer per node
Cache friendliness	good	poor
Binary search	possible	**not possible** (no random access)

7.6 The code patterns GATE asks (added)

Use these on a list `1 → 2 → 3 → 4 → 5`. All outputs below were checked by running.

(a) Counting nodes

```c
int count(struct Node *h) { return h ? 1 + count(h->next) : 0; }
```

Recursion depth = n.

(b) Printing in reverse with recursion

```c
void rp(struct Node *h) { if (!h) return; rp(h->next); printf("%d ", h->data); }
```

Prints **5 4 3 2 1** (print *after* the recursive call — Chapter 5).

(c) Reversing the list — iterative (memorise this!)

```c
struct Node *rev(struct Node *h) {
    struct Node *prev = NULL;
    while (h) {
        struct Node *next = h->next;  // 1. remember the rest
        h->next = prev;               // 2. flip the arrow
        prev = h;                     // 3. advance prev
        h = next;                     // 4. advance h
    }
    return prev;
}
```

Trace on `1→2→3→4` — state after each round:

Round	`prev` list	remaining `h`
start	NULL	1 2 3 4
1	1	2 3 4
2	2 1	3 4

Round	prev list	remaining h
3	3 2 1	4
4	4 3 2 1	NULL

Return prev: 4→3→2→1. **O(n) time, O(1) extra space.**

(d) Reversing recursively

```c
struct Node *r(struct Node *h) {
    if (!h || !h->next) return h;
    struct Node *rest = r(h->next);   // reverse the tail
    h->next->next = h;                // make the next node point back to h
    h->next = NULL;
    return rest;
}
```

O(n) time, O(n) stack space.

(e) Finding the middle — slow and fast pointers

```c
struct Node *mid(struct Node *h) {
    struct Node *s = h, *f = h;
    while (f && f->next) { s = s->next; f = f->next->next; }
    return s;
}
```

Fast moves two steps per slow's one. For 5 nodes → value **3**; for 6 nodes → value **4** (the second of the two middle nodes).

(f) Detecting a cycle (Floyd)

Same two pointers; if `s == f` at any time, there is a cycle. Time O(n), space O(1).

(g) Swapping data of adjacent pairs

```c
void swp(struct Node *h) {
    while (h && h->next) {
        int t = h->data; h->data = h->next->data; h->next->data = t;
        h = h->next->next;
    }
}
```

On 1 2 3 4 5 → **2 1 4 3 5**.

(h) Deleting every second node

```c
void alt(struct Node *h) {
    while (h && h->next) { struct Node *t = h->next; h->next = t->next; h = h->next; }
}
```

On 1 2 3 4 5 6 → **1 3 5**.

(i) Delete a given node when you only have a pointer to it (not the tail)

Copy the next node's data into it, then bypass the next node: O(1).

```c
p->data = p->next->data;
p->next = p->next->next;
```

It fails if `p` is the last node.

(j) Insert into a sorted list

Walk until the right place, then rewire: **worst case O(n)**, best O(1). (Same for deleting a value.)

Complexity of a typical "insert at end" in each variant

- Singly, head only: **O(n)**.
- Singly, head + tail: **O(1)**.
- Doubly, head + tail: **O(1)**.

7.7 Practice set — linked lists

Q1 MCQ · 1 In a singly linked list with only a head pointer, the time to **insert a new node at the end** is
(A) O(1) **(B)** O(log n) **(C)** O(n) **(D)** $O(n^2)$ — **Answer: (C).**

Q2 MCQ · 1 In a **circular** singly linked list with only a pointer `head` to the first node, deleting the first node takes
(A) O(1) **(B)** O(n) **(C)** O(log n) **(D)** depends on the data — **Answer: (B).** The last node's `next` must be repaired, so you must walk to it.

Q3 MSQ · 2 You are given a pointer `p` to a node (not the last) in a **singly** linked list and nothing else. Which operation can be done in **O(1)**?
(A) delete the node before `p` **(B)** delete `p` itself by copying **(C)** insert a new node *before* `p` by copying **(D)** reverse the list
Answer: (B) and (C) — if both appear as separate options, both are O(1) using the copy trick (insert-before: create new node, copy `p`'s data into it, link it after `p`, put the new value in `p`). **(A)** needs the predecessor: O(n).

Q4 NAT · 1 What is the output of `swp` on `1 2 3 4 5` (digits printed with spaces)? **Answer: 2 1 4 3 5.**

Q5 NAT · 1 For the list `1→2→3→4→5→6`, what does `mid()` return (the data)? **Answer: 4.**

Q6 MCQ · 2 After the code `alt(h)` on `1→2→3→4→5→6`, the list is
(A) 1 3 5 **(B)** 2 4 6 **(C)** 1 2 3 **(D)** 1 3 5 6 — **Answer: (A).**

Q7 NAT · 2 In the iterative `rev` on `1→2→3→4`, what are the lists `prev` and `h` after the **second** iteration?
Answer: `prev = 2→1, h = 3→4`.

Q8 MCQ · 1 Which uses **O(1) extra space** and O(n) time to reverse a list?
(A) iterative `rev` **(B)** recursive `r` **(C)** both **(D)** neither — **Answer: (A).** The recursive one uses O(n) stack.

Q9 MSQ · 2 Which statements about doubly linked lists are true?
(A) Deleting a node given a pointer to it is O(1). **(B)** Each node needs more memory than in a singly list. **(C)** Searching an unsorted doubly list is O(log n). **(D)** With head and tail pointers, inserting at both ends is O(1).
Answer: A, B, D.

Q10 MCQ · 1 Binary search on a sorted **linked list** takes
(A) O(log n) **(B)** O(n) **(C)** O(n log n) **(D)** O(1)
Answer: (B). You cannot jump to the middle in O(1), so even a "binary" search walks O(n) in total.

Q11 MCQ · 1 `int count(node *h){ return h ? 1+count(h->next) : 0; }` on a list of n nodes has stack depth
(A) O(1) **(B)** O(log n) **(C)** O(n) **(D)** $O(n^2)$ — **Answer: (C).**

Q12 MCQ · 2 A node is inserted after `p` with: `p->next = n; n->next = p->next;` — what happens?
Answer: `n->next` becomes `n` itself (because `p->next` was just overwritten with `n`). This creates a **self-loop** and loses the rest of the list. Correct order: `n->next = p->next; p->next = n;`.

Self-check

SELF-CHECK

- ☐ I can say which operations are O(1) vs O(n) for singly, doubly and circular lists.
- ☐ I can write and trace iterative list reversal.
- ☐ I use slow/fast pointers for the middle and for cycles.

CHAPTER 8

Stacks, Queues & Expression Evaluation

8

TIER 2

EXAM WEIGHT

Exam weight: 0.94 marks per paper; asked in 7 of the last 9 papers, and in 2024, 2025 and 2026.
Styles (2018–2026): operation-sequence puzzles — push/pop/enqueue order, capacity, which output orders are possible (6 questions, mostly 2-markers: 3 NAT + 3 MSQ); implementing a queue with stacks (or the reverse) (1); expression evaluation / conversion; stack-vs-heap memory (Chapter 2).
All of the notes' content is here: stack permutations, array/linked implementations, queue types, queue-using-2-stacks, stack-using-2-queues, bracket balancing, 2 stacks in 1 array, infix/prefix/postfix.

8.1 Stack — Last In, First Out (LIFO)

DEFINITION

A stack is a First-In-Last-Out data structure in which data can be inserted and extracted from only one side. At any time only a single element is accessible: the top element. (notes)

Think of a pile of plates: you put a plate on top and take a plate from the top.

Operation	Meaning	Time
push(x)	put x on top	O(1)
pop()	remove (and return) the top	O(1)
top()/peek()	look at the top without removing	O(1)

Real-life uses: function calls (the call stack, Ch 2/5), undo, bracket matching, expression evaluation, DFS, reversing.

Implementation 1 — array

```c
int stack[capacity];
int top = -1;                        // -1 means empty

push(x) {
    top += 1;
    if (top == capacity) {           // overflow
        top -= 1;
        raise StackFullError;
    }
    stack[top] = x;
}

pop() {
    if (top == -1) raise StackEmptyError;   // underflow
    x = stack[top];
    top -= 1;
    return x;
}
```

Implementation 2 — linked list (push/pop at the head)

```c
LLNode *stack;                       // head pointer = top

push(x) {
    LLNode *new = malloc(...);
    new->data = x;
    new->next = stack;
    stack = new;
}

pop() {
    if (stack == NULL) raise StackEmptyError;
    x = stack->data;
    stack = stack->next;
    return x;
}
```

KEY POINT

Push and Pop are both O(1) in both implementations. For the array version, **the size must be fixed in advance**; the linked version grows freely (but uses extra pointer memory).

8.2 Stack permutations (very important)

DEFINITION

Given a sequence of push operations (input order), a valid stack permutation is the order in which the elements are popped off the stack.

You push in a fixed order (say 1, 2, 3, 4) but you may pop at any time. Which output orders can you get?

Example (notes). Input 1, 2, 3, 4. Which is **not** a valid stack permutation?

Option	Sequence of operations that produces it	Valid?
(A) 1, 2, 3, 4	P(1) Pop, P(2) Pop, P(3) Pop, P(4) Pop	✓
(B) 2, 1, 3, 4	P(1) P(2) Pop(2) Pop(1) P(3) Pop P(4) Pop	✓
(C) 4, 2, 3, 1	to output 4 first you must push 1,2,3,4; the stack is `1 2 3 4`, pop 4; now top is 3, so **next pop gives 3, not 2**	✗ — **no sequence possible**
(D) 3, 4, 2, 1	P(1) P(2) P(3) Pop(3) P(4) Pop(4) Pop(2) Pop(1)	✓

ANSWER (C).

Counting valid permutations

FORMULA

Number of valid stack permutations of n elements = Catalan number = ${}^{2n}C_n$ / (n+1).
Number of invalid permutations = n! − ${}^{2n}C_n$/(n+1).

n	1	2	3	4	5	6	7
Valid	1	2	5	14	42	132	429

(All verified by brute-force simulation.) For n = 4: invalid = 4! − 14 = **10**.

The shortcut for checking a permutation in 10 seconds — "no 3-1-2 pattern"

KEY POINT

An output order (for input 1, 2, …, n) is invalid exactly when it contains three numbers `a < b < c` that appear in the output in the order `c … a … b` (a "312" pattern: high, then low, then middle).

Why? When `c` comes out, `a` and `b` are both already pushed and both lie below `c`, with `b` above `a`. After `c` is popped, `b` is on top of `a`, so `b` must leave before `a`.

Check on the notes' example: `4, 2, 3, 1` → take `4, 2, 3` = (high, low, mid) → **invalid** ✓. `3, 4, 2, 1` → is there high-low-mid? 3,2,? no number between 2 and 3 after. 4,2,? no → valid ✓.

`3, 4, 5, 1, 2` (n=5) → 3,1,2 is a 312 pattern → **invalid**. `2, 5, 4, 3, 1` → valid. `1, 5, 2, 3, 4` → 5,2,3 → invalid.

Another useful fact: the number of **ways to interleave pushes and pops** to push 1..n and pop all is also the Catalan number (e.g., n = 3 → 5).

8.3 Queue — First In, First Out (FIFO)

DEFINITION

A queue is a first-in-first-out data structure in which data is inserted from one end and extracted from the other end. (notes)

A queue keeps two pointers:

- **front** — points to the next element to be extracted.
- **rear** — points to the place where the next/last element goes (the notes use `r` as the index of the last inserted element).

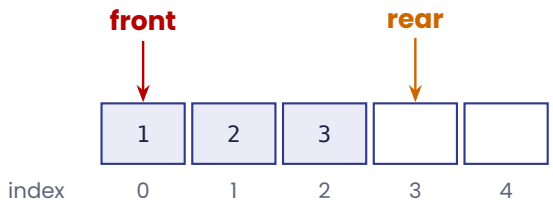


Figure 8.1 An array-based queue with its `front` and `rear` pointers.

Operation	Meaning
Enqueue(x)	insert x at the rear
Dequeue()	remove and return the element at the front

The only possible output order of a queue is the **input order** (just one permutation).

Implementation 1 — array, linear queue

```c
int Queue[capacity];  int f = -1, r = -1;

enqueue(x) {
    if (r + 1 == capacity) raise QueueFullError;
    r = r + 1;
    Queue[r] = x;
    if (r == 0) f = 0;           // first element inserted
}

dequeue() {
    if (f == -1) raise QueueEmptyError;
    x = Queue[f];
```

```
    if (f == r) f = r = -1;      // queue became empty
    else        f = f + 1;
    return x;
}
```

Problem: the front slots freed by dequeue are **never reused**, so the queue can say "full" while there is free space at the beginning.

Implementation 2 — array, circular queue

Treat the array as a ring using `% capacity`:

```c
enqueue(x) {
    if (r != -1 && (r + 1) % capacity == f) raise QueueFullError;
    r = (r + 1) % capacity;
    Q[r] = x;
    if (f == -1) f = 0;
}

dequeue() {
    if (f == -1) raise QueueEmptyError;
    x = Q[f];
    if (f == r) f = r = -1;
    else        f = (f + 1) % capacity;
    return x;
}
```

KEY POINT

Elements are always between front and rear (going around the ring if needed).

Rules to remember:

State	Linear (f, r) test	Circular test
Empty	`f == -1`	`f == -1`
Full	`r + 1 == capacity`	`(r+1) % capacity == f`
Number of elements	`r - f + 1`	`(r - f + capacity) % capacity + 1`

*(Many textbooks use the other convention — `front == rear` means empty and one cell is kept unused so the queue holds `capacity - 1` elements. If a GATE question gives its own code, follow **its** convention.)*

Worked example (verified by simulation). Circular queue, capacity 5. Operations: `enq1, enq2, enq3, enq4, deq, deq, enq5, enq6, enq7, deq, enq8`.

Step	Result	f	r
enq1–enq4	ok	0	3
deq → 1, deq → 2		2	3
enq5, enq6, enq7	ok (r wraps: 4, 0, 1)	2	1

Step	Result	f	r
deq → 3		3	1
enq8	ok (r = 2)	3	2

Final array `[6, 7, 8, 4, 5]`, `f = 3`, `r = 2` — queue order 4, 5, 6, 7, 8.

Implementation 3 — linked list

Keep **two** pointers: `f` (head) and `r` (tail).

```c
LLNode *f, *r;

enqueue(x) {
    LLNode *n = malloc(...);
    n->data = x;                    // n->next = NULL
    if (r != NULL) r->next = n;
    r = n;                          // (notes: r = r->next)
    if (f == NULL) f = n;
}

dequeue() {
    if (f == NULL) raise QueueEmptyError;
    x = f->data;
    f = f->next;
    if (f == NULL) r = NULL;
    return x;
}
```

KEY POINT

Enqueue and dequeue are both O(1) for all these implementations.

8.4 Queue using 2 stacks (a GATE favourite)

Use `s1` for input, `s2` for output.

```c
enqueue(x) { push(s1, x); }

dequeue() {
    if (isEmpty(s2)) {
        if (isEmpty(s1)) raise QueueEmptyError;
        while (!isEmpty(s1))
            push(s2, pop(s1));        // reverses the order
    }
    x = pop(s2);
    return x;
}
```

Why it works: moving all of `s1` into `s2` reverses the order, so the *oldest* element ends up on top of `s2`.

	Time
Enqueue	**O(1)** in all cases
Dequeue	**O(n) in the worst case** (when `s2` is empty and `s1` holds n items); but **amortized O(1)** because each element is moved at most once

Counting operations (typical NAT). Every element that is eventually dequeued is **pushed on s1, popped from s1, pushed on s2, popped from s2 = 4 stack operations**. Example (verified): enqueue 1,2,3,4; dequeue; enqueue 5; dequeue ×4 → all 5 elements are dequeued → 5 × 4 = **20** operations.

RULE

only move from `s1` to `s2` when `s2` is **empty**. Moving earlier breaks the order.

8.5 Stack using 2 queues

```c
queue q1, q2;  int curr = 1;     // curr = which queue currently holds the data

push(x) {
    if (curr == 1) enqueue(q1, x);
    else           enqueue(q2, x);
}

pop() {
    if (isEmpty(q1) && isEmpty(q2)) raise StackEmptyError;
    if (curr == 1) {
        while (1) {
            x = dequeue(q1);
            if (!isEmpty(q1)) enqueue(q2, x);   // not the last one → move it
            else break;                          // last one = the answer
        }
        curr = 2;  return x;
    } else { /* symmetric */ }
}
```

The last-in element is at the *back* of the queue; to get it we move everything before it into the other queue.

KEY POINT

Push = O(1). Pop = O(n) in all cases. (Alternatively you can make push O(n) and pop O(1).)

8.6 Checking balanced brackets with a stack

```c
Bal(s) {
    Stack st;
```

```
    for (char c in s) {
        if (c == '(' || c == '[' || c == '{')
            push(st, c);
        if (c == ')' || c == ']' || c == '}') {
            if (isEmpty(st) || !isMatching(c, top(st)))
                return "Unbalanced";
            pop(st);
        }
    }
    if (!isEmpty(st)) return "Unbalanced";
    return "Balanced";
}
```

- An **opening** bracket is pushed; a **closing** bracket must match the **top**.
- At the end the stack must be **empty**.
- Time O(n), extra space up to O(n) (maximum stack size = maximum nesting depth).

Examples: `{[()]}` balanced; `([)]` unbalanced (when `)` arrives the top is `[`); `((` unbalanced (stack not empty at end).

8.7 Two stacks in one array

Start the two stacks from **opposite ends** and let them grow towards each other.

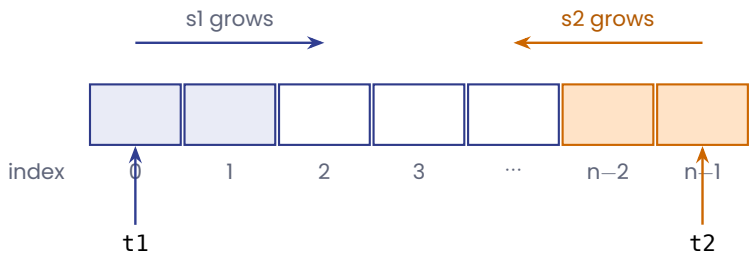


Figure 8.2 Two stacks sharing one array, growing towards each other.

- **Initially:** `t1 = -1`, `t2 = n`.
- After one element in each: `t1 = 0`, `t2 = n - 1`.
- **Stack full (the whole array is used):** `t1 + 1 == t2` (equivalently `t1 == t2 - 1`).
- Total capacity is shared: either stack can use up to `n - (elements of the other)` — **no space is wasted** (that is the whole point).

For **more than two** stacks in a single array you must **divide the array into parts** (each stack gets its own region with its own top pointer), and a region can fill up while others are empty.

Notation	Operator is…	Also called	Example for `a + b`

8.8 Infix, prefix and postfix notation

The names are based on the **position of the operator**:

Notation	Operator is…	Also called	Example for `a + b`
Infix	between operands	–	`a + b`
Prefix	**before** operands	**Polish**	`+ a b`
Postfix	**after** operands	**Reverse Polish**	`a b +`

Prefix and postfix need **no brackets and no precedence rules** — the order is built in. That is why compilers convert to them.

Precedence and associativity

`^` (highest, **right to left**), then `* /` (left to right), then `+ -` (left to right). Brackets override everything.

Method to convert by hand (always works)

1. **Fully bracket** the infix expression using precedence/associativity.
2. For **postfix**: move each operator to just **after its right bracket's second operand**. For **prefix**: move it to just **before** its first operand.

Worked examples from the notes (all verified)

Example 1. `(a + b*c – d) / (e*f)`

- Postfix: **`a b c * + d – e f * /`**
- Prefix: **`/ – + a * b c d * e f`**

Example 2. `a + b*c*d^e^f – g/h + a*b/c`
(Here `^` is right-associative: `d^e^f = d^(e^f)`; `+ –` left-to-right.)

- Postfix: **`a b c * d e f ^ ^ * + g h / – a b * c / +`**
- Prefix: **`+ – + a * * b c ^ d ^ e f / g h / * a b c`**

Example 3. `x + y – z / e * a ^ b / c + d`

- Prefix: **`+ – + x y / * / z e ^ a b c d`**
- Postfix: `x y + z e / a b ^ * c / – d +`

Example 4 (function call). `fn(a, b, c)` with the **comma treated as a left-to-right associative operator**:

- Postfix: **`a b , c , fn`**
- Prefix: **`fn , , a b c`**

More: `a*(b+c)/d` → postfix `a b c + * d /`, prefix `/ * a + b c d`. `a – b – c` → postfix `a b – c –`, prefix `– – a b c`. `2^3^2` → postfix `2 3 2 ^ ^` ($= 2^9 = 512$).

Infix → Postfix with a stack (the algorithm in the notes)

Scan the input left to right:

INFIX TO POSTFIX: SCAN THE INPUT LEFT TO RIGHT

`if operand`	→ output it
`if '('`	→ push it on the stack
`if ')'`	→ pop (and output) until `(` is popped; discard both brackets
`if operator`	repeat the following:
	→ while *precedence(top)* > *precedence(current)*: pop and output
	→ if same precedence: pop and output (*except* `^`, which is right-associative: push)
	→ then push the current operator
`at the end`	→ pop and output everything left on the stack

(Implementation note: the loop keeps popping while the top has **higher, or equal-with-left-associativity** precedence than the incoming operator, stopping at `(`.)

Trace of `a + b * c - d`:

Symbol	Stack (bottom→top)	Output
a		a
+	+	a
b	+	a b
*	+ * (higher than +, so push)	a b
c	+ *	a b c
−	incoming −: pop *, pop + (equal), push −	a b c * +
d	−	a b c * + d
end	pop	**a b c * + d −**

Maximum stack size. For `a + b*(c^d - e)^(f + g*h) - i` the postfix is `a b c d ^ e - f g h * + ^ * + i -`; the stack (counting `(`) reaches **6** at `g*h` (`+ * ^ ( + *`). Questions of this kind ask for the **maximum number of stack elements** — simulate and count.

Postfix evaluation

POSTFIX EVALUATION: SCAN LEFT TO RIGHT

`if operand`	→ push it
`if operator`	→ pop two operands (the first popped is the *right* operand), apply the operator, push the result
`at the end`	→ the final value on the stack is the answer

Example: `8 2 3 ^ / 2 3 * + 5 1 * −`
`2^3 = 8`; `8/8 = 1`; `2*3 = 6`; `1+6 = 7`; `5*1 = 5`; `7−5 = 2` → **2**.

Example: `5 3 + 2 * 4 /` → `8`, `16`, `4` → **4**.

PITFALL

for `−` and `/` the **order of the two popped operands matters**. If the stack holds `... a b`, then `b` is popped first; the result is `a − b`, not `b − a`.

Prefix evaluation

Scan **right to left**; push operands; on an operator pop two (first popped = **left** operand), apply, push. `− * 3 + 2 4 / 8 2` → `3*(2+4) − 8/2 = 18 − 4 = 14`.

8.9 Practice set — stacks, queues & expressions (GATE style)

Q1 NAT · 1 Number of valid stack permutations of 5 distinct pushed elements (pushed in a fixed order)? **Answer: 42.** ($^{10}C_5/6$.)

Q2 MSQ · 2 Elements 1, 2, 3, 4 are pushed in this order, with pops allowed at any time. Which output sequences are possible?
(A) 4 3 2 1 **(B)** 3 1 4 2 **(C)** 2 4 3 1 **(D)** 1 3 4 2 **(E)** 4 1 2 3
Answer: A, C, D. `3 1 4 2` has 3-1-2; `4 1 2 3` has 4-1-2 → invalid.

Q3 NAT · 2 Elements 1–5 are pushed in order. How many of the following outputs are valid: `3 4 5 1 2`, `2 5 4 3 1`, `1 5 2 3 4`, `5 4 3 2 1`, `3 2 5 4 1`?
Answer: 3 (`2 5 4 3 1`, `5 4 3 2 1`, `3 2 5 4 1`).

Q4 NAT · 2 The number of **invalid** permutations for 4 elements? **Answer: 24 − 14 = 10.**

Q5 NAT · 2 Circular array queue, capacity 5, using the notes' code. Operations: `enq1, enq2, enq3, enq4, deq, deq, enq5, enq6, enq7, deq, enq8`. What are the final values of `f` and `r`? Give `f + r`.
Answer: f = 3, r = 2 → 5.

Q6 NAT · 2 A queue is implemented with two stacks as in §8.4. Operations: `enq 1,2,3,4; deq; enq 5; deq; deq; deq; deq`. Total number of **push and pop operations on the two stacks** performed?
Answer: 20.

Q7 MCQ · 1 In the 2-stack queue, worst-case time of a single dequeue is
(A) O(1) **(B)** O(log n) **(C)** O(n) **(D)** O(n^2) — **Answer: (C).**

Q8 MCQ · 1 Stack implemented with two queues (§8.5): push O(1), pop is
(A) O(1) **(B)** O(n) **(C)** O(log n) **(D)** O(n^2) — **Answer: (B).**

Q9 NAT · 1 The postfix expression `2 3 4 * + 5 −` evaluates to? **Answer: 9.**

Q10 NAT · 2 Evaluate the postfix expression `6 2 / 3 − 4 2 * +`. **Answer: 8.** (3, 0, 8, 8.)

Q11 MCQ · 1 Postfix of `a * (b + c) / d` is
(A) `a b c + * d /` **(B)** `a b * c + d /` **(C)** `a b c * + d /` **(D)** `a * b c + / d`
Answer: (A).

Q12 MCQ · 1 Prefix of `(a + b) * (c − d)`: **(A)** `* + a b − c d` **(B)** `+ a b * − c d` **(C)** `* a + b c − d` **(D)** `a b + c d − *`
Answer: (A).

Q13 NAT · 2 Prefix expression `+ * 2 3 / 8 4` evaluates to? **Answer: 8.** (`2*3 + 8/4 = 6 + 2`.)

Q14 NAT · 2 Maximum number of stack entries (including `(`) while converting `a*(b+(c*(d+e)))` to postfix with the notes' algorithm? **Answer: 7.**
(*, (, +, (, *, (, + all present when `e` is read.)

Q15 MCQ · 1 Which data structure is the natural choice for matching brackets? **Answer: stack.**

Q16 MCQ · 2 In a stack implemented with an array where `top = −1` means empty, after the operations `push 1, push 2, pop, push 3, push 4, pop, pop` the value of `top` is? **Answer: 0.** (Elements: after the sequence 1 remains → top = 0.)

Q17 MCQ · 2 Two stacks share one array of size 10 (stack 1 from the left, stack 2 from the right). Stack 1 holds 4 elements and stack 2 holds 5. How many more elements can be pushed in total? **Answer: 1.** (`t1 = 3`, `t2 = 5`; full when `t1 + 1 == t2`; only index 4 is free.)

Q18 MSQ · 1 For a **queue** with elements inserted in order 1, 2, 3: which outputs are possible? **Answer: only 1, 2, 3.**

Self-check

SELF-CHECK

- ☐ I can test a stack output in seconds with the "312" rule and count valid ones with Catalan numbers.
- ☐ I know the circular-queue full/empty tests from the code I am given.
- ☐ I can convert by hand using full bracketing, and evaluate postfix with operand order right.
- ☐ I know queue-with-2-stacks: enqueue O(1), dequeue O(n) worst, 4 ops per element.

CHAPTER 9

Binary Trees & Traversals

TIER 2

EXAM WEIGHT

Exam weight: 0.83 marks per paper; asked in 6 of the last 9 papers, and in 2025 and 2026.
Styles (2018–2026): node / leaf / height properties of full & complete binary trees (4 questions: 2 NAT, 1 MCQ, 1 MSQ); **traversal-based reconstruction or tracing** (3 questions).
This chapter contains every topic from the notes: types of trees, node-count formulas, min/-max height, array and pointer representation, counting trees, traversals, and building a tree from two traversals.

9.1 From scratch: what is a tree?

A **tree** is a way of storing data in **levels**, like a family tree or a folder structure. There is one top node, the **root**; each node can have **children**; a node with no children is a **leaf**.



Figure 9.1 A binary tree with its levels marked.

A **binary tree** is a tree in which **every node has at most 2 children**: a **left child** and a **right child**. (Each node can have 0, 1 or 2 children.)

Vocabulary

Term	Meaning
Root	topmost node
Leaf	node with 0 children
Internal node	node with at least 1 child

Term	Meaning
Parent / child / sibling	the usual family meaning
Edge	a link between parent and child
Depth (level) of a node	**number of edges from the root to that node** (root has depth 0)
Height of a node	**number of edges on the longest path from that node down to a leaf** (a leaf has height 0)
Height of the tree	**height of the root**
Subtree	a node together with all its descendants

GATE CONVENTION

height is measured in **edges**, so a tree with a single node has **height 0**, and an empty tree has height -1. (GATE 2026 stated this definition explicitly. If a question counts levels or nodes instead, it will say so.)

9.2 Special kinds of binary trees

Type	Definition (notes)
Full binary tree (also *strict*, *proper*, *2-tree*)	**Every node has 0 or 2 children** (never exactly 1)
Complete binary tree (CBT)	**All levels except the last are completely filled; the last level is filled from left to right**
Perfect binary tree	A complete tree **where even the last level is full**

Examples (shapes):

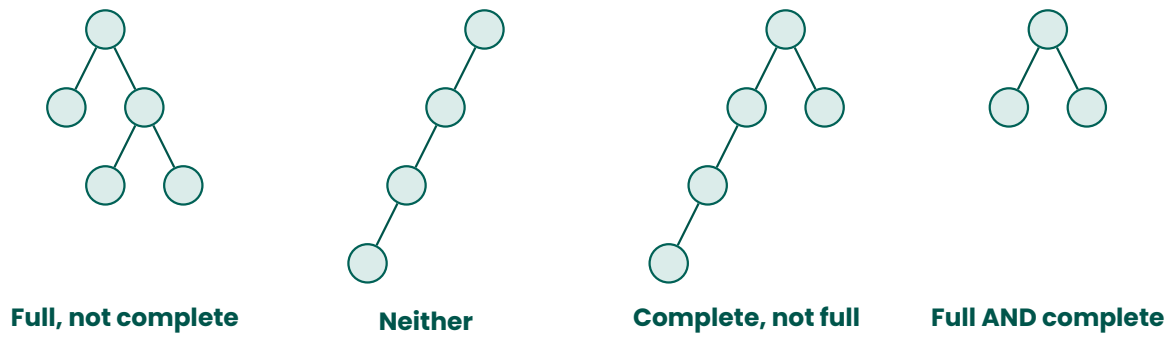


Figure 9.2 Full versus complete binary trees.

(From the notes, the four example trees were: Full ✓/Complete ✗; Full ✗/Complete ✗; Full ✗/Complete ✓; Full ✓/Complete ✓.)

KEY POINT

Perfect ⊂ Complete. A perfect tree is both full and complete. A complete tree need not be full (one node may have only a left child). A full tree need not be complete.

9.3 Formulas for counting nodes (the heart of the NAT questions)

Let n_0 = number of **leaves** L, I_1 = nodes with exactly **1** child, I_2 = nodes with exactly **2** children.

Fact 1 — leaves vs two-child nodes (any binary tree)

FORMULA

$$\mathbf{L} = I_2 + 1$$

Reason: count edges two ways. A tree with n nodes has n − 1 edges. Each node except the root has one incoming edge. Outgoing edges = $I_1 + 2 \cdot I_2$. So $n - 1 = I_1 + 2I_2$, and $n = L + I_1 + I_2$. Subtract: $L - 1 = I_2$.

Fact 2 — total number of nodes

FORMULA

$$\mathbf{n} = I_2 + I_1 + \mathbf{L} = I_2 + I_1 + (I_2 + 1) = \mathbf{2} \cdot I_2 + I_1 + 1$$

Special case — full binary tree (I_1 = 0)

KEY POINT

$\mathbf{n} = \mathbf{2} \cdot I_2 + \mathbf{1}$, and $\mathbf{n} = \mathbf{2L} - \mathbf{1}$ (a full tree with L leaves has 2L − 1 nodes).

Special case — K-ary tree (every internal node has exactly K children)

KEY POINT

$\mathbf{L} = \mathbf{I} \cdot (\mathbf{K} - \mathbf{1}) + \mathbf{1}$ and $\mathbf{n} = \mathbf{K} \cdot \mathbf{I} + \mathbf{1}$ (I = number of internal nodes)

Check K = 2: L = I + 1 ✓.

Perfect and complete trees — height vs size

NOTE

n = 2^{h+1} − 1, leaves = 2^h.
Complete tree of height h: minimum nodes = 2^h, maximum nodes = 2^{h+1} − 1.
Maximum nodes at level l: 2^l (root = level 0).

Notes' worked example — perfect tree with n nodes:

(1) *Number of leaves?* $I_2 + L = n$ and $L = I_2 + 1 \Longrightarrow 2I_2 + 1 = n \Longrightarrow I_2 = (n-1)/2 \Longrightarrow$ **L = (n+1)/2**.
(2) *Height?* $n = 2^{h+1} - 1 \Longrightarrow n + 1 = 2^{h+1} \Longrightarrow$ **$h = \log_2(n+1) - 1$**.

Notes' worked example — complete tree of height h:
(1) *Minimum nodes?* A complete tree of height h must have levels 0…h−1 full (2^h − 1) plus at least one node on the last level $\Longrightarrow$ **2^h**.
(2) *Maximum nodes?* The perfect tree $\Longrightarrow$ **2^{h+1} − 1**.

Any binary tree with n nodes — min and max height

KEY POINT

Minimum height $= \lceil \log_2(n+1) \rceil - 1 = \lfloor \log_2 \text{ n} \rfloor$ (the tree is as "bushy" as possible — complete).
Maximum height $=$ **n — 1** (a straight chain / "skewed" tree).

(From the notes: `n = 2^{h+1} - 1` $\Longrightarrow$ `h = log(n+1) - 1` for the minimum, rounded up; maximum `n - 1`.)

Complete binary tree with n nodes (very commonly asked; verified by enumeration)

Quantity	Value
Height	$\lfloor \log_2 \mathbf{n} \rfloor$
Leaves	$\lceil n/2 \rceil$
Internal nodes	$\lfloor n/2 \rfloor$
Nodes with exactly one child	1 if `n` is even, 0 if `n` is odd

9.4 Representing a binary tree in memory

9.4.1 Array representation — only suitable for complete binary trees

Store the tree **level by level, left to right** in an array. No pointers needed, because positions encode the relationships.

0-based indexing (index i $\geq$ 0):

> **NOTE**
>
> left child = **2i + 1**, right child = **2i + 2**, parent = $\lceil i/2 \rceil$ **− 1** (equivalently $\lfloor$(i−1)/2$\rfloor$ for i > 0)

1-based indexing (index i $\geq$ 1):

> **NOTE**
>
> left child = **2i**, right child = **2i + 1**, parent = $\lfloor i/2 \rfloor$ (i > 1)

Example (notes): the tree

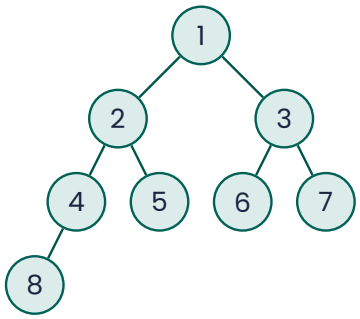


Figure 9.3 Example binary tree from the notes.

is stored as `[1, 2, 3, 4, 5, 6, 7, 8]`. Node 2 sits at 1-based index 2 → children at 4 and 5 ✓; node 4 at index 4 → left child at index 8 ✓.

> **NOTE**
>
> If the tree is **not** complete, the array has gaps (wasted space): a right-skewed tree with n nodes needs about $2^n - 1$ cells.

9.4.2 Node-based (pointer) representation

```c
struct Node {
    struct Node *lptr;
    int data;
    struct Node *rptr;
};
```

Each node stores the addresses of its left and right children (`NULL` if absent). Works for **any** shape.

9.5 Counting binary trees

Unlabelled (shape only)

FORMULA

The number of different binary tree structures with n unlabelled nodes = Catalan number = ${}^{2n}C_n$ / (n + 1).

n = 0,1,2,3,4,5,6 → 1, 1, 2, 5, 14, 42, 132.

Labelled (n distinct nodes/keys)

FORMULA

Number of binary trees with n distinct labelled nodes = $({}^{2n}C_n$ / (n+1)) × n!

Notes' example, n = 3: structures = ${}^{6}C_3/4 = 20/4 = \mathbf{5}$; labelled trees = $5 \times 3! = \mathbf{30}$.

The five 3-node shapes, written as `o(left, right)` with `.` for "no child":

#	Shape	Description
1	`o(o(o,.),.)`	left-left chain
2	`o(o(.,o),.)`	left child, then its right child (zig-zag)
3	`o(o,o)`	balanced (root with two children)
4	`o(.,o(o,.))`	right child, then its left child (zig-zag)
5	`o(.,o(.,o))`	right-right chain

KEY POINT

Same formula as the number of valid stack permutations (Ch 8) and the number of ways to bracket (parenthesise) a product.

9.6 Tree traversals

Visiting every node exactly once in some order. With **D** = visit the node (Data), **L** = traverse left subtree, **R** = traverse right subtree:

Traversal	Order	Memory trick
Preorder	**D L R**	root **pre**(before) children
Inorder	**L D R**	root **in** the middle
Postorder	**L R D**	root **post**(after) children
Level order	level by level (uses a queue)	

```c
void preorder (Node *t) { if (!t) return; visit(t); preorder(t->l);  preorder(t->r); }
void inorder  (Node *t) { if (!t) return; inorder(t->l);  visit(t);   inorder(t->r);  }
void postorder(Node *t) { if (!t) return; postorder(t->l); postorder(t->r); visit(t); }
```

Time **O(n)** (each node once); stack space **O(h)**.

Example.

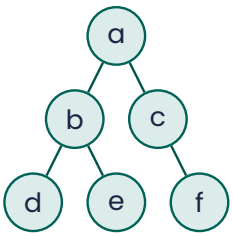


Figure 9.4 Example tree for traversal practice.

- Preorder: **a b d e c f**
- Inorder: **d b e a c f**
- Postorder: **d e b f c a**
- Level order: **a b c d e f**

Useful facts

- **Preorder:** the **first** element is the root. **Postorder:** the **last** element is the root.
- In **inorder**, everything to the **left** of the root is in the left subtree, everything to the right is in the right subtree.
- The leaves appear in the **same left-to-right order** in all three traversals.
- Inorder of a **BST** is **sorted** (Chapter 10).

9.7 Constructing a tree from traversals

KEY POINT

A tree can be constructed from Inorder + Preorder or Inorder + Postorder. (notes)

Reason: the root comes from pre/post; the inorder tells how many nodes go left and how many go right.

9.7.1 Preorder + Inorder (notes' example)

Preorder: `a b d g h e i c f j`
Inorder: `g d h b e i a f j c`

Algorithm:

1. First preorder element = root = `a`.
2. In inorder, split at `a`: left part `g d h b e i`, right part `f j c`.
3. The left part has 6 elements, so the next 6 preorder elements (`b d g h e i`) are the left subtree's preorder; the remaining (`c f j`) are the right subtree's preorder.
4. Recurse on both.

Left: pre `b d g h e i`, in `g d h b e i` → root `b`; in-left `g d h`, in-right `e i`. Then `d` is root of `g d h` (left `g`, right `h`); `e` is root of `e i` with right child `i`.
Right: pre `c f j`, in `f j c` → root `c`, left subtree `f j` with root `f`, right child `j`.

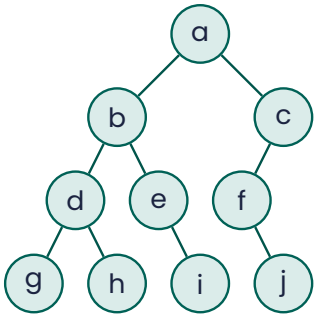


Figure 9.5 Tree reconstructed from a preorder and an inorder sequence.

In parenthesis notation: `a(b(d(g,h),e(.,i)),c(f(.,j),.))` (`k(L,R)`, a dot = empty).

9.7.2 Postorder + Inorder

Same idea, but the **root is the last** element of the postorder. Split inorder at the root; the first `|left part|` postorder elements form the left subtree.

9.7.3 Preorder + Postorder — ambiguous in general (notes)

Example: preorder `1 2 3`, postorder `3 2 1`. Every node has at most one child, and each single child can be left or right: **4 different trees** are possible:

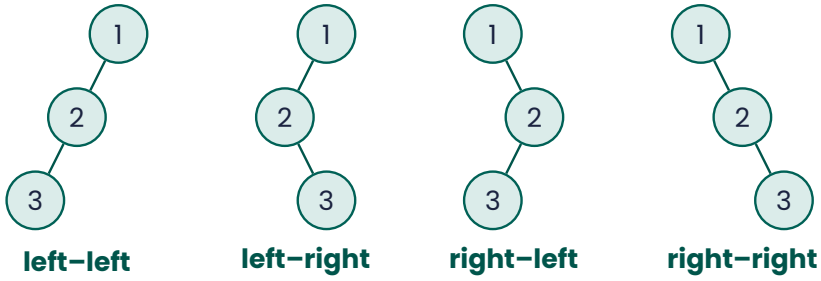


Figure 9.6 The four binary trees that share preorder 1 2 3 and postorder 3 2 1.

(left-left, left-right, right-left, right-right chains).

KEY POINT

Counting rule (verified by brute force for $n \leq 7$): the number of binary trees consistent with a given preorder **and** postorder = **2^k**, where **k = number of nodes that have exactly one child**.

9.7.4 Full binary tree: Pre + Post is enough (notes)

In a full tree every node has 0 or 2 children, so no ambiguity. Key fact:

DEFINITION

Whatever comes first in Preorder (D L R) comes last in Postorder (L R D) — it is the root.

Notes' example 1. Pre: `A D H L Z P C` Post: `H L D P C Z A`

- Root = `A`. The next preorder element `D` is the **root of the left subtree**. Find `D` in postorder: `H L D` — so the left subtree has 3 nodes (`H L D`), the right subtree is the rest `P C Z`.
- Left: pre `D H L`, post `H L D` → root `D` with leaves `H`, `L`.
- Right: pre `Z P C`, post `P C Z` → root `Z` with leaves `P`, `C`.

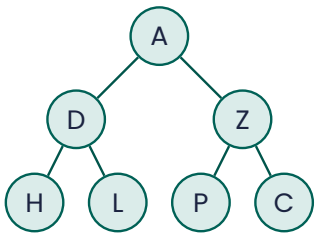


Figure 9.7 Tree reconstructed from preorder and postorder (full binary tree).

Notes' example 2. Pre: `a b c d f g e` Post: `c f g d b e a`

- Root `a`; next in pre is `b` = root of the left subtree; in post `b` ends the block `c f g d b` (5 nodes); right subtree = `e`.
- Left: pre `b c d f g`, post `c f g d b` → root `b`, its left child `c` (leaf), then `d` with leaves `f`, `g`.

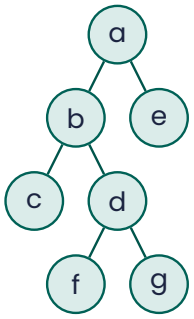


Figure 9.8 Tree reconstructed from preorder and postorder (full binary tree).

9.7.5 Complete binary tree: any one traversal is enough (notes)

Because the **shape** of a CBT with n nodes is fixed, one traversal is enough to fill in the labels.

Notes' example. Preorder: `1 9 5 4 2 6 8 7`, a CBT with 8 nodes.

- The CBT shape with 8 nodes has positions (numbered level-wise) 1…8. Its **preorder of positions** is `1, 2, 4, 8, 5, 3, 6, 7` (verified).
- Assign the given preorder values to those positions: pos1 = 1, pos2 = 9, pos4 = 5, pos8 = 4, pos5 = 2, pos3 = 6, pos6 = 8, pos7 = 7.
- Level order (array form): `[1, 9, 6, 5, 2, 8, 7, 4]`.

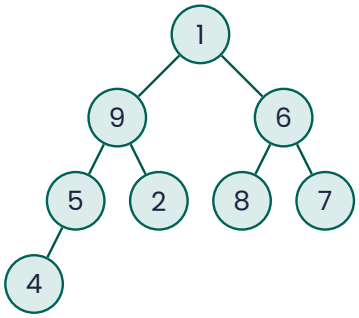


Figure 9.9 The tree whose level-order array is $[1, 9, 6, 5, 2, 8, 7, 4]$.

Summary: which traversals determine the tree?

Tree kind	Enough information
General binary tree	**Pre + In** or **Post + In** (or Level + In)
Full binary tree	Pre + Post is also enough
Complete binary tree	**Any single** traversal
BST	**Preorder alone** or **postorder alone** (Ch 10); inorder alone is *not* enough (it is just the sorted order)

9.8 Practice set — binary trees (GATE style)

Q1 NAT · 1 A full binary tree has 40 leaves. How many nodes does it have? **Answer: 79** ($2L - 1$).

Q2 NAT · 2 A binary tree has 20 nodes with two children and 15 nodes with exactly one child. How many nodes does it have in total? **Answer: 56.** (`L = 21; n = 20 + 15 + 21.`)

Q3 NAT · 1 A complete binary tree has 1000 nodes. How many leaves? **Answer: 500** ($\lceil 1000/2 \rceil$).

Q4 NAT · 1 Height (in edges) of a complete binary tree with 1000 nodes? **Answer: 9** ($\lfloor \log_2 1000 \rfloor = 9$).

Q5 NAT · 2 A tree in which every internal node has exactly 3 children has 12 internal nodes. How many nodes in total? **Answer: 37** (`3·12 + 1`). Leaves = 25.

Q6 MCQ · 1 A perfect binary tree has 255 nodes. Its height is **(A)** 6 **(B)** 7 **(C)** 8 **(D)** 9. **Answer: (B) 7.** (`2^{h+1} − 1 = 255.`)

Q7 NAT · 1 Minimum possible height of a binary tree with 20 nodes? **Answer: 4** ($\lfloor \log_2 20 \rfloor$). Maximum possible height? **19.**

Q8 MSQ · 2 Which statements are true for every non-empty binary tree?
(A) `L = I₂ + 1` **(B)** a full tree is always complete **(C)** a complete tree is always full **(D)** a perfect tree is always full and complete
Answer: A, D.

Q9 NAT · 2 Preorder of a binary tree: `P Q S T R`; inorder: `S Q T P R`. What is the postorder? Give the **last three** nodes.
Answer: root `P`; left (in): `S Q T` with pre `Q S T` → `Q(S,T)`; right: `R`. Postorder: `S T Q R P` → last three: **Q R P**.

Q10 NAT · 2 How many different binary trees (shapes) with 5 nodes exist? **Answer: 42.** Labelled with 5 distinct keys: $42 \times 120 =$ **5040**.

Q11 NAT · 2 Preorder `1 2 3 4` and postorder `4 3 2 1` of a binary tree. How many binary trees are possible? **Answer: 8** (every internal node has exactly one child: `k = 3`, `2³`).

Q12 MCQ · 2 A complete binary tree is stored in a 1-based array. The node at index 25 has its parent at index **(A)** 12 **(B)** 13 **(C)** 24 **(D)** 50. **Answer: (A) 12.** Its children (if present) would be at 50 and 51.

Q13 MCQ · 2 In a 0-based array representation of a complete binary tree, the right child of the node at index 7 is at index **(A)** 14 **(B)** 15 **(C)** 16 **(D)** 17. **Answer: (C) 16** (`2·7+2`).

Q14 NAT · 1 A complete binary tree with 25 nodes: how many nodes at the last level? **Answer: 10.** Levels 0–3 hold `1+2+4+8 = 15` nodes; remaining 10 are on level 4.

Q15 NAT · 2 Full binary tree: preorder `A B D E C F G`, postorder `D E B F G C A`. Find the number of **leaves**. **Answer: 4** (`D, E, F, G`).

Q16 MCQ · 1 Postorder of the tree `a(b(d,e),c(.,f))` is **(A)** d e b f c a **(B)** d b e a c f **(C)** a b d e c f **(D)** d e f b c a. **Answer: (A).**

Q17 MSQ · 2 Which traversals pairs *uniquely* determine **every** binary tree? **(A)** preorder + inorder **(B)** preorder + postorder **(C)** postorder + inorder **(D)** preorder alone
Answer: A, C.

Q18 NAT · 2 A binary tree has `n` nodes and its inorder traversal is `1 2 3 … n`. How many such binary trees exist for n = 4? **Answer: 14** (Catalan; the inorder fixes the labels, only the shape varies).

Self-check

SELF-CHECK

- ☐ I know $L = I_2 + 1$ and can build $n = 2I_2 + I_1 + 1$.
- ☐ I can write height/size formulas for perfect and complete trees.
- ☐ I can rebuild a tree from Pre+In, Post+In, and Pre+Post (full tree).
- ☐ I know the count $^{2n}C_n/(n+1)$ and when to multiply by $n!$.

CHAPTER 10

Binary Search Trees (BST)

10

TIER 3 with AVL: 0.83 marks/paper

EXAM WEIGHT

Exam weight: BST + AVL together average 0.83 marks per paper, asked in 5 of the last 9 papers, including 2024 and 2025. In 2020 this cluster gave 4 marks.
Styles (2018–2026): build a BST from an insertion order or a traversal and ask for its height / post-order / shape (4 questions); **time complexity and properties of BST & AVL operations** (4 questions, mostly MCQ/MSQ).
Everything in the notes is here: definition, search/insert, the three deletion cases with examples, range search, and the "search sequence" trick with the counting example.

10.1 From scratch: why a BST?

If you keep numbers in a **sorted array**, search is fast (binary search, O(log n)) but insertion is slow (shifting, O(n)). In a **linked list**, insertion is fast but search is slow. A **BST** tries to give both: search and insertion in **O(height)**.

FORMULA

Binary Search Tree = a binary tree in which, for every node K, all values in K's left subtree are *less than* K, and all values in K's right subtree are *greater than* K. (notes)

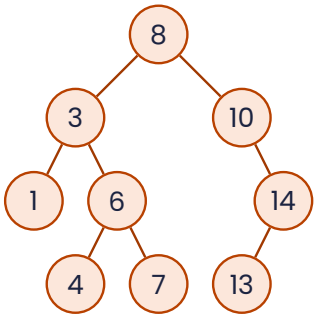


Figure 10.1 A binary search tree.

Check node 6: left subtree $\{4\} < 6$ ✓; right subtree $\{7\} > 6$ ✓. Check root 8: left subtree $\{3,1,6,4,7\}$ all < 8 ✓; right $\{10,14,13\}$ all > 8 ✓. The rule must hold at **every** node and for the **whole** subtree (not just the immediate children!).

FACT

Inorder traversal of a BST gives the elements in ascending sorted order. (notes) (Duplicates are not allowed unless the question says how to treat them.)

Here: inorder = `1 3 4 6 7 8 10 13 14`.

Handy facts

- **Minimum** = go left as far as possible. **Maximum** = go right as far as possible.
- **Inorder successor** of a node = the smallest key bigger than it (min of its right subtree, if it has one). **Inorder predecessor** = the largest key smaller than it (max of its left subtree).

10.2 Search and insertion

Search for K: start at the root. If `K` equals the node → found. If `K < node` → go left. Else go right. If you fall off the tree → not present.

```c
Node *search(Node *t, int k) {
    if (!t || t->key == k) return t;
```

```
    return k < t->key ? search(t->left, k) : search(t->right, k);
}
```

Insert K: search for K; when you fall off, attach a new node at that empty spot.

```c
Node *insert(Node *t, int k) {
    if (!t) return newNode(k);
    if (k < t->key) t->left  = insert(t->left,  k);
    else            t->right = insert(t->right, k);
    return t;
}
```

Complexity (notes)

KEY POINT

Searching in a BST takes **O(log n)** in the average case, **O(n)** in the worst case, **O(1)** in the best case.
Insertion takes **O(log n)** in the average case, **O(n)** in the worst case, **O(1)** in the best case.

Both depend on the **height h**: cost = O(h). A random BST has h ≈ log n. A **skewed** BST (built from sorted input 1, 2, 3, …, n) has $\mathbf{h = n - 1}$, so operations are O(n). That is exactly the problem AVL trees fix (Chapter 11).

Building a BST from a sequence — the standard exam task

Insert the keys **in the given order**, one by one, always starting from the root.

Worked example (verified). Insert `50, 30, 70, 20, 40, 60, 80, 35, 45, 65`.

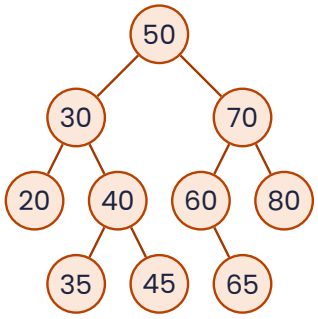


Figure 10.2 BST built by inserting 50, 30, 70, 20, 40, 60, 80, 35, 45, 65.

- Height = **3** (path 50 → 30 → 40 → 35).
- Inorder = `20 30 35 40 45 50 60 65 70 80`.
- Postorder = `20 35 45 40 30 65 60 80 70 50`.

Building from a traversal

- **From a preorder:** insert the elements **in the given preorder order** — the BST comes out uniquely. *(Preorder visits a parent before its children, so inserting in that order re-creates the same tree.)*
- **From a postorder:** insert the elements **in reverse postorder** (last element first).
- **From an inorder alone:** *not* unique (it is just the sorted list; every BST shape has the same inorder).
- **Inorder + any other traversal:** unique, as in Chapter 9 — but for a BST you never need the inorder, because it is simply the **sorted** keys.

Example (verified). Preorder `30 20 10 25 40 35 50` → insert in this order: root 30; 20 left; 10 left of 20; 25 right of 20; 40 right of 30; 35 left of 40; 50 right of 40. Tree `30(20(10,25),40(35,50))`. **Postorder = `10 25 20 35 50 40 30`.**

Example (verified). Preorder `8 3 1 6 4 7 10 14 13` → tree `8(3(1,6(4,7)),10(.,14(13,.)))`, height 3, postorder `1 4 7 6 3 13 14 10 8`.

Counting BSTs

KEY POINT

Number of structurally different BSTs on n distinct keys = Catalan(n) = ${}^{2n}C_n/(n+1)$.
n = 3 → **5** shapes (1 balanced + 4 of height 2); n = 4 → 14; n = 5 → 42.

KEY POINT

Number of different insertion orders that produce one *given* BST on n keys = n! / ∏ (size of subtree of every node).
Example: the perfect BST `4(2(1,3),6(5,7))` has subtree sizes 7, 3, 3, 1,1,1,1 → `7!/(7·3·3) = 5040/63 = 80` insertion orders (verified by brute force). A chain BST has sizes n, n−1, …, 1 → exactly **1** order. (This is the same arithmetic as "probability a random labelling is a heap", Chapter 12.)

10.3 Deletion — three cases (notes)

Depending on the number of children of the node to be deleted:

1. **0 children (leaf):** just delete it.
2. **1 child:** replace the node by its child, then delete it. (Splice it out.)
3. **2 children:** find either the **inorder predecessor** (largest in the left subtree) **or** the **inorder successor** (smallest in the right subtree) of the node, say `X`. **Remove X from its position and put X's value in the current node's place**, then delete the old node. (X has at most one child, so removing it is case 1 or 2.)

Notes' worked example. Start with

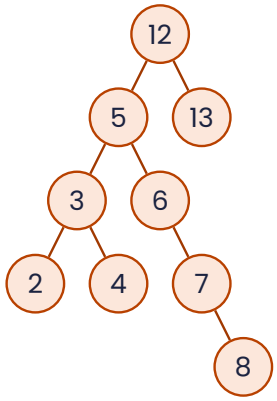


Figure 10.3 The notes' BST example.

`12(5(3(2,4),6(.,7(.,8))),13)`

Delete	Case	Result
2	leaf → simply deleted	`12(5(3(.,4),6(.,7(.,8))),13)`
6	one child (7) → replace 6 by its child	`12(5(3(2,4),7(.,8)),13)`
5 (using **inorder predecessor**)	two children. Predecessor = max of left subtree = **4**. Remove 4 from its place; put 4 where 5 was.	`12(4(3(2,.),6(.,7(.,8))),13)`
5 (using **inorder successor**)	successor = min of right subtree = **6** (6 has one child 7, spliced out).	`12(6(3(2,4),7(.,8)),13)`

(All four results verified by running code.)

KEY POINT

After deletion the tree is still a valid BST, but the shape depends on the choice (predecessor vs successor). GATE questions state which one to use, or the answer is independent of it.

Cost: O(h) (find the node + find the predecessor/successor).

10.4 Range search (notes)

NOTE

Given a range `[lo, hi]`, report all keys of the BST that lie in the range.

C

```c
void range(Node *t, int lo, int hi) {
    if (!t) return;
    if (lo < t->key) range(t->left,  lo, hi);   // left subtree may contain keys ≥ lo
    if (lo <= t->key && t->key <= hi) visit(t);
    if (t->key < hi) range(t->right, lo, hi);   // right subtree may contain keys ≤ hi
}
```

(The output comes out **sorted**.)

Cost analysis (notes):

- **Best case:** we only touch nodes inside the range — say **K** such nodes.
- **Worst case:** we may touch **at most 2 nodes per level** that are *outside* the range (one along each boundary path).
- So for a tree of height `h`, time is $\mathbf{O(2h + K) = O(h + K)}$; for a balanced tree `h = O(log n)`, so $\mathbf{O(\log n + K)}$.

Notes' examples: with range `[10, 15]` on a small tree only **2** out-of-range nodes were visited; with `[19, 21]` the nodes visited out of range (10, 12, 17, 27, 24, 22) were **at most 2 per level**, plus the K in-range nodes.

GATE PATTERN

"report K keys in a range in a balanced BST" $\Longrightarrow \Theta(\log n + K)$.

10.5 The "search sequence" trick (notes) — important for MSQ/NAT

KEY POINT

When searching for a node with value K in a BST, we may encounter nodes with values less than K or more than K — but all the encountered nodes with value *less than* K must appear in *increasing* order, and all with value *greater than* K must appear in *decreasing* order.

Why? Suppose while searching for `45` you meet `12`. Since `45 > 12` you go **right**; you can never meet a number smaller than 12 afterwards (they are in the left subtree you skipped). Meet `18` next: again go right, so everything later is > 18. Hence the "less than 45" values met form an **increasing** sequence. Symmetrically, the "greater than 45" values form a **decreasing** sequence. Each node met narrows the allowed interval `(lo, hi)`.

Notes' example 1. Searching for **55**, which sequence is possible?
(A) `90, 12, 68, 34, 62, 45, 55` **(B)** `9, 85, 47, 68, 43, 57, 55`

- (A): values > 55 are `90, 68, 62` — decreasing ✓. Values < 55 are `12, 34, 45` — increasing ✓. **Possible.**
- (B): values < 55 are `9, 47, 43` — not increasing (47 then 43) ✗. **Impossible.**

(Verified with an interval-narrowing check.)

Notes' example 2 (counting). While searching for **60** these nodes were encountered, *not in this order*: `10, 20, 40, 50, 70, 80, 90`. How many different search orders are possible?

- Less than 60: `10, 20, 40, 50` — must be **increasing** → one fixed order.
- Greater than 60: `70, 80, 90` — must be **decreasing** → one fixed order.
- The two groups can be **interleaved in any way**. Total positions 7; choose which 4 are the "small" ones: $\mathbf{7!\ /\ (4! \cdot 3!) = 35}$.

Another (verified by brute force): searching 45 where the nodes met are `20, 30, 40, 70, 60, 50`, followed by 45 itself: small group {20,30,40} (increasing) and large group {70,60,50} (decreasing) → `6!/(3!·3!) = 20` orders.

FORMULA TIP

if `a` encountered nodes are smaller than the key and `b` are larger, the number of possible search orders is $\mathbf{C(a+b,\ a)}$.

10.6 Other BST facts commonly tested

- **BST property check:** an inorder traversal must be strictly increasing. (Checking only parent–child pairs is *not* enough.)
- **Height of the BST** built from sorted input of n distinct keys: $\mathbf{n - 1}$. Maximum number of comparisons to search = h + 1.
- **Worst-case search/insert/delete = O(n)**; for a **balanced** BST = O(log n).
- **Two BSTs with the same set of keys can have different shapes**, but the **same inorder**.
- **Merging, sorting:** inserting n keys then doing inorder = sorting in O(n log n) average, $O(n^2)$ worst.
- **Preorder of a BST uniquely identifies it** (so does postorder).

10.7 Practice set — BST (GATE style)

Q1 NAT · 2 Insert `50, 30, 70, 20, 40, 60, 80, 35, 45, 65` into an empty BST. Height (edges)? **Answer: 3.**

Q2 MCQ · 2 Postorder of the BST obtained from the preorder `30 20 10 25 40 35 50` is
(A) 10 25 20 35 50 40 30 **(B)** 10 20 25 30 35 40 50 **(C)** 25 10 20 50 35 40 30 **(D)** 10 20 25 35 40 50 30
Answer: (A).

Q3 NAT · 2 Number of different binary search trees on the keys 1, 2, 3, 4, 5? **Answer: 42.**

Q4 NAT · 2 Number of BSTs with 3 keys whose height is exactly 2? **Answer: 4** (5 total minus the 1 balanced one).

Q5 NAT · 2 Keys 1…7 are inserted in some order to build the perfect BST `4(2(1,3),6(5,7))`. In how many different orders can this happen? **Answer: 80.**

Q6 MSQ · 2 In searching for **55** in a BST, which sequences of visited nodes are possible?
(A) 90, 12, 68, 34, 62, 45, 55 **(B)** 9, 85, 47, 68, 43, 57, 55 **(C)** 70, 30, 40, 50, 55 **(D)** 30, 60, 40, 50, 55
Answer: A, C, D. (C: >55: 70; <55: 30, 40, 50 increasing ✓. D: <55: 30, 40, 50 ✓; >55: 60 ✓. B fails.)

Q7 NAT · 2 While searching for 60, the nodes visited (in unknown order) are `10, 20, 40, 50, 70, 80, 90` and then 60. How many different visiting orders are possible? **Answer: 35.**

Q8 MCQ · 1 The inorder traversal of a BST is **(A)** always sorted ascending **(B)** always sorted descending **(C)** same as preorder **(D)** arbitrary. **Answer: (A).**

Q9 MCQ · 2 Delete 5 from the BST of §10.3 using the inorder **successor**. The new root's left child is
(A) 3 **(B)** 4 **(C)** 6 **(D)** 7 — **Answer: (C) 6.** (Tree becomes `12(6(3(2,4),7(.,8)),13)`; **the node replacing 5 is 6.**)

Q10 MCQ · 1 In a BST with n nodes built by inserting 1, 2, …, n in this order, the time to search for key n is
(A) O(1) **(B)** O(log n) **(C)** O(n) **(D)** $O(n^2)$ — **Answer: (C).**

Q11 MCQ · 2 In a balanced BST with n keys, the time to list all K keys in `[lo, hi]` is
(A) O(K) **(B)** O(log n) **(C)** O(log n + K) **(D)** O(n) — **Answer: (C).**

Q12 MSQ · 2 Which statements are true for a BST?
(A) Inorder traversal gives sorted order. **(B)** Preorder traversal alone determines the BST uniquely. **(C)** Inorder traversal alone determines the BST uniquely. **(D)** Deleting a node with two children can always be done by replacing it with its inorder successor.
Answer: A, B, D.

Q13 NAT · 2 Insert `8, 3, 10, 1, 6, 14, 4, 7, 13` into an empty BST. What is the sum of the keys at the **last level**? Level order: 8 | 3 10 | 1 6 14 | 4 7 13 → **4 + 7 + 13 = 24.**

Q14 MCQ · 1 A node of a BST has two children. Its inorder predecessor is
(A) the largest key in its left subtree **(B)** the smallest key in its right subtree **(C)** its parent **(D)** its left child
Answer: (A).

Q15 NAT · 2 A BST with 15 nodes is a perfect tree. What is the maximum number of nodes that can be examined during any one search? **Answer: 4** (height 3 ⟹ at most 4 nodes on a root-to-leaf path).

Self-check

SELF-CHECK

- ☐ I can build a BST from an insertion order and from a preorder/postorder.
- ☐ I can perform all three deletion cases and know pred vs succ.
- ☐ I can test a search sequence by narrowing the interval, and count orders with `C(a+b, a)` .
- ☐ I remember `O(h)`, `O(log n + K)` for range, Catalan for counting shapes.

CHAPTER 11

AVL Trees (Self-Balancing BST)

11

TIER 3

EXAM WEIGHT

Exam weight: shares the BST+AVL cluster (0.83 marks/paper, 5 of 9 years; 4 marks in 2020). Styles: properties and time complexity of AVL operations; insert/delete with rotations; min/max nodes for a given height (a favourite NAT).
Everything in the notes is here: balance factor, the four rotations, the 14-key insertion example, deletion rules and all deletion examples, the min/max-node recurrences, max height from n, and the count of minimal AVL trees.

11.1 Why AVL? (from scratch)

A BST can become **skewed** (all nodes on one side), e.g. when keys arrive sorted. Search, insertion and deletion all cost **O(h)**, so on a skewed tree (h = n − 1) they cost **O(n)**.

DEFINITION

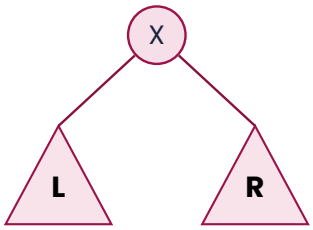
AVL tree = a BST in which, for **every** node, the heights of its left and right subtrees differ by **at most 1**.

Define the **balance factor** of node X:

FORMULA

$$\mathbf{BF(X) = height(left\ subtree\ of\ X) - height(right\ subtree\ of\ X) \in \{ -1, 0, +1 \}}$$

(with the height of an empty subtree = **−1**; a leaf has BF 0.)



$$\mathrm{BF}(X) = \mathrm{height}(L) - \mathrm{height}(R)$$

Figure 11.1 The balance factor of a node X.

DEFINITION

An AVL tree is a BST in which every node has balance factor between −1 and +1.

Because the height of an AVL tree with n nodes is **O(log n)**, all operations become **O(log n) even in the worst case**.

Searching: exactly as in a BST; thanks to the balance, the worst case becomes **O(log n)**.

11.2 Rotations — the repair tools

When an insertion/deletion makes some node's BF become **+2 or −2**, we repair with **rotations**. A rotation rearranges a few pointers, keeps the BST order, and lowers the height. Each rotation is **O(1)**.

Let **z** be the **lowest (deepest) unbalanced node**, **y** its child on the *heavy* side, **x** the relevant child/grandchild.

The 4 cases (named by the path from z down to the new node)

Case	Where is the extra height?	BF(z)	BF(y)	Fix
Left–Left (LL)	left child's **left** subtree	$+2$	$+1$	**one right rotation** at z
Right–Right (RR)	right child's **right** subtree	-2	-1	**one left rotation** at z
Left–Right (LR)	left child's **right** subtree	$+2$	-1	**left rotation at y, then right rotation at z**
Right–Left (RL)	right child's **left** subtree	-2	$+1$	**right rotation at y, then left rotation at z**

(1) Left–Left → right rotation

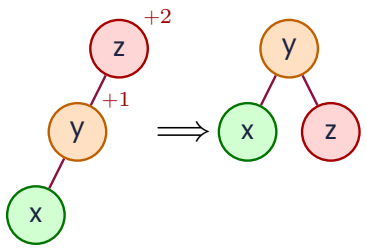


Figure 11.2 Left–Left case: a single right rotation about z.

y becomes the root; z becomes y's right child. (If y had a right subtree, it becomes z's left subtree.)

(2) Right–Right → left rotation

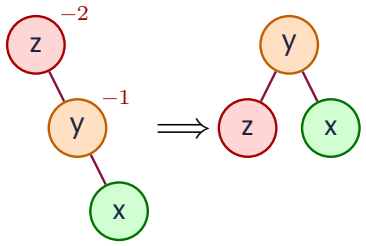


Figure 11.3 Right–Right case: a single left rotation about z.

(3) Left–Right → double rotation

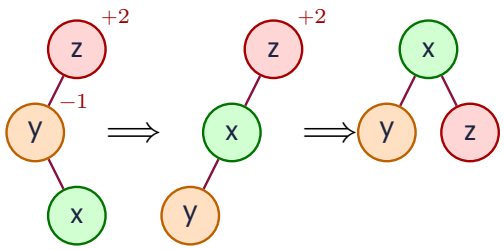


Figure 11.4 Left–Right case: rotate y left, then z right (double rotation).

First rotate y to the left (turning it into the LL shape), then rotate z to the right. The middle value x ends up as the root.

(4) Right–Left → double rotation

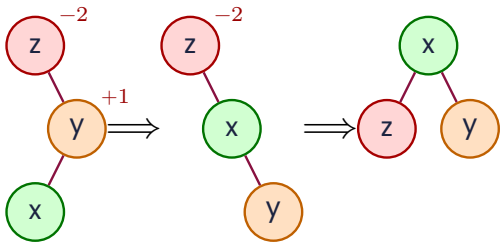


Figure 11.5 Right–Left case: rotate y right, then z left (double rotation).

KEY POINT

The simple rule (learn this, not the pictures): look at the **three nodes** z, y, x on the path. Whichever of the three is the **middle key** becomes the new root of that part; the smaller goes left, the larger goes right. This works for all four cases.

MEMORY AID

if z and y lean the **same** way (+2,+1 or −2,−1) → **single** rotation. If they lean **opposite** ways (+2,−1 or −2,+1) → **double** rotation.

11.3 Insertion

1. Insert as in a normal BST (new leaf).
2. Walk **back up** towards the root, updating heights/BFs.
3. At the **first** node whose BF becomes ±2, apply the matching rotation(s). **Stop.**

KEY POINT

A single insertion needs at most ONE rotation operation (a single or a double rotation) — for the first imbalance. i.e. **O(1) rotations.** (notes)

Time for insertion = search position `O(log n)` + insert `O(1)` + at most one rotation `O(1)` = **O(log n)**.

The notes' big worked example

Insert, in order: 10, 85, 15, 70, 20, 60, 30, 50, 65, 80, 90, 40, 5, 55 into an empty AVL tree.

Notation `k(L,R)`, `.` = empty. (Every line below was reproduced by running an AVL implementation; the final tree matches the notes.)

Step	Insert	What happens	Tree after the step
1	10	root	`10`
2	85	right child of 10; BF(10) = −1, fine	`10(.,85)`
3	15	goes left of 85. BF(10) = −2, child 85 has BF +1 → **RL** (double): 15 becomes root	`15(10, 85)`
4	70	left of 85; BF(85) = +1 fine	`15(10, 85(70,.) )`
5	20	left of 70; BF(85) = +2, child 70 BF +1 → **LL** at 85: single **right** rotation → 70 becomes subtree root	`15(10, 70(20, 85))`
6	60	left of 20; BF(15) = −2, right child 70 has BF +1 → **RL** at 15: 20 becomes root	`20(15( 10,.), 70(60, 85))`
7	30	left of 60; no imbalance (BF(70) = +1, BF(20) = −1)	`20(15( 10,.), 70(60( 30,.), 85))`
8	50	right of 30 under 60: BF(60) = +2, child 30 BF −1 → **LR** at 60: 50 becomes subtree root	`20(15( 10,.), 70(50( 30,60), 85))`

Step	Insert	What happens	Tree after the step
9	65	right of 60; BF(70) = +2, child 50 BF −1 → **LR** at 70: 60 becomes subtree root	`20(15(10,.),60(50(30,.),70(65,85)))`
10	80	left of 85 (under 70's right); BF(20) = −2, right child 60 BF −1 → **RR** at 20: single **left** rotation → 60 becomes root	`60(20(15(10,.),50(30,.)),70(65,85(80,.)))`
11	90	right of 85; balanced	`60(20(15(10,.),50(30,.)),70(65,85(80,90)))`
12	40	right of 30 under 50… BF(50) = +2 with child 30 BF −1 → **LR** at 50: 40 becomes subtree root	`60(20(15(10,.),40(30,50)),70(65,85(80,90)))`
13	5	left of 10: BF(15) = +2, child 10 BF +1 → **LL** at 15 → 10 becomes subtree root	`60(20(10(5,15),40(30,50)),70(65,85(80,90)))`
14	55	right of 50; balanced everywhere	`60(20(10(5,15),40(30,50(.,55))),70(65,85(80,90)))`

Final tree:

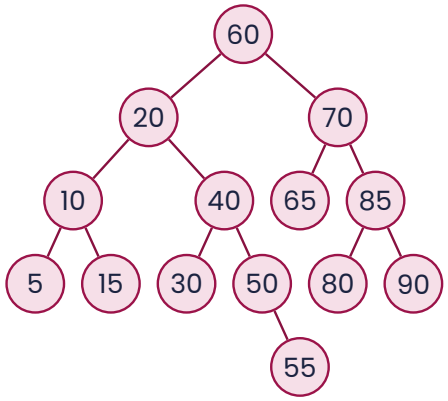


Figure 11.6 The final AVL tree after the insertions.

The order of rebalancing events was: RL, LL, RL, LR, LR, RR, LR, LL — **8 rebalancings** in 14 insertions.

EXAM TIP

after each insertion, check **only the nodes on the path from the new leaf to the root**, bottom-up; the *first* node with |BF| = 2 is where you rotate. Everything else stays untouched.

11.4 Deletion

FORMULA

AVL deletion = ordinary BST deletion + rebalancing. (notes)

1. Delete as in a BST (3 cases of Chapter 10).
2. Walk **back up** from the deleted position to the root, checking each node's BF.
3. At each node with |BF| = 2, rotate. **Unlike insertion, you must keep going up** — after a rotation the subtree may be 1 shorter than before, which can unbalance an ancestor.

KEY POINT

In case of deletion a single rotation may not be enough. In the worst case every level might need a rotation, i.e. O(log n) rotations. (notes)

Which rotation? Look at the heavy side's child

After a deletion at node X (the first unbalanced node), the *heavy* side is the side **opposite** to where the deletion happened. Let c = the heavy child.

X's BF	Heavy child c's BF	Rotation at X
+2 (left heavy)	+1	single **right** rotation (LL case)
+2	**0**	single **right** rotation (this case occurs **only in deletion**)
+2	−1	**double**: left rotation on c, then right rotation on X (LR)

X's BF	Heavy child c's BF	Rotation at X
−2 (right heavy)	−1	single **left** rotation (RR case)
−2	**0**	single **left** rotation (deletion only)
−2	+1	**double**: right rotation on c, then left rotation on X (RL)

The notes list it from the deleted side: "deleted node on X's **left** → look at BF of X→right: `+1` → double rotation (rotate X→right then X); `0` → single; `-1` → single" and mirror-image for a deletion on the right. (They call the cases **Right,−1 / Right,0 / Right,+1** meaning *(heavy side, child's BF)*.)

KEY DIFFERENCE

the heavy child can have BF **0**; then a **single** rotation is used, and the subtree's height **does not decrease**, so the process can **stop**.

The notes' deletion examples (all re-verified)

Example 1 — delete 1 from

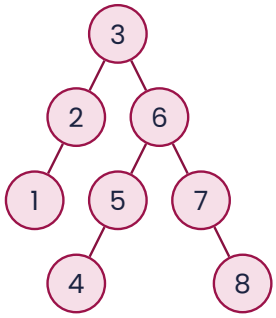


Figure 11.7 The BST before deleting 1.

`3(2(1,.),6(5(4,.),7(.,8)))`

- Remove 1. Node 3: left height 0, right height 2 → BF = −2.
- Heavy child 6: BF = 0 → **"Right, 0"** → single left rotation at 3.
- Result: `6(3(2,5(4,.)),7(.,8))`. Now 6 is balanced. Done.

Example 1b — then delete 8 from `6(3(2,5(4,.)),7(.,8))`:

- Node 7 simply becomes a leaf. Node 6: left height 2, right height 0 → BF = +2.
- Heavy child 3: its BF = −1 (right-heavy) → **left–right**: double rotation: rotate 3 left, then rotate 6 right.
- Result: `5(3(2,4),6(.,7))`. ✓

Example 2 — a deeper tree, delete 1
`8(3(2(1,.),5(4,6(.,7))),16(11(9(.,10),13(12,14)),18(17,19(.,20))))`

- Node 3 becomes −2; heavy child 5 has BF −1 → **"Right, −1"** → single left rotation at 3.

- Result: `8(5(3(2,4),6(.,7)),16(…))` — the left subtree keeps its old height (3), so **8 is balanced** and we stop.

Example 3 — delete 1 from
`8(3(2(1,.),5(4(3.5,.),6)),16(…))`

- Node 3 becomes −2; heavy child 5 has BF **+1** → **"Right, +1"** → **double** rotation (right rotation at 5, then left rotation at 3).
- Result: `8(4(3(2,3.5),5(.,6)),16(…))`. ✓ (4 becomes the root of that subtree.)

Example 4 — delete 1 from
`8(3(2(1,.),5(4(3.5,.),6(.,7))),16(…))`

- Node 3 becomes −2; heavy child 5 has BF **0** ("Right, 0") → single left rotation at 3.
- Result: `8(5(3(2,4(3.5,.)),6(.,7)),16(…))`. The subtree height is unchanged (3), so **8 is balanced — done.**

Example 5 — delete 4, then 2 from

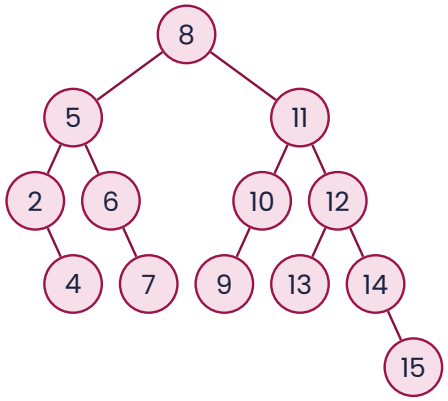


Figure 11.8 The BST before deleting 4 and then 2.

`8(5(2(.,4),6(.,7)),11(10(9,.),12(13,14(.,15))))`

- Delete 4: leaf. Node 2: BF 0; node 5: left height 0, right height 1 → BF −1 (fine); node 8: left height 2, right height 3 → −1 (fine). **No rotation.**
- Delete 2: node 5 now has BF −2; its right child 6 has BF −1 → **RR** → single left rotation at 5 → subtree root 6 with children 5, 7. The left subtree of 8 has dropped from height 2 to height 1, so **node 8 is now −2**; its right child 11 has BF −1 → **RR** again → single left rotation at 8.
- Final: `11(8(6(5,7),10(9,.)),12(13,14(.,15)))` — **two rotations were needed, one at two different levels.**

Another deletion needing a double rotation (verified)

`50(30(20,.),70(60(55,.),80))`: delete 20 → node 50 has BF −2, child 70 has BF +1 → **RL** → `60(50(30,55),70(.,80))`.

Time for deletion

search for the node **O(log n)** + delete **O(1)** + up to `h` rotations × O(1) = **O(log n)**. Overall: **O(log n)**.

Summary: insertion vs deletion

	Insertion	Deletion
After fixing the first imbalance	**stop** (height restored)	**continue upward**
Rotations needed	≤ 1 (single or double)	up to O(log n)
Heavy child BF = 0 possible?	no	yes (single rotation)
Time	O(log n)	O(log n)

11.5 How many nodes can an AVL tree of height h have?

Maximum nodes (notes)

The most nodes you can fit in height h is the **perfect binary tree**, which is itself an AVL tree:

FORMULA

$$\textbf{Max nodes} = 2\hat{}\{h+1\} - 1.$$

Derivation from balance factors: for the root there are three choices (BF = −1, 0, +1); BF = 0 gives both sides of height h−1, which maximises. So `N(h) = N(h-1) + N(h-1) + 1 = 2N(h-1) + 1` with `N(0) = 1`. Unrolling: `N(h) = 2^{h+1} - 1`.

*(Same fact asked the other way: "given a fixed number n of nodes, the **minimum height** is $\lceil \log_2(n+1) \rceil - 1$.")*

Minimum nodes (notes) — the Fibonacci-like recurrence

The *sparsest* AVL tree of height h has a root whose two subtrees have heights **h−1** and **h−2** (BF = ±1 — the least balanced allowed):

KEY POINT

N(h) = N(h−1) + N(h−2) + 1, N(0) = 1, N(1) = 2.

h	0	1	2	3	4	5	6	7	8
min nodes N(h)	1	2	4	7	12	20	33	54	88

(Closed form: `N(h) = F(h+3) - 1` with Fibonacci `F(1)=F(2)=1`. Check: `F(8) = 21`, `N(5) = 20` ✓.)

Maximum height for a given number of nodes (notes' "another framing")

Given n nodes, what is the maximum possible height of an AVL tree? Find the largest `h` with `N(h) ≤ n`.

Notes' example: n = 14. `N(4) = 12 ≤ 14` but `N(5) = 20 > 14` $\Longrightarrow$ **maximum height = 4.**

More (verified): n = 20 → 5; n = 32 → 5; n = 33 → 6; n = 54 → 7; n = 88 → 8.

NOTE

Asymptotically `h ≤ 1.44·log₂(n+2)` — an AVL tree is at most about 44 % taller than a perfect tree.

Number of minimal AVL trees of height h (notes)

Let `#min(h)` = number of *distinct* AVL trees of height h that have the **minimum** number `N(h)` of nodes.

- `#min(0) = 1`, `#min(1) = 2` (the two 2-node trees: left child or right child).
- For h ≥ 2: the root has one subtree of height h−1 and the other of height h−2, both minimal. Choose which side is taller: **2 ways**.

FORMULA

$$\#\text{min}(h) = 2 \cdot \#\text{min}(h-1) \cdot \#\text{min}(h-2)$$

h	0	1	2	3	4	5
#min(h)	1	2	4	**16**	**128**	**4096**

(h = 2: the four 4-node trees drawn in the notes ✓.)

11.6 Quick facts for MCQ/MSQ

- AVL ⊂ BST; inorder is still sorted.
- Height of an AVL tree with n nodes: Θ(log n). Min height $\lceil \log_2(n+1) \rceil - 1$, max ≈ 1.44 $\log_2$ n.
- Search, insert, delete: **O(log n)** worst case. (Plain BST: O(n) worst case.)
- Insertion: **at most one (single or double) rotation**. Deletion: up to **O(log n)** rotations.
- Extra storage: one BF (or height) per node.
- Inserting the sorted keys `1, 2, …, 2^k − 1` into an AVL tree gives a **perfect** tree (e.g., `1..7` → `4(2(1,3),6(5,7))`, 4 rebalancings).
- A **perfect** binary tree is always an AVL tree; a **complete** binary tree is always an AVL tree too (heights differ by ≤ 1). A full tree need not be AVL.
- Red-black trees (not in the notes) are a looser balanced BST: fewer rotations, height up to 2 log n.

11.7 Practice set — AVL (GATE style)

Q1 NAT · 2 The keys `21, 26, 30, 9, 4, 14, 28, 18, 15, 10, 2, 3, 7` are inserted one by one into an empty AVL tree. What is the key at the **root**? **Answer: 14.** (Final tree `14(4(3(2,.),9(7,10)),21(15(.,18),28(26,30)))`, height 3.)

Q2 NAT · 2 Keys `1, 2, 3, 4, 5, 6, 7` are inserted in this order into an empty AVL tree. What is the root, and what is the height? Give root + height. **Answer: 4 + 2 = 6.**

Q3 NAT · 1 Minimum number of nodes in an AVL tree of height 5? **Answer: 20.**

Q4 NAT · 2 An AVL tree contains 33 nodes. What is the maximum possible height? **Answer: 6** ($N(6) = 33$).

Q5 NAT · 1 An AVL tree contains 32 nodes. Maximum possible height? **Answer: 5.**

Q6 NAT · 2 How many different AVL trees of height 3 have the minimum number of nodes? **Answer: 16.**

Q7 MSQ · 2 Which statements about AVL trees are true?
(A) Insertion needs at most one rotation operation (single or double).
(B) Deletion may need more than one rotation.
(C) Worst-case search time is O(log n).
(D) A complete binary tree need not be an AVL tree.
Answer: A, B, C. (A complete tree has |height difference| ≤ 1 at every node, so it is always AVL; D is false.)

Q8 MCQ · 2 In an AVL tree, a node `z` has BF = +2, and its left child `y` has BF = −1. The required rebalancing is
(A) single right rotation at z **(B)** single left rotation at z **(C)** left rotation at y then right rotation at z **(D)** right rotation at y then left rotation at z
Answer: (C) — Left–Right case.

Q9 NAT · 2 AVL tree `50(30(20,.),70(60,80(.,90)))`. After deleting 20, what is the root key? **Answer: 70.** (Node 50 becomes −2; child 70 has BF −1 → single left rotation: `70(50(30,60),80(.,90))`.)

Q10 NAT · 2 In the AVL tree `50(30(20,.),70(60(55,.),80))`, delete 20. What is the root afterwards? **Answer: 60.** (RL double rotation.)

Q11 MCQ · 1 The BF of a leaf in an AVL tree is **(A)** 0 **(B)** 1 **(C)** −1 **(D)** undefined. **Answer: (A) 0.**

Q12 MCQ · 2 An AVL tree of height 4 can have at most how many nodes? **(A)** 15 **(B)** 16 **(C)** 31 **(D)** 32. **Answer: (C) 31** ($2^5 - 1$).

Q13 NAT · 2 Number of rebalancing operations (single or double rotations) when inserting `10, 85, 15, 70, 20, 60, 30, 50, 65, 80, 90, 40, 5, 55`? **Answer: 8.**

Q14 MCQ · 2 The worst-case time to insert n keys one by one into an initially empty AVL tree is **(A)** O(n) **(B)** O(n log n) **(C)** O(n^2) **(D)** O(log n) — **Answer: (B).**

Q15 MSQ · 1 After inserting a node into an AVL tree, a rotation is needed at node z. Which of these must hold?
(A) BF(z) is +2 or −2. **(B)** z is the deepest unbalanced node on the path to the new node. **(C)** After the rotation, the height of the subtree rooted at z's position is the same as before the

insertion. **(D)** All ancestors of z are now balanced.
Answer: A, B, C, D. (After a rotation the subtree regains its pre-insertion height, so ancestors' BFs return to their old values.)

Self-check

SELF-CHECK

- ☐ I can identify LL/RR/LR/RL from BF(z) and BF(y).
- ☐ I only recheck the path from the new node up to the root.
- ☐ I know insertion = ≤ 1 rotation; deletion = continue upward, and heavy child BF = 0 $\rightarrow$ single rotation.
- ☐ I can do `N(h) = N(h-1) + N(h-2) + 1` and find max height for n nodes.

CHAPTER 12

Heaps & Priority Queues

12

TIER 2 1.22 marks/paper

EXAM WEIGHT

Exam weight: 1.22 marks per paper — the 3rd-heaviest cluster. Asked in 7 of the last 9 papers (2018–2021, 2023–2025). In 2023 it gave 3 marks.
Styles (2018–2026):
- **Counting heaps, array positions of min/max, number of levels** — 4 questions (3 NAT, 1 MCQ; mostly 2-markers)
- **Heap operations** — is this array a heap? insert/delete cost, priority-queue implementation — 4 questions
- In the *algorithms* part: heap-sort tracing (2024), binary-heap statements (2026).

This chapter covers **every page of the notes on heaps**: definition, where min/max live, k-th minimum levels, insertion, build-heap (both ways, with full complexity proofs), heapify, find-min/delete-min, counting heaps, probability of being a heap, heap sort and priority queues.

12.1 From scratch: what is a heap?

You often need to repeatedly **find and remove the smallest (or largest) item** — a hospital queue where the most urgent patient goes first, a scheduler picking the highest-priority task. Sorting every time is wasteful. A **heap** keeps *just enough* order to give the smallest item instantly.

DEFINITION

A heap is a Complete Binary Tree which satisfies the heap property. (notes)

(Recall from Chapter 9: **complete** = all levels full except possibly the last, which is filled from the left.)

Heap type	Heap property
Min-heap	**every node's value is less than (or equal to) its children's values** → the root is the **minimum**
Max-heap	**every node's value is larger than (or equal to) its children's values** → the root is the **maximum**

Notes' examples (written as `k(left,right)`):

- `5(4(2,1),3)` — every parent ≥ children → **max-heap**.
- `1(2)` → **min-heap**.
- A tree that is **not complete** is **not a heap** at all (the notes' third example: "Not CBT").
- `1(2,3)` → **min-heap**.
- `2(1,3)` → **neither** (2 is bigger than 1 but smaller than 3), so it fails both properties.

IMPORTANT

a heap does **not** keep siblings ordered, and it is **not** a BST. In a min-heap `1(5,2)` is fine.

Heaps live in arrays

Because a heap is a CBT, it is stored compactly in an **array** with **no pointers** (Chapter 9):

- 1-based: children of `i` are `2i` and `2i+1`; parent is `⌊i/2⌋`.
- 0-based: children of `i` are `2i+1`, `2i+2`; parent is `⌊(i−1)/2⌋`.

To check whether an array is a heap: for every index `i ≥ 2` (1-based) verify `A[⌊i/2⌋] ≤ A[i]` (min-heap) or `≥` (max-heap).
Fun facts: a **sorted ascending** array is a valid **min-heap**; a **sorted descending** array is a valid **max-heap**.

Handy numbers for a heap with n elements

Quantity	Value
Height (edges)	$\lfloor \log_2 n \rfloor$
Number of leaves	$\lceil n/2 \rceil$ (they are at 1-based indices $\lfloor n/2 \rfloor + 1 \ldots n$)
Number of internal nodes	$\lfloor n/2 \rfloor$
Last internal (non-leaf) node, 1-based	index $\lfloor n/2 \rfloor$
Level (root = 0) of 1-based index i	$\lfloor \log_2 i \rfloor$

12.2 Where can the smallest and the largest elements be? (notes)

In a min-heap:

1. The **least element is always at the root.** (If there are duplicates of the minimum, they can also sit elsewhere, but one copy is at the root.)
2. The **maximum element is in some leaf node.** *Reason:* the max can't be smaller than any other value, and the min-heap rule says every node is smaller than its children — so a node with a child can't hold the strict maximum. **If there are duplicates** and the max sits in an internal node, then **every node in that subtree equals the max** (nothing larger may exist below it).

Example of a min-heap with a repeated maximum in an internal node:

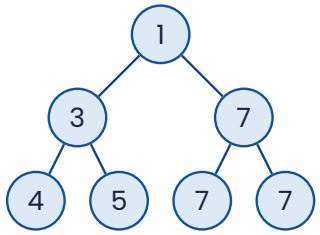


Figure 12.1 A min-heap with a repeated maximum in an internal node.

The maximum value 7 sits in the internal node on the right, and **every node under it is also 7** (a legal min-heap, because the rule is "$\leq$ children").

In a max-heap (mirror image):

1. The **root contains the maximum** (copies of the max may also appear elsewhere).
2. The **minimum is on a leaf.** With duplicates and the min in a non-leaf, its whole subtree holds only that min value.
3. The **k-th largest** element is on a level `l` with `0 < l < k` (for k $>$ 1) — it cannot be on level `k` or deeper. With duplicates, other levels are also possible along with `l`.

SHORTCUT

Shortcut — where can the minimum of a max-heap be? Among the **leaves**, i.e., 1-based indices $\lfloor n/2 \rfloor + 1$ … n. For n = 7 these are positions 4, 5, 6, 7 (confirmed by brute force). For n = 100: positions 51–100 (50 choices). This is the "array position of min/max" question style.
The second largest element of a max-heap is always a **child of the root**: position 2 or 3 $\rightarrow$ find it in O(1).

12.3 The k-th minimum in a min-heap (notes)

DEFINITION

The k-th minimum is the value such that there are k elements less than or equal to it.

Question: In a min-heap holding the distinct values 1, 2, 3, …, 100, on which levels can the value **5** be?

- Level 0 (root): impossible — the root must be the minimum, 1.
- Level 1: possible — 5 can be a child of the root.
- Level 2, 3, 4: possible — build a path `1 – 2 – 3 – 4 – 5` going down: 5 on level 4 is allowed, since 1, 2, 3, 4 are its four ancestors.
- Level 5: **impossible** — it would need 5 ancestors that are smaller, but only 4 values (1–4) are smaller than 5.

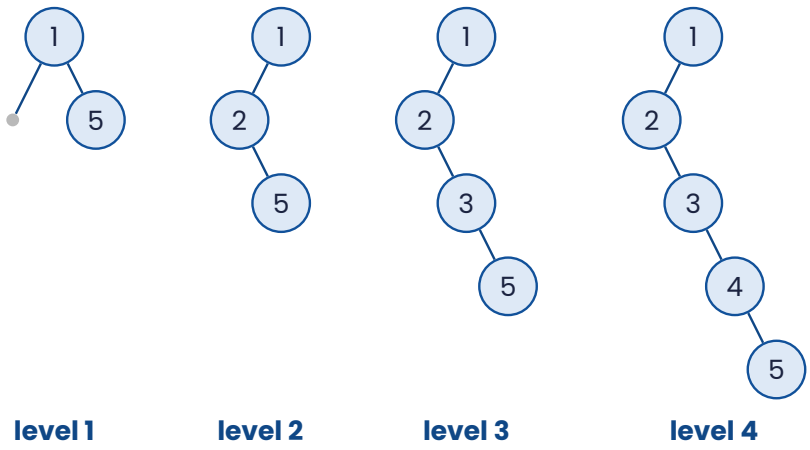


Figure 12.2 Placements of the value 5 at levels 1–4 of a min-heap.

RULE

Rule: in a min-heap the k-th minimum can be only at levels 1 … (k−1) (`1 ≤ l ≤ k-1`, i.e. `0 < l < k`, for `k > 1`). Each level it goes deeper needs one more smaller ancestor.

With duplicates (notes' example). A min-heap holds `1, 4, 5, 7, 7, 9, 10, …`. On which levels can the value **7** be?
7 is **both the 4th and the 5th minimum** (the two 7s take ranks 4 and 5). So a copy of 7 can be at level `l` with `1 ≤ l ≤ 5 – 1 = 4` — levels 1, 2, 3 or 4.

A famous consequence (notes' example)

KEY POINT

In a min-heap with n elements, the smallest element at the root. How long does it take to find the 7th smallest element?
(A) $\Theta(n \log n)$ **(B)** $\Theta(n)$ **(C)** $\Theta(\log n)$ **(D)** $\Theta(1)$

The 7th smallest lies on a level between 1 and 6. A heap's levels 1…6 contain at most `2^7 - 1 - 1 = 126` nodes (levels 0–6 hold $2^7-1 = 127$, minus the root). The answer is among these **constant** number of nodes, so searching them takes $\Theta(1)$. **ANSWER** **(D).**

PATTERN

General pattern: for a **constant k**, the k-th smallest is found by looking at only the first k levels — at most `2^k` nodes — so it is $\Theta(1)$. For $k \approx n/2$ it needs real work.

12.4 Insertion in a heap (notes)

Insertion happens in **two stages**:

1. **Add a new node according to the CBT property** — put it at the **next free position** (the last level, leftmost free spot, i.e., append to the array) and set its value.
2. **Compare the node with its ancestors repeatedly** (sift-up / bubble-up): while the new value is smaller than its parent (min-heap), **swap**.

Notes' worked example. Min-heap (array form `[1, 4, 3, 7, 5, 8, 9, 11, 9, 6]`):

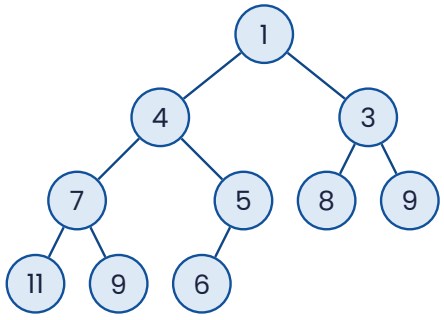


Figure 12.3 The min-heap $[1, 4, 3, 7, 5, 8, 9, 11, 9, 6]$ drawn as a complete binary tree.

Insert 2. Place it as the next leaf — the right child of 5 (array index 11).

- Compare with parent 5: $2 < 5 \rightarrow$ swap.
- Compare with parent 4: $2 < 4 \rightarrow$ swap.
- Compare with parent 1: $2 > 1 \rightarrow$ stop.
  Result array `[1, 2, 3, 7, 4, 8, 9, 11, 9, 6, 5]` (verified).

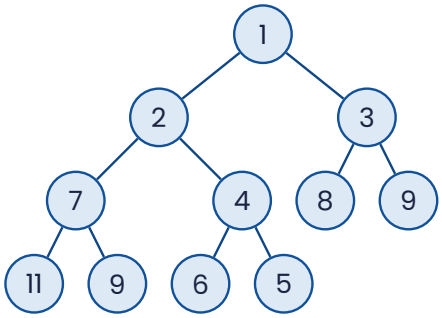


Figure 12.4 The heap after the operation: array $[1, 2, 3, 7, 4, 8, 9, 11, 9, 6, 5]$.

Time complexity of insertion

In the **worst case** the new leaf is compared all the way up to the root: that is the height, **O(log n)**.

- **Best case: O(1)** (the new value is already $\geq$ its parent).
- **Average case: O(1)** (for random values, most of the time it moves up only a constant number of levels).
- **Worst case: O(log n).**

Building a heap by repeated insertion (Top-down approach)

Notes' example. Build a **min-heap** from `1, 8, 6, 5, 3, 7, 4` by inserting one by one:

Insert	Array after
1	`[1]`
8	`[1, 8]`
6	`[1, 8, 6]`
5	`[1, 8, 6, 5]` $\rightarrow 5 <$ parent $8 \rightarrow$ swap $\rightarrow$ `[1, 5, 6, 8]`
3	`[1, 5, 6, 8, 3]` $\rightarrow 3 < 5 \rightarrow$ swap $\rightarrow$ `[1, 3, 6, 8, 5]`
7	`[1, 3, 6, 8, 5, 7]`
4	`[1, 3, 6, 8, 5, 7, 4]` $\rightarrow 4 < 6 \rightarrow$ swap $\rightarrow$ **`[1, 3, 4, 8, 5, 7, 6]`**

Final tree `1(3(8,5),4(7,6))`.

Time complexity of this method (notes):
In a heap of `n` nodes about **n/2 are leaves**, and each leaf insertion can cost `O(log n)`, so a crude bound is `(n/2)·log n = O(n log n)`.
Exact analysis: a node inserted at **level l** can move up at most **l** levels (l comparisons/swaps), and level `l` has at most `2^l` nodes:

$$T = 0\cdot 2^0 + 1\cdot 2^1 + 2\cdot 2^2 + 3\cdot 2^3 + \ldots + h\cdot 2^h$$

Let $S = \Sigma\, l\cdot 2^l$. Then $2S - S$ gives $S = 2^{h+1}(h - 1) + 2$. With $h = \log n$ (so $2^{h+1} \approx 2n$): $S \approx 2n(\log n - 1) + 2 = \Theta(n \log n)$.

DEFINITION

Building a heap by n insertions costs $\Theta(n \log n)$ (worst case). *This is the slower of the two ways to build a heap.*

12.5 Heapify (notes)

KEY POINT

The Heapify method takes a node and compares it with its descendants recursively to satisfy the heap property.
Heapify can be applied on a node X only if both its left and right subtrees are already heaps.

For a **min-heap**: if `X` is larger than its smaller child, swap them, and continue sifting down in the child's position; stop when `X` is $\leq$ both children (or it is a leaf).

C

```c
void heapify(int A[], int i, int n) {          // 0-based, min-heap
    int l = 2*i+1, r = 2*i+2, m = i;
    if (l < n && A[l] < A[m]) m = l;
    if (r < n && A[r] < A[m]) m = r;
    if (m != i) { swap(A[i], A[m]); heapify(A, m, n); }
}
```

Notes' example. Min-heap tree `19(18(43,23),21(26,29))` — apply `Heapify(19)` at the root:

- Children are 18 and 21; the smaller child is 18, and `18 < 19` → swap: 18 goes up, 19 goes down to the left position.
- Now compare 19 with its new children 43 and 23: `19 < 23 < 43`, so the property holds. **No need to apply again.**
  Result `[18, 19, 21, 43, 23, 26, 29]` (verified).

Time complexity: in the worst case the node travels from the root **all the way down to a leaf** = height $\Longrightarrow$ **O(log n)**. For a node at height `k` it is **O(k)**.

12.6 Build Heap — the bottom-up approach (Floyd's algorithm)

KEY POINT

Build Heap takes a bottom-up approach. Every leaf is already a heap (a single node). (notes)

1. **Create the CBT structure** from the given elements **blindly** (just put them in the array in the given order).
2. **Starting from the last non-leaf node up to the root, run Heapify on each node.**

Notes' worked example. Create a **min-heap** from `4, 1, 3, 2, 16, 9, 10, 14, 8, 7`.

Step 1 — array as a CBT (10 nodes): `[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]`

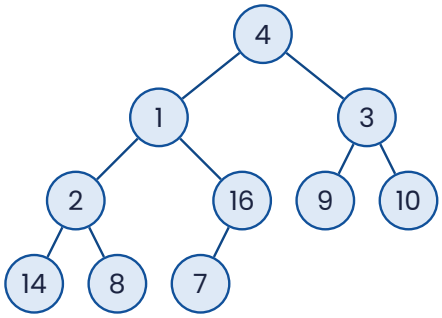


Figure 12.5 The array $[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$ viewed as a complete binary tree.

The last non-leaf is **16** (index 5 in 1-based; $\lfloor 10/2 \rfloor = 5$).

Step 2 — heapify from the last non-leaf backwards:

Heapify	Action	Array
`16` (idx 5)	child $7 < 16 \rightarrow$ swap	`[4,1,3,2,7, 9,10,14,8, 16]`
`2` (idx 4)	children 14, 8 are bigger $\rightarrow$ no change	same
`3` (idx 3)	children 9, 10 bigger $\rightarrow$ no change	same
`1` (idx 2)	children 2, 7 bigger $\rightarrow$ no change	same
`4` (idx 1)	smaller child $1 < 4 \rightarrow$ swap; then at new place children 2, 7: $2 < 4 \rightarrow$ swap; then children 14, 8: bigger $\rightarrow$ stop	**`[1, 2, 3, 4, 7, 9, 10, 14, 8, 16]`**

The result is a valid min-heap: `1(2(4(14,8),7(16)),3(9,10))` (verified).

Time complexity of Build Heap (notes): O(n)

Let the tree height be `h`. A node at **level l** has height `h - l`, and Heapify costs `O(height)`.

Level l	Number of nodes	Work per node (height)
0	2^0	h
1	2^1	h − 1
2	2^2	h − 2
…	…	…
h	2^h	0

`T = 2⁰·h + 2¹(h−1) + 2²(h−2) + … + 2^{h−1}·1 + 2^h·0`

Evaluating (by the usual `2S - S` trick, as in the notes): **`T = 2^{h+1} - 2 - h`**. With `h = log n`: `T ≈ 2n - log n - 2`, so

KEY POINT

Build-Heap (bottom-up) takes $\Theta(n)$ time.

Why is it linear while n insertions are n log n? In Floyd's method most nodes are near the bottom, where heapify is cheap (height ≈ 0 or 1); only the root pays the full `log n`. In the insertion method most nodes are near the bottom and each can travel *up* a long way.

Method	Time
Build heap by **n insertions** (top-down)	**Θ(n log n)**
Floyd's bottom-up build heap	**Θ(n)**

12.7 Find-Min and Delete-Min (min-heap)

KEY POINT

FindMin: the minimum is always at the root $\Longrightarrow$ just read it: **O(1)**.
DeleteMin takes 3 steps (notes):

Example (verified). Min-heap `[1, 3, 2, 6, 5, 4, 7]`:

- Delete-min #1: remove 1; move last element 7 to the root: `[7,3,2,6,5,4]`; sift: smaller child is 2 $\rightarrow$ swap $\rightarrow$ `[2,3,7,6,5,4]`; 7's children: 4 $\rightarrow$ swap $\rightarrow$ `[2,3,4,6,5,7]`.
- Delete-min #2: remove 2; move 7 to root: `[7,3,4,6,5]`; smaller child 3 $\rightarrow$ swap $\rightarrow$ `[3,7,4,6,5]`; 7's children: 6, 5 $\rightarrow$ 5 smaller $\rightarrow$ swap $\rightarrow$ **`[3,5,4,6,7]`**.

Other operations

- **Search for an arbitrary element:** O(n) (the heap gives no search structure).
- **Delete an arbitrary element given its index:** replace by last, then sift up or down: O(log n).
- **Decrease-key / increase-key (given index):** O(log n).
- **Find the maximum in a min-heap:** O(n) — it is in one of the $\lceil n/2 \rceil$ leaves.
- **Merge two heaps** of sizes m, n: O(m+n) by rebuilding.

Complexity table (notes)

Operation	Best	Average	Worst
Insertion	O(1)	O(1)	**O(log n)**
Deletion (delete-min)	O(1)	O(log n)	**O(log n)**
Find-min	O(1)	O(1)	**O(1)**
Build heap (Floyd)	–	–	**O(n)**
Build heap (n insertions)	–	–	**O(n log n)**

12.8 Counting heaps (a favourite NAT)

QUESTION

given `n` *distinct* numbers, in how many ways can they be arranged to form a min-heap (the CBT shape is fixed by `n`)?

Method (notes)

Take 5 numbers 1…5. The CBT shape for n = 5 has the root, a left subtree with 3 nodes, and a right subtree with 1 node.

- **Only one number can be at the root: the minimum** (it is forced).
- Of the remaining 4 values, **choose which 3 go to the left subtree**: $^4C_3$ ways; the other 1 goes right.
- The left and right subtrees must each be a heap on their chosen values: `L` ways for the left (3 nodes), `R` ways for the right (1 node).

KEY POINT

$\mathbf{H(n) = {}^{n-1}C_k \times H(k) \times H(n-1-k)}$, where `k` = the number of nodes in the **left** subtree of the CBT with `n` nodes.
Base cases: `H(0) = H(1) = 1`, `H(2) = 1`.

The shape determines `k`: with `h = ⌊log₂ n⌋` and `last = n − (2^h − 1)` nodes on the last level, `k = (2^{h−1} − 1) + min(last, 2^{h−1})`.

Worked examples (notes):

n	Computation	H(n)
1	–	1
2	–	1
3	$^2C_1 \cdot H(1) \cdot H(1)$	2
4	k = 2 → $^3C_2 \cdot H(2) \cdot H(1) = 3 \cdot 1 \cdot 1$	**3**
5	k = 3 → $^4C_3 \cdot H(3) \cdot H(1) = 4 \cdot 2 \cdot 1$	**8**
6	k = 3 → $^5C_3 \cdot H(3) \cdot H(2) = 10 \cdot 2 \cdot 1$	**20**
7	k = 3 → $^6C_3 \cdot H(3) \cdot H(3) = 20 \cdot 2 \cdot 2$	**80**
9	k = 5 → $^8C_5 \cdot H(5) \cdot H(3) = 56 \cdot 8 \cdot 2$	**896**

(The notes' n = 6 → **20** and n = 9 → **896** ✓. I verified H(n) against a brute force over all permutations for n ≤ 8: 1, 1, 2, 3, 8, 20, 80, 210. More values: H(10)=3360, H(11)=19200, H(12)=79200, H(15)=21964800.)

For n = 9 (notes), the shape has levels of sizes 1, 2, 4, 2:

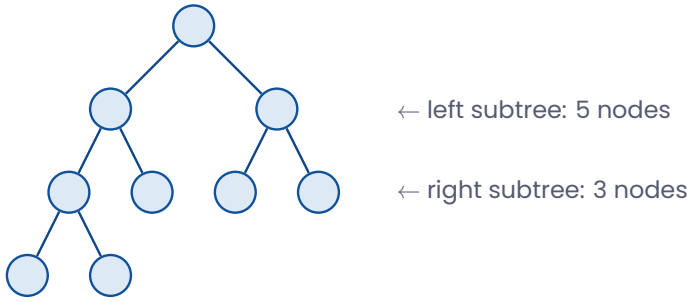


Figure 12.6 Shape of a complete binary tree with 9 nodes (levels of size 1, 2, 4, 2).

- Root = the minimum (forced). The other 8 values: choose which **5** go left: $^8C_5 = 56$.
- Left subtree (5 nodes) must be a heap: `H(5) = 8`. Right subtree (3 nodes): `H(3) = 2`.
- Total $56 \times 8 \times 2 = 896$. ✓

12.9 Probability that a structure is a heap (notes) — and the shortcut

FORMULA

P(a given tree is a heap) = (number of heaps possible)/n!, if the tree is a CBT; **0 if it is not a CBT.**

Notes' example. n = 5: `P = H(5)/5! = 8/120 = 1/15`.

The shortcut (the teacher's trick)

KEY POINT

P = ∏ over all nodes of 1/(size of that node's subtree).
Reason: for the root to hold the minimum of its subtree, the probability is `1/size`; the choices at different nodes are independent.

For the n = 5 shape: subtree sizes are 5 (root), 3 (left child), 1,1 (its leaves), 1 (right child): `P = 1/(5·3·1·1·1) = 1/15` ✓. Equivalently **H(n) = n! / ∏ sizes**: `120/15 = 8` ✓.

Notes' general-tree example (not necessarily binary)

NOTE

A tree with **13 nodes** is given (a root with 3 child subtrees). **What is the probability that every node receives the minimum value of its subtree**, when the labels `{1, …, 13}` are placed randomly?

Probability that a node gets the minimum of its subtree of size `s` = `1/s`. The non-leaf nodes had subtree sizes 13, 3, 6, 3, 3, so

$P = (1/13)\cdot(1/3)\cdot(1/6)\cdot(1/3)\cdot(1/3) = 1/13 \times 1/6 \times (1/3)^3 = 1/2106.$

(Leaves have probability 1.)

KEY POINT

Same arithmetic as counting BST insertion orders (Chapter 10): both are `n!/∏(subtree sizes)`.

12.10 Heap sort (notes)

Notes' version (with an output array):

```c
new_arr = [];
make_heap(arr);                    // O(n)
while (arr is not empty) {         // n iterations
    new_arr.insert(find_min(arr));    // O(1)
    delete_min(arr);                  // O(log n)
}
```

Time: `O(n) + n × O(log n) = O(n log n)`.

In-place version (what textbooks/GATE usually mean, uses a **max-heap**):

1. Build a max-heap in the array (O(n)).
2. Repeat n−1 times: **swap the root with the last element of the heap, shrink the heap by 1, heapify the root.**
   The array ends up **sorted ascending**.

	Heap sort
Time (best, average, worst)	**Θ(n log n)**
Extra space	O(1) (in place)
Stable?	**No**

12.11 Priority queue (notes)

KEY POINT

A priority queue ensures that every element popped off the queue has the highest priority of all current elements.

Choose an implementation by the costs of **insertion** and **deletion (of highest priority)**:

Implementation	Insertion	Deletion
Unsorted array / linked list	**O(1)**	**O(n)** (search for the best)
Sorted array / linked list	O(n)	**O(1)**
Balanced BST (AVL)	O(log n)	O(log n)
Binary heap	**O(log n)** (average O(1))	**O(log n)**

KEY POINT

On average a heap performs fewer operations than a balanced BST (notes), and uses simple arrays. That is why binary heaps are the standard priority queue.

Uses: Dijkstra / Prim (Algorithms), Huffman coding, event simulation, OS scheduling, heap sort, top-k problems.

12.12 Practice set — heaps (GATE style)

Q1 MCQ · 1 Which array is a **max-heap**?
(A) `10 8 9 4 5 7 6 1 2` **(B)** `10 8 9 4 5 7 6 1 12` **(C)** `1 2 3 4` **(D)** `5 7 6`
Answer: (A). (B: 12 is a child of 4 → violates; C is a min-heap; D: 7 > 5 violates.)

Q2 NAT · 2 A min-heap is built from `3, 9, 2, 1, 4, 5` using the **bottom-up (Floyd)** method. What is the array? Give the element at index 2 (1-based).
Answer: `[1, 3, 2, 9, 4, 5]` → element at index 2 is 3.
(Heapify idx 3 (value 2): child 5 → no; idx 2 (value 9): children 1, 4 → swap with 1 → `[3,1,2,9,4,5]`, then 9's child? it's now at idx 4 with no children → stop; idx 1 (value 3): children 1, 2 → swap with 1 → `[1,3,2,9,4,5]`; 3's children 9, 4 → fine.)

Q3 NAT · 2 Elements `10, 2, 5, 1, 7` are inserted one by one into an empty **min-heap**. The final array is? **Answer: `[1, 2, 5, 10, 7]`.**

Q4 NAT · 2 From the min-heap `[1, 3, 2, 6, 5, 4, 7]`, two delete-min operations are performed. The final array is? **Answer: `[3, 5, 4, 6, 7]`.**

Q5 NAT · 2 How many distinct min-heaps can be formed from the numbers 1 to 5 (the CBT shape being fixed)? **Answer: 8.**

Q6 NAT · 2 Same for 1 to 7. **Answer: 80.**

Q7 NAT · 2 The numbers 1–7 are placed in a perfect binary tree of 7 nodes in a uniformly random way. Probability that it is a min-heap? **Answer: 1/63** (`80/5040`; or `1/(7·3·3)`).

Q8 NAT · 2 A min-heap has `n = 127` elements (all distinct). The maximum element can be at how many different array positions? **Answer: 64** (the leaves, $\lceil 127/2 \rceil$).

Q9 MCQ · 2 The second largest element of a max-heap with $n \geq 3$ distinct elements can be found in
(A) O(1) **(B)** O(log n) **(C)** O(n) **(D)** O(n log n) — **Answer: (A).** It is one of the root's two children.

Q10 NAT · 2 In a min-heap of 100 **distinct** keys, the **5th smallest** key can be on how many different levels (root = level 0)? **Answer: 4** (levels 1, 2, 3, 4).

Q11 NAT · 2 In a min-heap with 1000 distinct elements, on how many levels can the **4th smallest** element be? **Answer: 3** (levels 1–3).

Q12 MCQ · 1 The time to build a heap from n arbitrary elements using the bottom-up method is
(A) $\Theta(n)$ **(B)** $\Theta(n \log n)$ **(C)** $\Theta(\log n)$ **(D)** $\Theta(n^2)$ — **Answer: (A).**

Q13 MCQ · 1 The time to build a heap by inserting n elements one at a time is **in the worst case**
(A) $\Theta(n)$ **(B)** $\Theta(n \log n)$ **(C)** $\Theta(n^2)$ **(D)** $\Theta(\log n)$. **Answer: (B).**

Q14 MCQ · 2 A priority queue is implemented as an **unsorted array**. Insertion and delete-max cost respectively
(A) O(1), O(n) **(B)** O(n), O(1) **(C)** O(log n), O(log n) **(D)** O(n), O(n) — **Answer: (A).**

Q15 MSQ · 2 Which statements about a binary **min-heap** with n distinct elements are true?
(A) The minimum is at the root. **(B)** The maximum is at a leaf. **(C)** Searching for an arbitrary element is O(log n). **(D)** Delete-min is O(log n).
Answer: A, B, D. (C is false: O(n).)

Q16 NAT · 1 A heap has 1000 elements. How many leaves does it have? **Answer: 500.** Height? **9.**

Q17 NAT · 2 In the array of a min-heap of 25 elements (1-based), what is the index of the **last internal node**? **Answer: 12.**

Q18 MCQ · 2 The heap property is **violated at only the root** of an otherwise valid min-heap. What is the cheapest way to repair it?
(A) Rebuild from scratch O(n) **(B)** **Heapify the root, O(log n)** **(C)** Sort the array O(n log n) **(D)** Insert a new element.
Answer: (B).

Q19 NAT · 2 A max-heap is built by inserting `5, 3, 8, 1, 9, 2` in this order. The final array is?
Answer: `[9, 8, 5, 1, 3, 2]`.

Q20 MSQ · 2 On the 3-level complete tree with 7 elements `1…7` stored as a max-heap, in which **array positions** can the value 1 be? **Answer: 4, 5, 6, 7** (the leaves).

Self-check

SELF-CHECK

- ☐ I can check a heap array in 10 seconds and know min/max positions.
- ☐ I can build a heap both ways and state **Θ(n)** vs **Θ(n log n)**.
- ☐ I can do `H(n) = ⁿ⁻¹C_k·H(k)·H(n−1−k)` and the `n!/∏sizes` shortcut.
- ☐ I know the k-th smallest is on levels `1 … k−1`.

CHAPTER 13

Hashing

13

TIER 2 0.67 marks/paper

EXAM WEIGHT

Exam weight (filed under Algorithms in your analysis): 0.67 marks per paper; asked in 6 of the last 9 papers (incl. 2025 and 2026).
Styles (2018–2026): insert keys into a hash table and give the probe sequence / final layout for linear probing, double hashing or chaining (5 questions); **simple uniform hashing — expected collisions** and behaviour under fixed hash functions (2 questions).
All the notes' content is here: hash function & division method, choosing m, simple uniform hashing, chaining, load factor, the average-case proofs, linear / quadratic / double hashing, clustering, deletion markers, number of probe sequences, and the open-addressing theorem.

13.1 From scratch: what problem does hashing solve?

To find a key in an **array** you scan (O(n)); in a **balanced BST** you need O(log n). Hashing aims for **O(1) on average** by *computing the address from the key itself.*

DEFINITION

Hashing is a method which uses a Hash Table data structure to perform searching in O(1). Hashing is suitable when the order between the data is not important. (notes)
On average, hashing can perform insertion, deletion and searching in O(1).

The vocabulary (notes)

Term	Meaning
Hash table	the actual structure (an array of `m` slots) in which data is stored
Hash function `h`	a function that maps a **key** to a **location (slot index)** in the table
Universe of keys `U`	all possible keys — e.g., all 10-digit phone numbers; all strings of any length
Actual keys	the subset of keys that actually get stored (may or may not be known in advance)
m	number of slots; **n** = number of keys stored

The universe is huge but the table is small, so **different keys can land in the same slot** — a **collision** (§13.4).

13.2 Hash functions

Division method (notes)

FORMULA

$$h(k) = k \bmod m$$

Example: `m = 10`. `k = 5025` → `h = 5`; `k = 100289` → `h = 9`.

How to pick m (notes)

- **If `m = 2^k`:** `h(k)` depends **only on the last k bits** of the key and ignores the rest. Example: `k = 2`, `m = 4`, key $(10101100101)_2$ → `h(key) = 01`. **Not good.**
- **Similarly `m = 10^k`** looks only at the **last k decimal digits** of the key.
- **Generally a prime number is used as m**, because it shares **no common factor with the keys** (so the keys spread out), preferably not close to a power of 2.

A good hash function (notes)

1. It should **consider every portion of the key**.
2. It should **spread out the values** over the hash table efficiently.
3. It should be **efficient to compute**.

Simple Uniform Hashing (SUH) — the idealised assumption (notes)

FORMULA

Every element is equally likely to hash into any of the m slots, independently of where any other element has hashed.
$P(h(k) = i) = 1/m$ **for every key k and every slot i.**

All the average-case results below assume SUH.

13.3 Collision resolution — the two families (notes)

Closed addressing / Open hashing	Open addressing / Closed hashing
Open chaining (separate chaining)	Linear probing
	Quadratic probing
	Double hashing
	Random hashing

NOTE

The two naming conventions ("closed addressing = open hashing") are confusing but both appear in books — remember **chaining = keys stay in a chain at their own address; open addressing = every key lives in the table itself, you probe for a free slot.**

13.4 Open chaining (separate chaining)

KEY POINT

If multiple keys map to the same location, append a chain (linked list) to that location and add the new element.

Notes' example. `h(k) = k mod 5`. Keys `9, 14, 19, 24` all map to slot **4**:

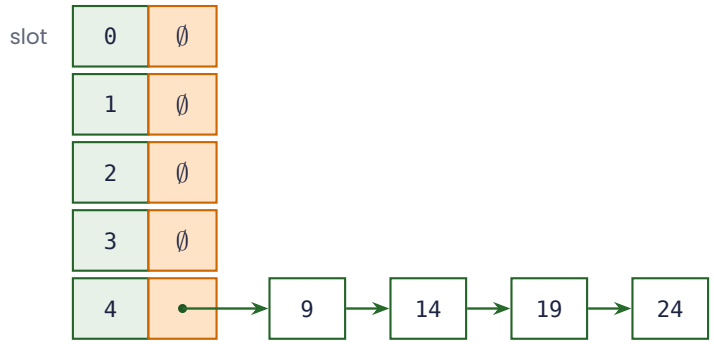


Figure 13.1 Hashing with chaining: keys 9, 14, 19, 24 all hash to slot 4.

- **Searching in the worst case is O(n)** (all keys in one chain), **but the average-case is still O(1).**
- **Load factor:** `α = n / m` (number of elements ÷ table size).
- With chaining, α **is the average number of elements in a chain**, and α **can be less than, equal to, or greater than 1.**

Average-case analysis of searching with chaining (assumption: SUH) — notes

THEOREM

Theorem 1. In a hash table with chaining, an **unsuccessful search** takes average-case time $\Theta(1 + \alpha)$.

Intuition/proof: computing `h(k)` takes $\Theta(1)$; then you must check **every element in the chain**, whose average length is $\alpha \Longrightarrow \Theta(\alpha)$. Total $\Theta(1 + \alpha)$.

- If `n` $\gg$ `m` ($\alpha \approx$ n): $\Theta(1 + n)$ — **not constant.**
- If `n` $\approx$ `m` ($\alpha \approx 1$): $\Theta(1 + \alpha) = \Theta(1)$ — constant.
- **Therefore we prefer `m = 0(n)`.**

THEOREM

Theorem 2. In a hash table with chaining, a **successful search** also takes $\Theta(1 + \alpha)$ under SUH.

Proof sketch (notes): When the i-th item was inserted, `i - 1` items were present; inserting it required an *unsuccessful* search in a table of `i-1` items (expected chain length `(i-1)/m`) and then the insertion itself, so the cost of searching it later is `1 + (i-1)/m`. Averaging over all `n` items:

`(1/n) Σ_{i=1}^{n} ( 1 + (i-1)/m ) = 1 + (n-1)/(2m) = Θ(1 + α).`

Chaining facts

- Deletion is easy (remove from the list). The table never "fills up" (α can exceed 1).
- Extra memory for pointers; performance degrades gracefully.
- Worst-case search O(n) (or O(log n) if chains are balanced trees).

13.5 Open addressing — probing

KEY POINT

There is no chain. All elements are stored in the hash table itself. Hence $n \leq m$, so $0 \leq \alpha \leq 1$.

The hash function now takes **two** arguments — the key and the **probe number i**:

KEY POINT

$$H : U \times \{0, 1, \ldots, m-1\} \to \{0, 1, \ldots, m-1\}$$

For a key `k` the **probe sequence** is `H(k,0), H(k,1), H(k,2), …, H(k,m-1)`. To insert, try the probes in order until you meet an empty slot. To search, follow the same probes until you find the key or reach an empty slot.

General form (notes)

FORMULA

$$H(k, i) = [\, h(k) + f(i) \,] \bmod m$$

Scheme	f(i)
Linear probing	$f(i) = i$
Quadratic probing	$f(i) = i^2$ (more generally $c_1 \cdot i + c_2 \cdot i^2$; usually $c_1 = 0$, $c_2 = 1$)
Double hashing	$f(i) = i \cdot h_2(k)$ (a second hash function)

13.6 Linear probing (notes)

`H(k, i) = (h(k) + i) mod m`. **Look at the next slot, then the next, ... wrapping around.**

- **Search:** continue at successive locations until the key is found or an **empty** location is found (then the key is absent).
- **Deletion:** search the element and **replace it with a marker** (tombstone). The marker means:
  1. the location is **empty for insertion** (a new key may reuse it);
  2. while **searching**, this location is **NOT an empty location** (keep probing past it). (If you simply emptied the slot, you would break the probe chains of other keys.)

Notes' worked example

`h(k) = k mod 10`, `H(k,i) = (h(k) + i) mod 10`. Table (10 slots):

slot	0	1	2	3	4	5	6	7	8	9
key	8	101							38	19

Insert K = 28. `h(28) = 8`.
`H(28,0) = 8` (occupied by 38), `H(28,1) = 9` (19), `H(28,2) = 0` (8), `H(28,3) = 1` (101), `H(28,4) = 2` (**empty**). **28 goes to slot 2** (verified).

Primary clustering (notes)

KEY POINT

When multiple keys map to the same location, small clusters start forming — called primary clusters. Linear probing suffers from primary clustering.

A long run of occupied slots (a *cluster*) makes any key that hashes into it probe to the end of the run, and the run grows — "the rich get richer".



Figure 13.2 Linear probing creates long runs of occupied slots (primary clusters).

13.7 Quadratic probing (notes)

`H(k, i) = h(k) + c₁·i + c₂·i²`, usually `c₁ = 0, c₂ = 1` ⟹ **`H(k,i) = (h(k) + i²) mod m`**.

- **Does NOT suffer from primary clustering** (the probes jump further and further).
- **It does suffer from secondary clustering:** two keys that map to the **same initial location have exactly the same probe sequence**.
- **It may fail to visit all slots.** Because of **cycle formation** the probe sequence may keep returning to the same few slots, so **empty locations can exist but never be found**. (notes' picture)

Verified examples of how many different slots the offsets `i² mod m` can reach (i = 0 … m−1):

m	7	8	10	11
distinct slots reached	4	**3**	6	6

For prime `m`, quadratic probing is guaranteed to find an empty slot **only while the table is less than half full** ($\alpha < ½$). For `m = 8` only offsets {0, 1, 4} ever occur — three slots out of eight.

13.8 Double hashing (notes)

FORMULA

$$\mathbf{H(k,\ i) = [\ h_1(k) + i \cdot h_2(k)\ ] \bmod m}$$

Uses **two hash functions**. The first probe is `h₁(k)`; the step size is `h₂(k)`, which **depends on the key**.

NOTE

Since `f(i)` depends on **k itself**, there are **no secondary clusters**: even if two different keys map to the same initial location, their probe sequences differ (different `h₂`).

Requirement: `h₂(k)` must be **non-zero** and **relatively prime to m** (e.g., `m` prime, `h₂(k) = 1 + (k mod (m−1))` or `R − (k mod R)` for a prime R < m) so that the probe sequence visits all slots.

13.9 Number of possible probe sequences (notes)

KEY POINT

How many different probe sequences can the scheme generate? (A measure of how "random" it is.)

Scheme	Possible probe sequences	Reason
Linear probing	**m**	the sequence is determined only by the **starting index** `h(k)` (e.g., 2,3,4,… or 5,6,7,8,9,0,1,…) — `ᵐC₁ = m`
Quadratic probing	**m**	again only the **starting index** is a choice
Double hashing	m^2	the pair `(h₁(k), h₂(k))` fixes everything: m choices for the start × m for the step
Uniform hashing (ideal)	**m!**	every permutation of the slots is equally likely

(In the notes: "once the values of h_1(k) and h_2(k) are fixed, the rest of the probe becomes constant, ∴ m × m = m^2".)

13.10 Analysis of open addressing (assumption: uniform hashing) — notes

Uniform hashing: if we choose a location and it is already filled, the next location is chosen **with equal probability among all locations**, so the number of possible probe permutations is **m!**. Here `n ≤ m`, so `0 ≤ α ≤ 1`.

THEOREM

Theorem. Given an open-addressing hash table with load factor $\alpha = \mathbf{n/m} < \mathbf{1}$, the **expected number of probes in an unsuccessful search is at most 1 / (1 − α)**, assuming uniform hashing.

Proof (notes).

- Currently filled slots = n $\Longrightarrow$ the probability that a probe hits a **filled** slot is `n/m = α`.
- So the probability of hitting an **empty** slot is `1 − α`.

Number of probes needed	Probability
1	`(1 − α)`
2	`α (1 − α)`
3	`α² (1 − α)`
…	…
m	`α^{m−1} (1 − α)`

Expected probes `E = 1·(1−α) + 2α(1−α) + 3α²(1−α) + … = (1−α)·[1 + 2α + 3α² + … ]`. Let `S = 1 + 2α + 3α² + ….` Then `S·(1−α)` telescopes (for `m ≈ ∞`) to `1 + α + α² + … = 1/(1−α)`, so `S = 1/(1−α)²` and

FORMULA

$$\mathbf{E = (1 - \alpha) \cdot 1/(1-\alpha)^2 = 1 \,/\, (1 - \alpha).}$$

Useful values:

α	0.5	0.75	0.9	0.99
Expected probes (unsuccessful) `1/(1−α)`	**2**	**4**	**10**	**100**

(Checked numerically.) The cost **explodes as** $\alpha \to \mathbf{1}$ — that is why open-addressing tables are resized when they get $\sim\frac{2}{3}$ full.

NOTE

the notes state that the cost of a successful search in an open-addressed table is the same bound, $\mathbf{1/(1 - \alpha)}$. (A sharper textbook bound is `(1/α)·ln(1/(1−α))`, which is smaller, so `1/(1−α)` is a valid upper bound for both.)

Chaining vs open addressing

	Chaining	Open addressing
α allowed	any (can be > 1)	$\leq \mathbf{1}$
Unsuccessful search (SUH)	$\Theta(1+\alpha)$	$\approx 1/(1-\alpha)$
Deletion	easy	needs tombstones
Memory	pointers extra	no pointers, cache-friendly
Clustering	none	primary/secondary clusters

13.11 More facts GATE uses (added)

Assume SUH, `n` keys, `m` slots.

- **Expected number of keys in a slot:** `n/m = α`.
- **Expected number of colliding pairs:** `C(n,2) · (1/m)` — each pair collides with probability `1/m`.
  Example: `n = 100`, `m = 1000` → `4950/1000 = 4.95`.
- **Probability that n keys land in n distinct slots:** `m(m−1)…(m−n+1) / mⁿ`. (n = 3, m = 10 → 0.72; the **birthday** case n = 23, m = 365 → 0.493.)
- **Expected number of empty slots:** `m·(1 − 1/m)ⁿ ≈ m·e^{−α}` (n = m = 100 → about 36.6).
- **Fixed hash function ⟹ adversary:** for any fixed function `h`, an adversary who knows it can pick many keys that all hash to one slot (possible because the universe is much larger than `m`), forcing $\Theta(\mathbf{n})$ per operation. *Universal hashing* picks `h` **randomly** from a family to defeat this.
- **Pigeonhole:** if the universe has more than `(n−1)·m` keys, any `h` has some slot receiving $\geq$ n keys.
- Linear probing with all keys hashing to the same slot: n inserts cost `1 + 2 + … + n = Θ(n²)`.

13.12 Worked layout examples (verified by simulation)

Example A — linear probing. `m = 10`, `h(k) = k mod 10`, insert `42, 23, 34, 52, 46, 33`:

- 42 → slot 2; 23 → 3; 34 → 4.
- 52 → h=2 (taken), 3 (taken), 4 (taken), **5** → 4 probes.
- 46 → 6.
- 33 → h=3, then 4, 5, 6, **7** → 5 probes.

slot	0	1	2	3	4	5	6	7	8	9
key			42	23	34	52	46	33		

Example B — quadratic probing, same keys, `H(k,i) = (k mod 10 + i²) mod 10`:

- 42 → 2; 23 → 3; 34 → 4.
- 52 → h=2 ✗, 2+1=3 ✗, 2+4=**6** ✓.
- 46 → h=6 ✗, 6+1=**7** ✓.
- 33 → 3 ✗, 4 ✗, 3+4=7 ✗, 3+9=12 mod 10 = 2 ✗, 3+16=19 mod 10 = **9** ✓.

slot	0	1	2	3	4	5	6	7	8	9
key			42	23	34		52	46		33

Example C — double hashing. `m = 11`, h_1`(k) = k mod 11`, h_2`(k) = 7 − (k mod 7)`, insert `10, 22, 31, 4, 15, 28, 17, 88, 59`:

key	h_1	h_2	probes (slots tried)	placed at
10	10	4	10	10
22	0	6	0	0
31	9	4	9	9
4	4	3	4	4
15	4	6	4, 10, **5**	5
28	6	7	6	6
17	6	4	6, 10, **3**	3
88	0	3	0, 3, 6, 9, **1**	1
59	4	4	4, **8**	8

Final table: slots 0..10 = `22, 88, –, 17, 4, 15, 28, –, 59, 31, 10`.

Example D — chaining. `m = 9`, `h(k) = k mod 9`, insert `5, 28, 19, 15, 20, 33, 12, 17, 10`: slot 1: 28 → 19 → 10; slot 2: 20; slot 3: 12; slot 5: 5; slot 6: 15 → 33; slot 8: 17. Longest chain: **3**.

13.13 Practice set — hashing (GATE style)

Q1 NAT · 2 Keys `42, 23, 34, 52, 46, 33` are inserted into a table of size 10 with `h(k) = k mod 10` and **linear probing**. In which slot does 33 end up? **Answer: 7.**

Q2 NAT · 2 Same keys, **quadratic probing** `H(k,i) = (h(k) + i²) mod 10`. In which slot does 33 end up? **Answer: 9.**

Q3 NAT · 2 Same keys, linear probing. What is the total number of probes (slot examinations) to insert all six keys? **Answer: 1+1+1+4+1+5 = 13.**

Q4 NAT · 2 With the double-hashing setup of Example C (`m = 11`, h_1 `= k mod 11`, h_2 `= 7 − k mod 7`), the key 88 is placed in which slot? **Answer: 1.**

Q5 NAT · 2 Chaining with `m = 9`, `h(k) = k mod 9`, keys `5, 28, 19, 15, 20, 33, 12, 17, 10`. Length of the longest chain? **Answer: 3.**

Q6 MCQ · 1 A hash table has 2000 slots and 1000 keys are stored with chaining under simple uniform hashing. Expected time of an unsuccessful search is Θ of
(A) 1 **(B)** log n **(C)** n **(D)** n log n — **Answer: (A).** `α = 0.5`; $\Theta(1+\alpha) = \Theta(1)$.

Q7 NAT · 2 An open-addressing table (uniform hashing) has load factor 0.9. Upper bound on the expected number of probes in an unsuccessful search? **Answer: 10.**

Q8 NAT · 2 100 keys are hashed into 1000 slots under simple uniform hashing. Expected number of **pairs** of keys that collide? **Answer: 4.95.**

Q9 NAT · 2 3 keys are hashed under SUH into 10 slots. Probability that **no two** keys collide? **Answer: 0.72.**

Q10 MSQ · 2 Which statements about hashing are true?
(A) Linear probing suffers from primary clustering. **(B)** Quadratic probing suffers from secondary clustering. **(C)** Double hashing has no secondary clustering. **(D)** Chaining requires $\alpha \leq 1$.
Answer: A, B, C. (D false: chaining works for $\alpha > 1$.)

Q11 MCQ · 1 With double hashing the number of distinct probe sequences is **(A)** m **(B)** m^2 **(C)** m! **(D)** 2^m. **Answer: (B).**

Q12 MCQ · 1 Which choice of m is **worst** for the division method `k mod m`? **(A)** 97 **(B)** 101 **(C)** 128 **(D)** 211. **Answer: (C)** — a power of two uses only the low 7 bits of the key.

Q13 MCQ · 2 In linear probing, a key is deleted by simply setting its slot to "empty". What can go wrong?
Answer: A later **search** for another key whose probe sequence passed through that slot would stop at the new empty slot and wrongly report "not found". Fix: use a **deletion marker** (tombstone) that insertion may reuse but search must skip past.

Q14 MCQ · 2 For `m = 8` and `H(k,i) = (h(k) + i²) mod 8`, how many different slots can a single key's probe sequence reach? **(A)** 3 **(B)** 4 **(C)** 6 **(D)** 8. **Answer: (A) 3** (offsets 0, 1, 4).

Q15 NAT · 2 Seven keys, all with the same hash value, are inserted into an empty linear-probing table. Total number of slot examinations? **Answer: 1+2+...+7 = 28.**

Q16 MCQ · 1 A hash table with chaining holds n keys in m slots. The **expected** length of a chain is **(A)** 1 **(B)** n **(C)** n/m **(D)** m/n. **Answer: (C).**

Q17 MSQ · 2 A fixed hash function is used and the keys come from a universe much larger than m. Which are true?
(A) There exists a set of n keys that all hash to the same slot. **(B)** Search can take $\Theta(n)$ in the worst case with chaining. **(C)** Choosing the hash function randomly from a universal family avoids this worst case on average. **(D)** Simple uniform hashing can always be guaranteed.
Answer: A, B, C. (D is an assumption, not a guarantee.)

Q18 MCQ · 1 Which collision-resolution scheme can have a load factor greater than 1? **Answer: chaining.**

Self-check

SELF-CHECK

- ☐ I can fill a table using linear, quadratic and double hashing and count probes.
- ☐ I know which scheme has which clustering and how many probe sequences.
- ☐ I know `θ(1+α)` for chaining and `1/(1−α)` for open addressing.
- ☐ I know why deleted slots need markers.

CHAPTER 14

Master Revision Sheet

14

read this the night before

NOTE

Everything you must be able to recall instantly, grouped by topic and ordered by exam weight.

14.1 C tracing (T1 · 2.11 marks) — traps checklist

Trap	Rule
Integer division	truncates toward 0: `-7/2 = -3`; `-7%3 = -1` (sign of left operand)
`float r = 7/2;`	integer division first → `3.0`
Wrap-around	k-bit unsigned: value mod 2^k; `char 120+10 → −126`; `unsigned 0−1 → 4294967295`
Signed vs unsigned compare	signed converts to unsigned: `3u > -5` is **false**
`&& ||`	short-circuit: the right side may never run
`a < b < c`	`(a<b) < c` → compares 0/1
`if (x = 0)`	assignment, not comparison
`for(...);`	stray `;` = empty body
`x & (x-1)`	clears lowest set bit; loop count = popcount
`x & -x`	isolates lowest set bit
`<<` / `>>`	$\times 2^n$ / $\div 2^n$
Macros	text substitution: `SQ(2+3) = 11`
`static` local	one copy, initialised once, persists
`switch`	falls through without `break`
`printf`	returns number of chars printed
Floats	`0.1+0.2 != 0.3`; `float` sums of `0.1f` $\times 10 \neq 1.0$f
Unsequenced `i++ + ++i`	undefined — question must specify order
`while(m--)`	runs m times; m = −1 afterwards

14.2 Pointers, arrays, structs (T1 · 1.28)

- `p+1` moves **one element** (`sizeof(*p)` bytes). `p2-p1` = number of **elements**.
- `a[i]` ≡ `*(a+i)` ≡ `*(i+a)` ≡ `i[a]`. `sizeof(array)` = whole; `sizeof(pointer)` = 8 (or 4).
- `*p++` → read then move pointer. `(*p)++` → read then increment value. `*++p` → move then read. `++*p` → increment value then read.
- C is **call by value**; pass `&x` to change `x`. Arrays decay to pointers.
- **RMO** address: `b + [(i-L1)·cols + (j-L2)]·sz`. **CMO**: `b + [(j-L2)·rows + (i-L1)]·sz`.
- 3-D RMO: `n1·N2·N3 + n2·N3 + n3`. CMO: `n3·N1·N2 + n2·N1 + n1`. RMO: **last** index fastest; CMO: **first** index fastest. RMO = N1 arrays of N2×N3; CMO = N3 arrays of N1×N2.
- Struct size: each member at a multiple of its size; total rounded up to the largest member alignment. `{char,int,char}=12`, `{char,char,int}=8`, `{char,double,short}=24`.
- String: `strlen` excludes `\0`; `sizeof(char[])` includes it; `'0'` is 48.

14.3 Recursion

- Draw the call tree. Print before the call → call order; print after → reverse; between two calls → inorder.
- Calls to `fib(n)` = `2·fib(n+1) - 1` (fib(6): 25 calls). `h(n)=h(n-1)+h(n-1)`: $2^{n+1}-1$ calls.
- `static` variable is shared by all calls. C: static (lexical) scope, call by value.

14.4 Asymptotics (T1 · 1.61)

GROWTH-RATE LADDER

$$1 < \log\log n < \log n < (\log n)^k < n^{\varepsilon} < n < n\log n < n^2 < n^3 < 2^n < n! < n^n < 2^{2^n}$$

- Big-O $\leq$, Ω $\geq$, Θ $=$, o $<$, ω $>$. `lim f/g = 0 → o`, `const → Θ`, `∞ → ω`.
- Log method: cancel common factors → ignore safe constants → take logs. Equal logs $\Longrightarrow$ **can't say**. Log base never matters.
- `2^{n+1} = O(2ⁿ)` ✓; `2^{2n} = O(2ⁿ)` ✗; `(n+k)^m = Θ(n^m)`.
- `√(log n)` grows faster than `log log n`. `n^{log n} < 2ⁿ`. `n! = o(nⁿ)`.
- **Loops:** `i*=2` → log n; `i*=i` (from 2) → log log n; `i*=2` outer with inner `j ≤ i` → 2n (**linear**); `j += i` for `i=1..n` → n log n; triangular → $n^2/2$; `for(i=1;i<n;i=i*i)` → infinite loop.
- **Master theorem** `T = aT(n/b) + f(n)`: compare `n^{log_b a}` with `f`. Tie with `logᵏ n` → extra `log n` (`Θ(n^{log_b a} log^{k+1} n)`).
  `2T(n/2)+n → n log n · T(n/2)+1 → log n · 4T(n/2)+n → n² · 8T(n/2)+n² → n³ · 3T(n/2)+n² → n² · 2T(n/2)+n log n → n log² n · 2T(n/2)+n/log n → n log log n · T(n-1)+n → n² · T(n-1)+T(n-2)+1 → exponential · T(√n)+1 → log log n · T(n/3)+T(2n/3)+n → n log n.`

14.5 Linked lists

- Singly (head only): insert/delete **front O(1)**; insert/delete **end O(n)**; access/search O(n).
- Circular with only `head`: insert/delete front is **O(n)** (must fix the last node). Keep a **last pointer** → front and end O(1).
- Doubly: delete given node pointer O(1); with head+tail, both ends O(1).
- Reverse: `next=h->next; h->next=prev; prev=h; h=next;` O(n) time, O(1) space.
- Middle/cycle: slow & fast pointers.
- Binary search on a linked list is still O(n).

14.6 Stacks & queues (T2 · 0.94)

- Valid stack outputs for n inputs = **Catalan** = $^{2n}C_n/(n+1)$: 1, 2, 5, 14, 42, 132, 429. Invalid = `n! - Catalan`.
- **Invalid iff it contains pattern high–low–middle ("312").**
- Array stack: `top=-1` empty; push O(1), pop O(1).
- Circular queue: full `(r+1)%cap == f`; empty `f == -1`.
- Queue via 2 stacks: enqueue O(1), dequeue O(n) worst (amortised O(1)); **4 stack ops per element**. Stack via 2 queues: push O(1), pop O(n).
- 2 stacks in 1 array: `t1=-1, t2=n`; full when `t1+1==t2`.
- Infix→postfix: operand→output; `(`→push; `)`→pop to `(`; operator: pop while top has higher/equal (left-assoc) precedence; `^` right-assoc. Postfix eval: for `- /`, first popped = **right** operand.
- Bracket matching: push openers, match closers with top, stack empty at end.

14.7 Binary trees (T2 · 0.83)

- Height in **edges** (single node = 0). Depth of root = 0.
- **$L = I_2 + 1$**. $n = 2I_2 + I_1 + 1$. Full: $n = 2L - 1$. K-ary full: $n = K \cdot I + 1$, $L = (K-1)I + 1$.
- Perfect: `n = 2^{h+1}-1`, leaves `2^h`. Complete tree of height h: $2^h \le n \le 2^{h+1}-1$.
- Complete tree of n nodes: height $\lfloor \log_2 n \rfloor$, leaves $\lceil n/2 \rceil$, internal $\lfloor n/2 \rfloor$.
- Any tree: min height $\lfloor \log_2 n \rfloor$, max $n-1$.
- Array (0-based): children `2i+1, 2i+2`, parent $\lfloor (i-1)/2 \rfloor$. (1-based: `2i, 2i+1`, $\lfloor i/2 \rfloor$.)
- Shapes with n nodes: Catalan; labelled: × n!.
- Traversal orders: Pre DLR, In LDR, Post LRD. Pre+In or Post+In determine the tree; **Pre+Post** determine a **full** tree (else `2^k`, k = nodes with one child); **any one** traversal determines a **complete** tree.

14.8 BST & AVL (T3 · 0.83)

- BST inorder = sorted. Search/insert/delete **O(h)**: O(log n) avg, O(n) worst.
- Preorder (insert in that order) or postorder (insert in reverse) rebuilds a BST uniquely; inorder alone does not.
- Deletion: 0 children → remove; 1 child → splice; 2 children → replace by inorder predecessor (max of left) or successor (min of right).
- Range search: **O(h + K)**.

- Search sequence for key K: keys $<$ K met must **increase**, keys $>$ K met must **decrease**; orders = `C(a+b, a)`.
- BSTs on n keys = Catalan. Insertion orders giving one BST = `n!/∏ subtree sizes`.
- **AVL:** BF = h(L)−h(R) ∈ {−1,0,1}. Cases: LL → right rotate; RR → left rotate; LR, RL → double. Same lean ⟹ single; opposite lean ⟹ double.
- Insertion: ≤ 1 rotation. Deletion: may cascade up (O(log n) rotations); heavy child BF 0 ⟹ single rotation.
- Min nodes: `N(h)=N(h−1)+N(h−2)+1`, `N(0)=1, N(1)=2`: 1, 2, 4, 7, 12, 20, 33, 54, 88. Max nodes `2^{h+1}−1`. Max height for n nodes: largest h with `N(h) ≤ n`.
- Minimal AVL trees of height h: `#min(h) = 2·#min(h−1)·#min(h−2)`: 1, 2, 4, 16, 128, 4096.

14.9 Heaps (T2 · 1.22)

- Heap = **complete** binary tree + heap property. Stored in an array.
- Min-heap: min at root; **max in a leaf** (leaves = indices $\lfloor n/2 \rfloor+1 \ldots n$). Max-heap: max at root; min at a leaf. 2nd largest of a max-heap is a child of the root.
- k-th smallest in a min-heap is on levels `1 … k−1`.
- Insert: append, sift up — **O(log n)** worst, O(1) average.
- Delete-min: last → root, sift down — **O(log n)**. Find-min O(1). Search O(n).
- **Build heap bottom-up: Θ(n)**; **by n insertions: Θ(n log n)**. Heap sort Θ(n log n), in place, not stable.
- Count of heaps: `H(n) = ⁿ⁻¹C_k · H(k) · H(n−1−k)` (k = left subtree size): 1, 1, 2, 3, 8, 20, 80, 210, 896(n=9). `P(heap) = ∏ 1/(subtree size)`.
- Priority queue: unsorted O(1)/O(n); sorted O(n)/O(1); heap O(log n)/O(log n).

14.10 Hashing (T2 · 0.67)

- Division method `k mod m`; avoid `m = 2ᵏ`, `10ᵏ`; use a prime.
- Chaining: α = n/m (can exceed 1); search $\Theta(1+\alpha)$ both successful and unsuccessful.
- Open addressing: $\alpha \leq 1$; unsuccessful search ≤ `1/(1−α)` ($\alpha = 0.5 \to 2$, $0.9 \to 10$).
- `H(k,i) = h(k) + f(i)`: linear `i` (primary clustering), quadratic `i²` (secondary clustering, may miss slots), double `i·h₂(k)` (no clustering).
- Probe sequences: linear `m`, quadratic `m`, double `m²`, uniform `m!`.
- Delete in open addressing ⟹ tombstone marker.
- SUH: expected colliding pairs `C(n,2)/m`.

14.11 Last-hour exam strategy

1. **Do the C-tracing and pointer questions first** — they are mechanical marks.
2. For counting questions, write the formula before the numbers.
3. For tree/BST/AVL/heap drawings, **draw every step** — rushing causes most errors.
4. For MSQ, test each option separately.
5. For NAT, re-check units (edges vs nodes, 0-based vs 1-based) before typing.

CHAPTER 15

Mixed Mock Test (28 questions) with Full Solutions

HOW TO USE

75 minutes, no notes. The mix follows the exam weights in Chapter 1 (C tracing and pointers first, then asymptotics, heaps, stacks/queues, trees, BST/AVL, hashing). Every answer was checked by running code or by exhaustive enumeration. After the test, go to the **real previous-year papers** and solve every question of the chapters where you lost marks. Marks are shown as (1) or (2). **Total = 45.**

Section A — Questions

M1 NAT · 1 Value printed?

```c
int x = 0, i;
for (i = 0; i < 5; i++) { if (i & 1) x += i; else x -= i; }
printf("%d", x);
```

M2 NAT · 1 On a machine with 32-bit `unsigned int`, how many times does the loop body execute?

```c
unsigned a = 1; int cnt = 0;
while (a) { a <<= 1; cnt++; }
printf("%d", cnt);
```

M3 NAT · 1 `int g(int n) { return n ? g(n/2) + n%2 : 0; }` — value of `g(100)`?

M4 NAT · 2 Value printed?

```c
int arr[5] = {1,2,3,4,5};
int *p = arr + 4; int s = 0;
while (p >= arr) { s = s*2 + *p; p--; }
printf("%d", s);
```

M5 NAT · 2 `int m[5][6]` is stored in **row-major** order starting at address 2000; each element takes 2 bytes. Address of `m[3][4]`?

M6 MCQ · 1 `sizeof(struct { char a; int b; short c; })` with usual alignment (int = 4, short = 2) is
(A) 7 **(B)** 8 **(C)** 12 **(D)** 16

M7 NAT · 2 `int fibc(int n){ calls++; if (n<=1) return n; return fibc(n-1)+fibc(n-2); }` — how many calls (including the first) are made by `fibc(5)`?

M8 MCQ · 2 `T(n) = 2T(n/2) + n²`, `T(1) = 1`. Then T(n) =
(A) $\Theta(n)$ **(B)** $\Theta(n \log n)$ **(C)** $\Theta(n^2)$ **(D)** $\Theta(n^2 \log n)$

M9 MCQ · 1 Time complexity of

```c
for (i = 1; i <= n; i *= 2)
    for (j = 1; j <= n; j++) stmt;
```

(A) $\Theta(n)$ **(B)** $\Theta(n \log n)$ **(C)** $\Theta(n^2)$ **(D)** $\Theta(\log n)$

M10 MCQ · 2 Which function grows **fastest**?
(A) n^{100} **(B)** 1.01^n **(C)** `n^{log n}` **(D)** `2^{√n}`

M11 MSQ · 1 For `f(n) = n log n` and `g(n) = n^{1.5}` which are true?
(A) `f = O(g)` **(B)** `f = Ω(g)` **(C)** `f = o(g)` **(D)** `g = O(f)`

M12 MCQ · 1 A **circular** singly linked list has only a pointer `last` to its **last** node. Inserting a new node at the **front** takes
(A) O(1) **(B)** O(log n) **(C)** O(n) **(D)** $O(n^2)$

M13 NAT · 2 Elements 1, 2, 3, 4 are pushed in this order (pops allowed at any time). How many of the following output sequences are valid?
`1 4 3 2`, `4 3 2 1`, `3 1 2 4`, `2 3 1 4`, `4 1 3 2`, `2 1 4 3`

M14 NAT · 1 Value of the postfix expression `9 3 / 2 4 * + 1 −`?

M15 NAT · 2 A circular array queue of capacity 4 uses the code of Chapter 8 (`f = r = -1` initially; full when `r != -1 && (r+1)%cap == f`). Operations: `enq a, enq b, enq c, deq, enq d, enq e, deq, enq f`. Report `f + r` at the end.

M16 NAT · 2 A complete binary tree has 200 nodes. Report (number of leaves) + (height in edges).

M17 MCQ · 2 A full binary tree has preorder `A B C D E F G` and postorder `C D B F G E A`. Its inorder is
(A) C B D A F E G **(B)** C D B A F G E **(C)** B C D A E F G **(D)** C B D A E G F

M18 NAT · 2 Keys `15, 10, 20, 8, 12, 17, 25, 6` are inserted into an empty BST. What is the **4th key** printed by a postorder traversal?

M19 NAT · 1 Minimum number of nodes in an AVL tree of height 6?

M20 NAT · 2 Keys `30, 20, 10, 25, 28, 27` are inserted into an empty AVL tree. Root key?

M21 NAT · 2 A **max-heap** is built from the array `[4, 10, 3, 5, 1]` using the bottom-up (Floyd) method. What is the element at index 2 (1-based)?

M22 NAT · 2 Number of distinct min-heaps that can be formed from 6 distinct numbers?

M23 NAT · 2 From the max-heap `[10, 5, 3, 4, 1]`, two delete-max operations are performed. Sum of the elements remaining?

M24 NAT · 2 Table size 8, `h(k) = k mod 8`, linear probing. Keys `18, 26, 34, 10, 42` are inserted in order. Total number of slot examinations?

M25 NAT · 1 Open addressing with uniform hashing, load factor 0.8. Upper bound on the expected probes of an unsuccessful search?

M26 MCQ · 1 In a table of size 8 with `H(k,i) = (h(k) + i²) mod 8`, how many distinct slots can one key's probe sequence reach? **(A)** 3 **(B)** 4 **(C)** 6 **(D)** 8

M27 MSQ · 2 Which statements are true?
(A) Building a heap of n elements by n successive insertions takes $\Theta(n \log n)$ in the worst case.
(B) The inorder traversal of any binary search tree is sorted.
(C) Deleting a node from an AVL tree needs at most one rotation.
(D) In a max-heap, the minimum element is always in a leaf.

M28 MCQ · 2 `T(n) = 3T(n/3) + n log n`. Then $T(n) =$
(A) $\Theta(n \log n)$ **(B)** $\Theta(n \log^2 n)$ **(C)** $\Theta(n^2)$ **(D)** $\Theta(n)$

#	Answer	Working

Section B — Answers with solutions

#	Answer	Working
M1	**−2**	i=0: −0; i=1: +1; i=2: −2 → −1; i=3: +3 → 2; i=4: −4 → **−2**
M2	**32**	one bit shifts out each time; after 32 shifts `a` becomes 0
M3	**3**	`g(n)` = number of 1-bits; 100 = 1100100_2 → 3
M4	**129**	p runs 5,4,3,2,1: s = 5; 14; 31; 64; 129 (Horner's rule)
M5	**2044**	$2000 + (3 \cdot 6 + 4) \cdot 2 = 2000 + 44$
M6	**(C) 12**	a@0, pad 3, b@4–7, c@8–9, pad 2 → 12
M7	**15**	`2·fib(6) − 1 = 2·8 − 1`
M8	**(C)**	a=b=2, watershed n; `f = n²` polynomially larger → $\Theta(n^2)$
M9	**(B)**	outer runs $\lfloor \log_2 n \rfloor + 1$ times, inner n times → n log n
M10	**(B)** 1.01^n	logs: A: 100 log n; B: 0.0144 · n; C: $\log^2$n; D: $\sqrt{n}$. Linear in n beats the others eventually
M11	**A, C**	`n log n / n^{1.5} = log n/√n → 0`, so f = o(g) ⟹ f = O(g); B and D false
M12	**(A)**	`last->next` is the first node: insert after `last` in O(1)
M13	**4**	valid: `1 4 3 2`, `4 3 2 1`, `2 3 1 4`, `2 1 4 3`; invalid: `3 1 2 4` (contains 3-1-2) and `4 1 3 2` (contains 4-1-3)
M14	**10**	9/3 = 3; 2 · 4 = 8; 3+8 = 11; 11−1 = 10
M15	**3**	a,b,c → f=0,r=2; deq → f=1; enq d → r=3; enq e → r=0 (check (3+1)%4=0≠ 1 ok); deq → f=2; enq f → r=1 ((0+1)%4=1≠ 2 ok). f=2, r=1 → 3. Queue holds c, d, e, f (full)
M16	**107**	leaves $\lceil 200/2 \rceil = 100$, height $\lfloor \log_2 200 \rfloor = 7$
M17	**(A)**	tree `A(B(C,D),E(F,G))`; inorder C B D A F E G
M18	**10**	tree `15(10(8(6,.),12),20(17,25))`; postorder `6 8 12 10 17 25 20 15` → 4th = 10
M19	**33**	N(6) = 33 (1, 2, 4, 7, 12, 20, 33)
M20	**25**	30,20,10 → LL → `20(10,30)`; 25 → `20(10,30(25,.))`; 28 → LR at 30 → `20(10,28(25,30))`; 27 → RL at 20 → `25(20(10,.),28(27,30))`. Root = 25
M21	**5**	heapify idx 2 (10): children 5,1 fine; idx 1 (4): swap with 10 → `[10,4,3,5,1]`; then 4's children 5,1 → swap with 5 → `[10,5,3,4,1]`. Index 2 → 5
M22	**20**	`⁵C₃ · H(3) · H(2) = 10·2·1`
M23	**8**	delete-max #1 → `[5,4,3,1]`; #2 → `[4,1,3]`; sum = 8
M24	**15**	all keys ≡ 2 (mod 8): probes 1, 2, 3, 4, 5 (slots 2,3,4,5,6) → 15
M25	**5**	`1/(1−0.8)`
M26	**(A) 3**	`i² mod 8 ∈ {0,1,4}`
M27	**A, B, D**	C false: deletion can cascade O(log n) rotations
M28	**(B)**	a=b=3 → watershed n; `f = n log n` ties with k=1 → $\Theta(n \log^2 n)$

Scoring guide (out of 45)

Score	What to do
38–45	Excellent. Only solve real PYQs now.
30–37	Good. Redo the chapters of the questions you missed.
20–29	Re-read Ch 3, 4, 6, 12 (the heaviest clusters) and redo their practice sets.
< 20	Go back to Chapter 2 and work through the book in order again, tracing every example on paper.

After the mock: your PYQ plan

1. Take the **PYQ list for each chapter**, solve them untimed, then timed.
2. For each wrong answer, write the **rule you missed** on a one-line card (the cheat sheet in Chapter 14 is built the same way).
3. Re-solve every wrong question after 3 days and after 10 days.